

W  
e  
b  
S  
y  
s  
t  
e  
m

アーキテクチャーからHTML5まで

# Web システム 入門

羽田野 太巳 著

SYSTEMS

## はじめに

---

Web といったら何を想像するだろうか。Web を仕事にしていない一般の人はホームページを想像するだろう。ネットワーク技術者なら HTTP といった通信プロトコルを指すかもしれない。サーバー側のシステム開発者なら Java や PHP などで開発する Web システムを指すだろう。そして、Web フロントエンド開発者なら HTML や JavaScript といったキーワードを想像するだろう。さらに、デザイナーであれば、CSS やユーザーインターフェース、ユーザー体験というキーワードを想像する場合もあるかもしれない。

このように、Web という用語は非常に範囲が広く、自身に関わる Web は人によって範囲が大きく違う。しかし、それらすべてが Web であり、すべてを組み合わせると初めてシステムが出来上がる。とはいえ、それぞれの分野で技術要素が異なるため、そのトレンドも異なる。

近年の大きなトレンドを挙げるとすれば、それは HTML5 であろう。この HTML5 という用語も非常にあいまいで、人によって想像する技術要素は異なる。HTML5 という用語は、主に Web フロントエンド開発全体に関わる。キーワードを挙げるとすれば、HTML、CSS、JavaScript、通信プロトコル、映像・音声コーデック、2D/3D グラフィックス、ストレージ、センサーなどだろう。これまでの Web フロントエンドでは縁がなかった用語が並ぶ。

一昔前の Web からは想像もできないことが Web ブラウザー上で実現できるようになってきたのだ。実は以前から Web ブラウザー上でさまざまな高度な機能が提供されていたが、それは Adobe Flash などのプラグインのおかげだ。しかし、近年は、スマートフォンではプラグインが利用できない。さらに、PC などのデスクトップ向け Web ブラウザーですらプラグインがデフォルトでは使えないようになってきた。言い換えると、これまでと同じことをやるにしても、もはや、HTML5 ベースでフロントエンドを開発する必要があるのだ。

HTML5 で高度なことができるようになったこともあり、近年は、Web ブラウザーだけでなく、スマートフォンのアプリにも応用されるようになってきた。さらに、組み込み機器にまで HTML5 が進出している。テレビのデータ放送の最新バージョンであるハイブリッドキャストでは、表示されるコンテンツは HTML/CSS/JavaScript で作られている。その他、デジタルサイネージやカーナビなどへの応用も検討されてる。

このように、一般消費者向けサービスにおいては、すでに HTML5 は当たり前のよう採用されている。一方、企業システムは、ライフサイクルが長いという事情や、受託開発が多く、アジャイル開発を実践しにくいという事情もあり、新しい技術の採用が難しい環境にあるだろう。そのため、一般消費者向けサービスと比べて、HTML5 の採用は進んでいないのが実情だ。企業システムの開発に関わる ICT 従事者の方であれば、HTML5 を静的な Web ページに使ったことはあっても、Web アプリケーションという意味で実際に採用したことがないという方も多いのではないだろうか。

HTML5 によってフロントエンドでできることが高度になるにつれ、フロントエンドの開発ではアーキテクチャーも大きく変わってきている。サーバー側では当たり前のよう採用されている MVC モデルをフロントエンドにも採用する動きだ。そして、この流れの中で、フロントエンド（Web ブラウザー側）とサーバー側の役割すら変わってきている。

しかし、これまで HTML/CSS/JavaScript に関わりがなかった ICT 従事者にとって、HTML5 のトレンドや要素技術を俯瞰するのは難しい。それほど、HTML5 は範囲が広く、そして、そのトレンドによって Web 事情が大きく変わってしまったのだ。企業向けシステム開発の従事者といえども、今後は一昔前の Web の知識では通用しないだろう。とりわけ、受託でシステムを開発するシステムインテグレーションを主な業務にしている会社であれば、場合によっては顧客のほうがこの HTML5 のトレンドに敏感な場合すらあるだろう。

本書は、そういった ICT 従事者のために、手っ取り早く Web のトレンドを俯瞰してもらえるよう企画したものだ。前半ではフロントエンドのアーキテクチャーのトレンドを紹介する。後半では HTML5 で新たに利用できるようになった機能を紹介する。本書によって、Web 技術で何ができるのか、Web の最前線はどこに向かおうとしているのか、が一通り把握できる。

皆様が上手に最新の Web 技術を取り入れ、付加価値が高いシステムを企画するうえで、本書が役に立てれば幸いだ。

有限会社 futomi 代表取締役  
株式会社ニューフォリア 取締役 CTO  
羽田野 太巳

# 目次

---

|            |   |
|------------|---|
| はじめに ..... | 2 |
|------------|---|

## 第 1 章 Web システムの基本を押える .....

|                            |    |
|----------------------------|----|
| 1-1 Web 技術の進化 .....        | 8  |
| 1-2 アーキテクチャーの変化 .....      | 20 |
| 1-3 MVC ライブラリの活用 .....     | 32 |
| 1-4 モバイルアプリ開発 .....        | 44 |
| 1-5 企業システムを変える HTML5 ..... | 57 |
| 1-6 HTML5 の未来 .....        | 70 |

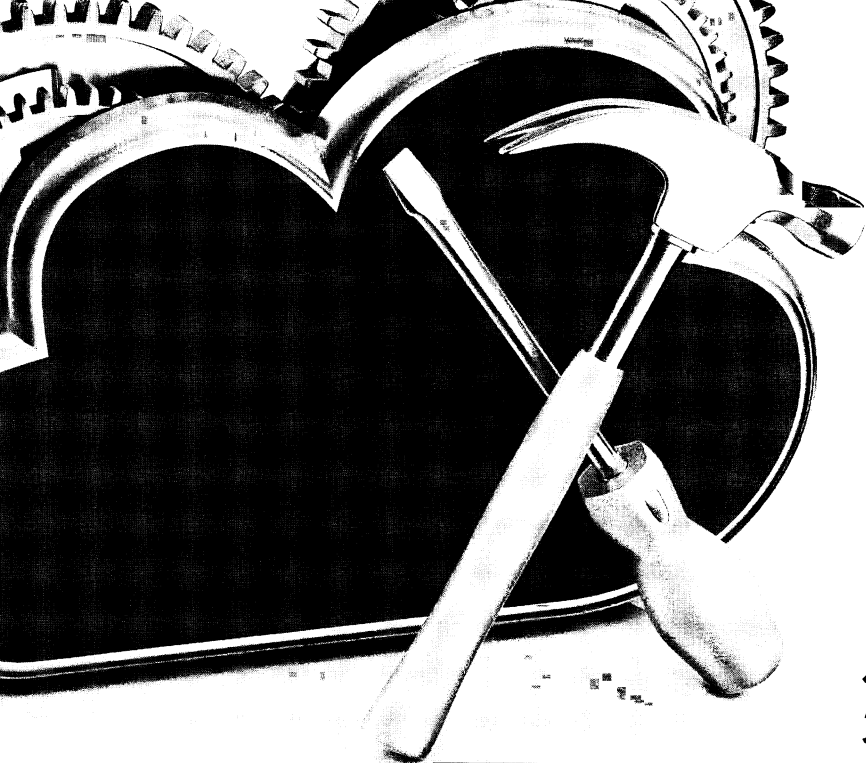
## 第 2 章 HTML5 の基本を学ぶ .....

|                          |    |
|--------------------------|----|
| 2-1 マークアップとスタイリング .....  | 84 |
| 2-2 入力とユーザーインタフェース ..... | 97 |



|                        |     |
|------------------------|-----|
| 2-3 マルチメディア.....       | 110 |
| 2-4 ハードウェアへのアクセス.....  | 123 |
| 2-5 ネットワーク .....       | 136 |
| 2-6 標準化提案中の API .....  | 149 |
| 索引 .....               | 162 |
| 日経 SYSTEMS の主要書籍 ..... | 166 |

本書は「日経 SYSTEMS」2015 年 4 月号～2015 年 9 月号に掲載した「こんなに変わった Web 開発」と、2015 年 10 月号～2016 年 3 月号に掲載した「HTML5 の基本テクニック」の各連載に加筆・修正をしたものです。本書の内容は 2016 年 6 月時点のものです



# 第 1 章

## Web システムの基本を押える

|                            |    |
|----------------------------|----|
| 1-1 Web 技術の進化 .....        | 8  |
| 1-2 アーキテクチャーの変化 .....      | 20 |
| 1-3 MVC ライブラリの活用 .....     | 32 |
| 1-4 モバイルアプリ開発 .....        | 44 |
| 1-5 企業システムを変える HTML5 ..... | 57 |
| 1-6 HTML5 の未来 .....        | 70 |

## 1-1 Web 技術の進化

# 標準技術のHTML5に注目 つなぐ技術としても活用進む

Web 技術は大きく進化を遂げ、さまざまな業界や分野で利用されるようになった。企業向けシステム開発でも、こうした Web 技術のトレンドを押さえておく必要がある。

Web 技術の進化の流れと、活用例を解説する。

筆者が仕事として Web にかかわり始めてから 15 年経つが、その様相は大きく変化してきた。特にこの 5 年ほどは、Web 技術そのものが進化し、これまで Web 技術だけではできなかったことができるようになったり、これまで Web と関係が薄かった業界や分野で Web 技術が応用されたりするなどして、筆者の仕事内容も大きく変わった。

あまりにも短期間に Web 技術や適用領域が変化したため、開発現場は混乱していることだろう。そこで本書では、Web 技術を活用した開発トレンドを紹介する。海外の事情も踏まえ、Web 技術が今どう進化しているのか、そして今後はどうなっていくのかを見ていく。

まずは、HTML5 をはじめとした Web 技術の進化の流れと、さまざまな業界への広がりを解説する。HTML5 が生まれた理由や、なぜ今、HTML5 が注目されているのかを理解することで、近年の Web のトレンドも理解しやすくなるはずだ。

HTML5 を押さえておくことは、企業向けシステム開発では大きな意味を持つ。理由は大きく二つある。一つは、端末や OS の多様化への対応だ。OS ごとの調整は必要となるが、コアとなる部分が同じ言語（HTML5）で開発できれば、開発コストを低減するだけでなく、今後新たに登場するデバイスにも対応しやすくなる。

もう一つは、HTML5 が標準技術であることだ。企業独自の技術は長く続くとは限らない。あれほど長きにわたって繁栄したプラグイン技術（Adobe Flash や Microsoft Silverlight）を見れば分かるだろう。一方、Web 技術のような標準技術

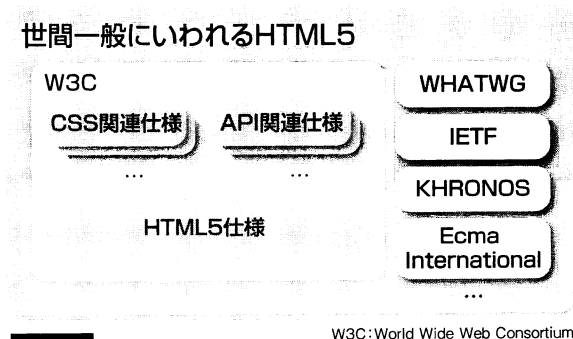
は、少なくとも Web ブラウザーベンダーが消滅しない限り使い続けられる。とりわけ、長きにわたって利用される企業システムにとっては重要な側面ではないだろうか。

## 曖昧な HTML5 という用語

「HTML5 とは何なのかを簡単に説明してほしい」と言ったら、恐らく人によって言い方が異なるだろう。HTML5 とは本来、Web 標準化団体である W3C (World Wide Web Consortium) が策定した仕様の名称だ (図 1-1)。本書では、この仕様を HTML5 仕様と呼ぶ。

HTML5 仕様は、HTML4.01 の後継であり、主に HTML マークアップに使われる要素を定義している。その他、JavaScript から利用できるいくつかの API (Application Programming Interface) も定義されているが、それは主に、HTML 要素に関連した API だ。想像している HTML5 と比べると範囲が狭いと多くの人は感じるだろう。

一方で、HTML5 仕様以外の標準仕様も含めて、HTML5 という言葉が使われることが多い。とりわけ、JavaScript から利用できる API の仕様や、スタイリングを定義する CSS (Cascading Style Sheets) に関連する仕様は、ここ数年で



**図 1-1** HTML5 の範囲

HTML5 は本来、Web 標準化団体である W3C が策定した仕様の名前のこと。しかし世間一般では、HTML5 仕様以外の Web 標準の仕様全体を指して、HTML5 という言葉が使われることもある

大幅に拡充され、Web ブラウザーにも実装されてきた。そのため、これら Web 標準技術全体の進化の象徴として、HTML5 という言葉が世間一般で使われている。実際には、W3C 以外の標準化団体が作成した仕様まで含んでいる場合が多いが、HTML5 という言葉が示す範囲は文脈次第で変わる。そういう意味では、非常に曖昧な用語として使われているといえる。

Web コンテンツを制作する上では、HTML5 仕様で定義された内容だけでは、ほとんど何も作れない。必要に応じて、その他の API や CSS を駆使しながらコンテンツを作らざるを得ない。従って、開発の現場では、世間一般にいわれる HTML5 を見ていく必要がある。

## 長らく進化がなかった Web 技術

W3C で HTML4.01 が勧告になったのは 1999 年だ。そして、HTML4.01 を XML の文法と適合させた XHTML1.0 が勧告になったのは 2000 年。誕生から 15 年も経過しているが、今でも HTML4.01 や XHTML1.0 が使われたシステムはよく見かける。

インターネットの世界は、使い古された言葉を使えばドッグイヤーといわれるほど変化が早いはずなのだが、Web 技術だけを見ると、15 年以上前に作られた仕組みが長く居座り続けたのだ。サーバー側の仕組みは多くの進化があったものの、クライアント側の HTML、CSS、JavaScript、通信プロトコルに関してはあまり変化がなかった。IT の世界で見れば、稀有な存在だ。

とはいえ、クライアント側も大きく進化してきたとを感じるだろう。そこには、Adobe Flash や Microsoft Silverlight といったプラグインが大きく寄与している。YouTube のようなビデオ配信や、3D 技術を駆使したゲームコンテンツを Web ブラウザー上で楽しめたのは、こういったプラグインのおかげだ。

プラグイン全盛期は 10 年以上も続き、何かしら Web ブラウザー上でリッチなコンテンツを表示したければ、プラグインによる開発は避けて通れなかった。Web コンテンツの開発といえばプラグイン向けコンテンツ開発を指し、多くのデザイナーや開発者が Adobe Flash や Microsoft Silverlight の技術習得にいそしんだ。当時、この繁栄の終焉<sup>しゅうえん</sup>を予測したものは誰もいなかっただろう。

## HTML5 躍進の三つの転機

Web 業界で HTML5 が大きく注目されるようになった転機は三つある。

最初の転機は、2007 年に米 Apple が iPhone を発表し、販売したときだ（日本では 2008 年に販売開始）。当初から iPhone は売れ続け、Web 制作の現場では iPhone に対応したサイトの開発が必須となった。

ところが、iPhone が搭載する Web ブラウザーである Safari には、Flash も Silverlight もインストールされていない。さらにユーザーは、それらのインストールもできない。開発者は、Flash の代わりに HTML5 を使うことを余儀なくされた。Web 制作の現場が大きく混乱したのは、記憶に新しい。

二つめの転機は、2009 年に米 Google が開発者向けイベント Google I/O で HTML5 に注力すると発表し、Chrome に数多くの HTML5 関連の API を実装したときだ。このイベントではさまざまなデモが披露され、多くの開発者が HTML5 に興味を持ったのは間違いない。しかし、企業で広く利用されている Internet Explorer (IE) には実装されなかったため、開発者の技術的興味の域を出ることはなく、なかなかシステム開発に生かされなかった。

HTML5 が注目されるようになった決定打といえるのが、三つめの転機だ。2009 年、米 Microsoft が開発者向けイベント MIX'09 で、HTML5 に注力すると発表した。リリースされた IE9 では、JavaScript エンジンゼロから作り直すという力の入れようだった。

これですべての主要 Web ブラウザーベンダーが HTML5 をサポートした。とはいえ、IE の最新版がリリースされてもすぐには世の中に浸透しない。いまだに企業では古い IE が残り続けているところも多いが、サポート期間が切れるに従って、徐々に IE の最新版の導入率は上がってきている。

## 端末のアプリ開発でも HTML5 を活用

今、最も HTML5 が使われているのは、スマートフォン向け Web コンテンツだろう。スマートフォンは、当初から HTML5 の機能を実装した Web ブラウザーが組み込まれていたからだ。

近年は「モバイルファースト」という言葉が出てきている。これは、コンテンツの企画から制作において、パソコン向けからではなく、スマートフォンを優先、または先行して進める、という考え方だ。企業向けシステム開発といえども、システムの種類によっては、モバイルファーストの考え方が必要な場合も出てきている。

HTML5 などの Web 技術が使われるようになったとはいえ、端末側で Web ブラウザーを起動していちいちアドレスを入力し、サービスを利用するのはためらわれる。このような場合、各 OS 向けに開発したネイティブアプリであれば、アドレスの入力を省ける。

では、ネイティブアプリには Web 技術は関係ないのだろうか。もし世の中に OS が 1 種類しかないなら、そうだろう。しかし、実際には多種多様な OS を搭載したスマートフォンが存在する。国内では Android と iOS だけといってもよい状況だが、世界的には、Windows Phone や Blackberry、Tizen などのさまざまな OS が使われている。地域によっては、それらの OS が無視できないほどのシェアを持つ場合がある。OS が多様化すると、ネイティブアプリも OS ごとに開発する必要がある。もちろん、プログラミング言語が違うため、同じアプリを作るためにゼロから OS の数だけ開発する必要があり、アプリ開発コストは大きくなってしまう。

そこでアプリ開発者から期待されたのが HTML5 である。すべての機能を HTML5 だけで実現するのは無理にせよ、多くの部分を HTML5 ベースで開発すれば、他の OS へ流用しやすくなる。このような開発手法は一般的に、ハイブリッドアプリ開発と呼ばれる。実際はネイティブアプリとしてアプリストアで配布されるため、利用者からは HTML5 ベースだとは分らない。すでに、ハイブリッドアプリとして開発された有名なアプリは多数ある。

ハイブリッドアプリを開発するために必要な環境やクラウドサービスも多数存在する。世界的に有名なのは、オープンソースソフトウェア (OSS) の Apache

---

**注1-1) Apache Cordova**

<http://cordova.apache.org/>

---

**注1-2) アシアル「Monaca」**

<https://ja.monaca.io/>

---

**注1-3) ニューフォリア「アプリカン」**

<http://www.applican.com/>

---

Cordova<sup>注1-1</sup>だ。このCordovaを使ってクラウドでアプリをビルドするサービスを、米 Adobe Systems が PhoneGapBuild として提供している。国内では、アシアルが Monaca<sup>注1-2</sup>というサービスを、ニューフォリアがアプリカン<sup>注1-3</sup>というサービスを提供している。Monaca では、Cordova を使い Web ブラウザーベースでアプリを開発できる。アプリカンは、Cordova を使わずに独自に基盤を開発して API を拡張したものだ。

いずれも Web アプリケーションを作る環境さえあれば、ネイティブアプリに必要な開発環境をインストールすることなくアプリを開発できる。アプリのビルドはすべてクラウド側でやってくれるのだ。

## リアルタイムに情報を更新できる

HTML5 以外の Web 技術にも目を向けよう。

今では、当たり前のように Facebook や Twitter などでは友達の投稿をリアルタイムで見られる。このような SNS のタイムラインなど、Web コンテンツをリアルタイムに共有する仕組みは「リアルタイム Web」と呼ばれている。Google などが提供しているリアルタイム検索もリアルタイム Web の一つだ。ちなみに、リアルタイム Web という言葉は、技術用語ではない。ここでいうリアルタイムとは、リアルタイム OS のようなシビアな話ではなく、あくまでもサービスの側面で使われる。

Web アクセスで一般的に利用されている HTTP (Hyper Text Transfer Protocol) は、オーバーヘッドが大きく、リアルタイム性を得意としない。そこで、リアルタイム表示を実現するために「WebSocket」が新たに開発された。WebSocket を使うと、クライアントが一度サーバーと接続すれば、その後は常時接続状態となる。個々のメッセージ送信では、可能な限りオーバーヘッドを減らし、遅延を低減する。

一方で HTTP 自体も進化しつつある。Google は HTTP を高速化した「SPDY」と呼ばれる通信プロトコルを開発。SPDY はデータを暗号化してやり取りする TLS (Transport Layer Security) をベースに、一つのセッションで複数のリクエストを送信できる仕組みを備える。このためサーバーは、一つのクライアント



から多くのリクエストを効率よく受け付けられる。

SPDY は Chrome だけでなく、Firefox、IE11、Safari、Opera などの Web ブラウザーが実装した。また、Web サーバー側も、Jetty、Node.js、Apache (mod\_spdy)、Nginx といった主要なサーバーソフトウェアが SPDY をサポート。Google のみならず、Facebook、Twitter、Flickr など多くのユーザーを抱えるサービスも SPDY への対応を進めた。

インターネット関連技術の標準化団体である IETF (Internet Engineering Task Force) は、SPDY をベースに、HTTP の次期バージョンの HTTP/2 を策定し、2015 年 2 月に標準化が完了したところだ。主要な Web ブラウザーがすでに HTTP/2 を実装しており、評価もできる。Google は 2016 年 5 月に SPDY のサポートを終了し、HTTP/2 のサポートを本格化した。

このように、Web のリアルタイム性はますます重要になり、それに合わせてサーバー側の対応も必要になってくる。旧来のように、まとめてバッチ処理を行って、1 日に一回だけ更新するような悠長なことをしていただける時代ではなくなってきている。一般消費者向けサービスがリアルタイム Web を当然のように取り入れている今、企業システムでも同様なことが求められてくるだろう。

## サーバー側とクライアント側は疎結合に

端末側が高度化するにつれて、Web システムのアーキテクチャーも大きく変わってきた。

旧来の Web のアーキテクチャーといえば、クライアントがリクエストを送り、サーバーが HTML を生成して、クライアントがそれを表示するといった仕組みだった。クライアントは単に静的な Web ページを表示するだけだ。ユーザーが何か操作するたびにページが置き換わるのは非効率だが、以前はクライアント側でできることが限られていたため、サーバー側で HTML まで生成する必要性があった。

しかし近年は、端末のパフォーマンスが向上し、Web ブラウザーに実装された API が大きく拡充したこともあり、アーキテクチャーが変わってきている。最近の代表的なアーキテクチャーは、SPA (Single Page Application) だろう。SPA は、

Web アプリケーションの画面遷移をページの遷移として実現するのではなく、一つの HTML ページの中で切り替える。

実際、多くのシーンでは全画面を切り替える必要はなく、特定の領域だけを書き換えればよいケースが多い。SPA の Web アプリケーションは、ユーザーの操作をきっかけに、必要な情報のみをサーバー側から取得する。そして、取得した情報を特定の領域に HTML コンテンツとして構築した上で表示するのだ。

極論をいえば、サーバー側では HTML という概念すら不要になる。サーバー側は、クライアントのリクエストに対して、JSON (JavaScript Object Notation) 形式などのデータを返せばよい。つまりサーバー側は、ページ出力ではなく、Web API の提供という側面を強く持つ。このようにサーバー側とクライアント側とが疎結合になることで、システムの開発を分散しやすくなる。

また、クライアント側のユーザーインターフェースやデザインについて、クライアント側のエンジニアだけで調整できるようになる。とりわけ企業向けシステムでは、ユーザーインターフェースやデザインがおろそかになりがちだ。だが、一般消費者向けサービスが洗練されてくるにつれ、企業システムでもユーザーインターフェースやデザインを強く意識していかなければならないだろう。

## あらゆるモノが Web 技術でつながるように

近年は、パソコンやスマートフォンに限らず、さまざまな分野で Web 技術を使う試みが始まっている。多様なデバイスに Web 技術が応用され、接続される世界観を WoT (Web of Things) と呼ぶ。

WoT に似た言葉はほかにもある。以前から IoT (Internet of Things) という言葉が使われているが、これはあらゆるモノがネットにつながるという世界観を表している。米 Cisco Systems は、IoE (Internet of Everything) という言葉を使っている。これは、モノだけでなくヒトも加えて、世界観を広くとらえたものと考えればよいだろう。さらに、米 Salesforce.com は IoC (Internet of Customers) を提唱し始めた。

これらの世界観は非常に曖昧で明快な定義はない。筆者の理解では図 1-2 のようになる。もはやパスワードのオンパレードだが、それほど、ネットに接続する

対象が急拡大している、または、そう期待されていることの表れだろう。

Web 技術が多様なデバイスに応用されている一例として、テレビが挙げられる。近年販売されたテレビは、データ放送の後継となるハイブリッドキャストに対応している。テレビがネットに接続されていれば、詳細な解像度でリッチなコンテンツが表示される。実は、ハイブリッドキャストはHTML5 ベースの Web ブラ

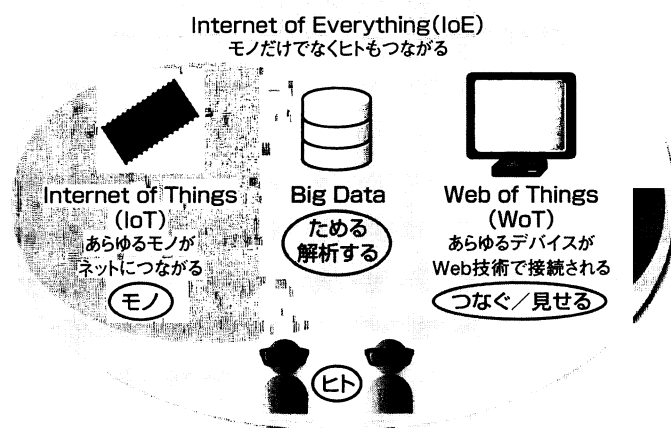


図1-2 IoE、IoT、WoTの関係

どれも世界観が曖昧で明確な定義はないため、筆者なりに整理した関係性を示した

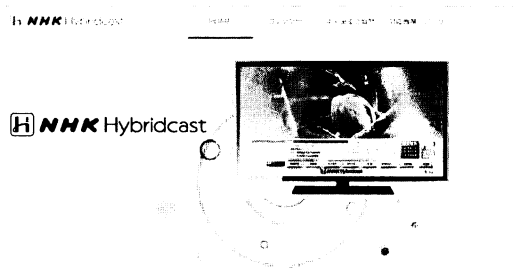


図1-3 HTML5を応用したNHKのハイブリッドキャスト

ハイブリッドキャストはHTML5ベースのWebブラウザで動作する。画面はNHKのハイブリッドキャストの紹介ページ。 <http://www.nhk.or.jp/hybridcast/online/>  
 画像提供：NHK

ユーザーによって動作する（図 1-3）。

また、国内では普及していないが、海外ではテレビメーカーがテレビ機器向けのアプリストアを提供している。一般の開発者もアプリを登録できる。例えば韓国 Samsung Electronics のテレビは、HTML5ベースでアプリを開発でき<sup>注14</sup>、開発用の SDK も配布されている。

## ゲームや自動車、コンピュータボードにも波及

コンソールゲーム機も Web とのかかわりが深い。国内でも有名な Nintendo Wii U には Web ブラウザーが搭載されており、Wii U のゲームを HTML5 ベースで開発できる（図 1-4）。もちろん、利用者は Web ブラウザーを起動して遊ぶのではない。一般のゲームアプリと同様に、専用のダウンロードサイト（Nintendo eShop）から取得してインストールするため、利用者からすれば Web ベースかどうかは区別できないだろう。

自動車業界では、カーナビに Web 技術の応用が模索され始めている。近年のカーナビはナビゲーションのみならず、音楽、ビデオなどさまざまなエンターテイン

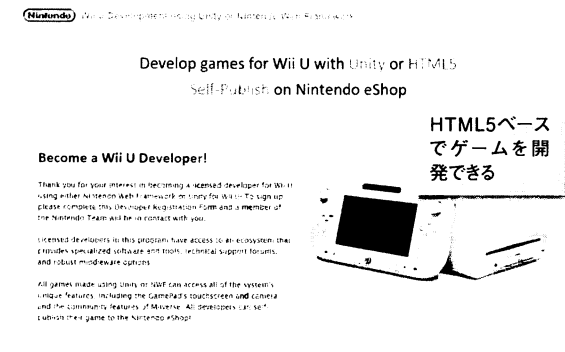
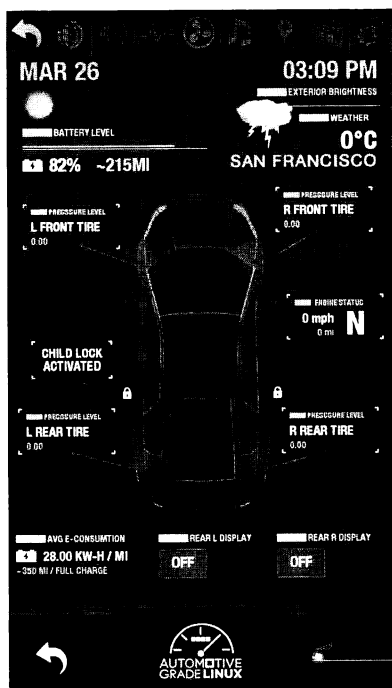


図 1-4 Wii Uの開発者向けWebサイト

ゲーム機であるWii UにはWebブラウザーが搭載されており、ゲームをHTML5ベースで開発できる。<https://wiiu-developers.nintendo.com/>

**注1-4) Samsung Smart TV Apps Developer Forum**

<http://www.samsungdforum.com/>



スマートデバイスや車載システム向けのOS「Tizen」ベースの基盤であるTizen IVI (In-Vehicle Infotainment) で動作する。アプリケーションはHTML、CSS、JavaScriptだけでも制作できる

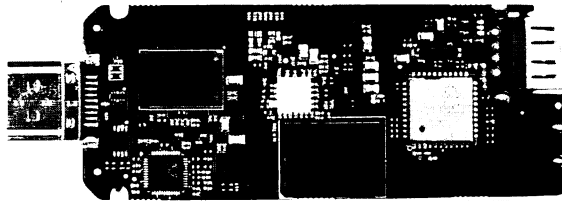
**図 1-5** Web技術を活用した車載システム向けアプリケーションの例

オープンソースソフトウェア (OSS) の車載システム向けアプリケーションを開発するプロジェクト「Automotive Grade Linux」が開発した画面の例

ト機能が盛り込まれており、IVI (In-Vehicle Infotainment) と呼ばれている。Web ベースの基盤を持つ IVI として、Linux Foundation が Tizen IVI の開発を進めている。Automotive Grade Linux というプロジェクトでは、Tizen IVI のデистриビューションの開発も行われている (図 1-5)。この Tizen IVI で動作するアプリケーションは、HTML、CSS、JavaScript だけでも制作が可能だ。まだ実車に搭載されてはいないが、Web 技術の応用範囲の広がりとしては注目に値する。

最後に SBC (Single Board Computer) にも触れておこう。ここ数年、Raspberry Pi をはじめとしたミニコンピューターボードが流行っている。さらに、Arduino といったマイコンボードも流行っており、かつての電子工作ブームが再来した。実は、ここにも Web 技術を応用する動きがある。KDDI が発表した

Firefox OSがインストールされている。  
Bluetooth Low Energy (BLE)に対応し  
ており、BLEセンサー機器と接続できる



**図 1-6** Web技術を応用した「Open Web Board」

KDDIが発表したSBC (Single Board Computer)

Open Web Board が代表的な例だ（図 1-6）。このボードには Firefox OS がインストールされている。他の SBC でも Android や各種 Linux ディストリビューションをインストールできるため、Chromium や Firefox をインストールして利用することも可能だ。

ここで紹介したトピックは一般消費者向けデバイスやサービスが多いが、この流れは当然ながら企業システムにも波及するだろう。これをコンシューマライゼーションという。環境さえ整えば、企業にも一気に押し寄せてくると考えられる。

### 1-1 のポイント

- リッチなコンテンツは、独自技術から標準化された技術である HTML5 にシフト。
- Single Page Application など、サーバーと疎結合なアーキテクチャーの採用が進む。
- Web 技術の応用範囲が、組み込み機器や IoT の世界にも広がりつつある。

## 1-2 アーキテクチャーの変化

# クライアントもMVCで設計 ページ遷移せずに画面表示

Web アプリケーションは複雑で高度な機能が求められるようになった。

旧来の手法を使い続けていては破綻してしまう。

Web のアーキテクチャーの変化を追っていき、今注目されている仕組みを解説する。

リッチな Web コンテンツが求められるようになり、コンテンツのコーディングが複雑化、かつ高度化している。Adobe Flash などのプラグインに頼れない昨今、リッチなコンテンツは JavaScript で開発しなければならない。

Web 業界がこの課題を抱える中、Web システムのアーキテクチャーも変化してきた。本パートでは、このアーキテクチャーの変化を解説する。

なお、アーキテクチャーの変化といっても、古いものが廃れるわけではない。実際の開発現場では、ここで紹介するアーキテクチャーのどれか一つだけを採用するのではなく、適材適所で使い分けたり、組み合わせて使ったりしている。

### 旧来の Web システムはサーバー側で HTML を生成

まずは Web システムで長く使われてきた仕組みと、その問題点を見ていこう。

クライアントがサーバーにコンテンツをリクエストしたら、サーバーはその要求に応じた HTML を生成してクライアントに出力する。このアーキテクチャーでは、Web ブラウザーは表示器でしかなく、そこにロジックは存在しない。そのため、Web システムのアーキテクチャーというと、「サーバー側のアーキテクチャー」を指すのが一般的だった。

この旧来のアーキテクチャーには、サーバーに処理が集中する欠点がある。サーバー側は、「テンプレートエンジン」という仕組みを備えている。テンプレートエンジンは、HTML コンテンツを生成する際に、表示すべきデータを、事前に用意

した HTML テンプレートに関連付ける（バインドする）。つまり、データをバインドして出来上がった HTML を Web ブラウザーに返すまでが、サーバー側の仕事だった。

このような仕組みだと、ユーザーがアクションを取るたびに、ページの生成が発生する。例えば、クライアントからサーバーへデータを送る際のフォームの送信をイメージすると分かりやすいだろう。本来であれば、フォームを送信したらその結果を画面に表示するだけでよいはずだが、実際には、ページ全体を書き換える形で結果を表示してしまう。

これは非常に無駄な処理だ。サーバー負荷、ネットワーク負荷、ブラウザー負荷のどれを取っても無駄が多い。そしてこれが、旧来の Web が Web アプリケーションになり切れなかった原因の一つといえる。

## 必要なコンテンツのみを取得する AJAX

この問題を解決するべく、2005 年ごろに <sup>エイジャックス</sup>AJAX（Asynchronous JavaScript + XML）と呼ぶ手法が開発者の注目を集めた。AJAX は、JavaScript が必要なコンテンツのみをサーバーから取得して、ページ内の必要な箇所だけを更新する。

AJAX は手法を表す用語であり、API の名前ではない。AJAX を実現する技術として、Web ブラウザーに実装された「XMLHttpRequest」という API を使う。XMLHttpRequest は、Web ブラウザーに組み込まれたプログラマブルな HTTP クライアントと考えるとよい。

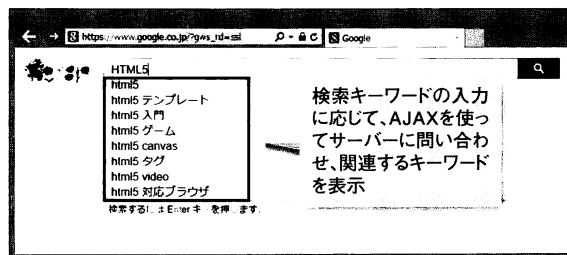


図 1-7 関連するキーワードを表示する Google サジェスト



XMLHttpRequest のような AJAX を実現する API は、2005 年以前に米 Microsoft が同社の Web ブラウザーである「Internet Explorer」に実装したのが始まりだった。しかし、しばらくは注目されず、米 Google が「Google サジェスト」や「Google マップ」をリリースしたことがきっかけで、一気に XMLHttpRequest、および AJAX が脚光を浴びるようになった。

Google サジェストは、検索キーワードを入力すると、関連するキーワードを候補としてリストアップしてくれる（図 1-7）。これは、検索キーワードの入力に応じて、AJAX を使ってサーバーに問い合わせ、応答を反映している。Google マップは、マウスでドラッグすると、表示エリアを移動できる。当然、世界中の地図情報を事前にロードできるはずはない。移動位置に合わせて、必要な地図情報をサーバーから動的に取得している。

このように、サーバー側での HTML 生成を伴わず、ユーザー操作に応じて必要なコンテンツが表示できるようになったことで、Web ページや Web アプリケーションのユーザビリティが一気に向上した。AJAX は、10 年以上前から普及し始めたとはいえ、今でも十分に注目すべき手法だ。

## REST でシンプルな Web システムを実現

次に注目され始めたのは「REST (Representational State Transfer)」である。REST の特徴は、HTTP で規定された仕組みを最大限活かす点だ。REST の概念を導入した Web システムを「RESTful」と呼ぶ。

REST の概念そのものは 15 年以上前から存在していたものの、実際に Web システムとして普及し始めたのは、かなり後になってからだ。RESTful なシステムは非常に分かりやすく、AJAX の普及によって一気に注目を浴びるようになった。

REST では、GET、POST、PUT、DELETE という HTTP メソッドを適切に使い分け、リソースごとにユニークな ID となる URI (Uniform Resource Identifier) を定義する。例えば、特定のコンテンツを取得するには、パラメーターとしてそのコンテンツを表す ID を URL に付け、GET メソッドでリクエストを送る（図 1-8）。もし、そのコンテンツを更新したいなら PUT メソッドを、削除したいなら DELETE メソッドを、新たなコンテンツを登録したいなら POST メソッ

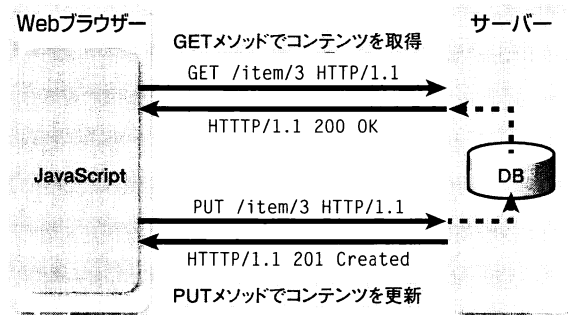


図 1-8 RESTで利用するメソッドの例

ドを使う。つまり、行為を表す動詞にあたる部分に、URL のパラメーターではなく、HTTP メソッドを使うのだ。

さらに、処理の結果が成功したかどうかは、HTTP レスポンスコードで確認できる。例えば、データの取得や更新、削除に成功したら「200 OK」を返し、データの追加に成功したら「201 Created」を返す。データ取得の際に該当のデータが存在しないなら「404 Not Found」を返し、リクエストのパラメーターが不適切なら「400 Bad Request」を返す。

RESTful なシステムは、「ステートレス」という点も特徴的だ。ステートレスとは、状態管理の仕組みを持たず、リクエスト URL が同じなら、クライアント側の状態にかかわらず、常に結果が同じになることを意味する。同じ URL によるリクエストにもかかわらず、クライアントの状態に合わせて応答内容が異なるのは「ステートフル」という。ステートレスは、サーバー側にとって非常にシンプルな設計になるだけでなく、クライアント側でも分かりやすい仕組みだ。

## サーバーは更新すべきコンテンツのみを返す

AJAX と REST の普及により、サーバー側の役割も変わってきた。AJAX と REST では、Web ページの一部だけを取得し、その領域だけを更新するケースが多いからだ。サーバーは従来、ページ全体を返していたが、現在は更新すべきコンテンツのみを表す「HTML スニペット」を返したり、データフォーマット

## XML

```
<?xml version="1.0" encoding="UTF-8" ?>
<result>
  <num>2</num>
  <members>
    <member>
      <id>3</id>
      <name>太郎</name>
      <sex>1</sex>
    </member>
    <member>
      <id>4</id>
      <name>花子</name>
      <sex>2</sex>
    </member>
  </members>
</result>
```

## JSON

```
{
  "num": 2,
  "members": [
    {
      "id": 3,
      "name": "太郎",
      "sex": 1
    },
    {
      "id": 4,
      "name": "花子",
      "sex": 2
    }
  ]
}
```

XMLよりデータ量が少ない。データ構造は、JavaScriptで直接的に扱える

## 図 1-9 XMLとJSONの比較

同じ情報をXMLとJSONで表現した例

「JSON (JavaScript Object Notation)」などのデータだけを返したりする機能も求められている。

AJAX のベースとなる XMLHttpRequest は当初、名前の通り、XML データを受信するための仕組みだった。だが現在では、プレーンテキスト、JSON、さらにはバイナリーデータを受信することも可能だ。

サーバーから返すデータのフォーマットとして XML も考えられるが、Web 業界ではあまり使われていない。なぜなら XML は、JavaScript からデータ構造を解析(パース)するのが面倒だからだ。つまり XML は、JavaScript の処理パフォーマンスにおいて、ハンディキャップを持つといえる。

XML と JSON で同じ情報を表現したものを比べてみてほしい (図 1-9)。XML は開始タグと終了タグが必要になり、データ量としては JSON と比べても大きくなりがちだ。一方、JSON のデータ構造は、JavaScript オブジェクトとして直接的に扱える。簡単に扱えるだけでなく、パースの手間が省ける分、パフォーマンスも良い。JSON は今の Web システムにおいて最もポピュラーなデータフォーマットといえる。

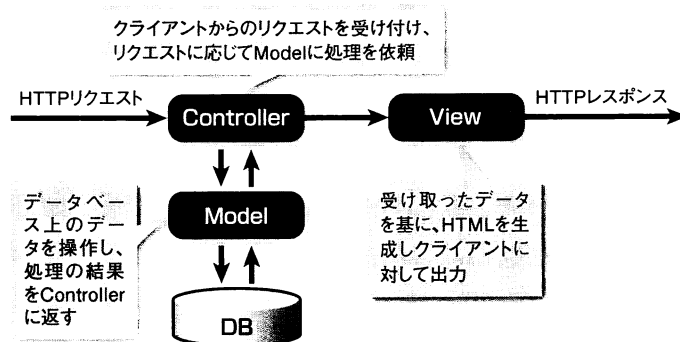
## JavaScript も MVC モデルを意識して実装

AJAX の普及により、一気に Web コンテンツは複雑、かつ高度化していった。これまで JavaScript といえば、表示コンテンツをちょっとだけリッチにする程度の存在として扱われてきた。AJAX が普及してからは、コンテンツ表示の要としての地位を確立していき、それに伴い JavaScript のコード量も増加した。

このような背景から、JavaScript の世界でもサーバー側システムと同様に、Model、View、Controller にプログラムを分けて実装する「MVC モデル」の概念が必要になった。

通常、サーバー側の MVC といえば、Controller がクライアントからのリクエストを受け、そのリクエストに応じて Model に処理を依頼する（図 1-10）。Model は、場合によってはデータベース上のデータを操作する。Model は処理の結果を Controller に返し、Controller 経由で、View に引き渡す。View は、受け取ったデータを基に、HTML を生成しクライアントに出力する。

今では Web コンテンツ（JavaScript）側でも、Model、View、Controller という概念を意識してプログラムを設計しなければ管理しきれない。実際にはサーバー側の MVC と全く同じではないが、それだけ JavaScript の仕事が増え複雑になってきたのだ。



**図 1-10** サーバー側の MVC

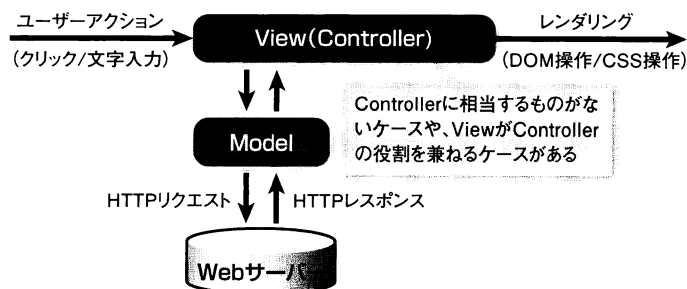
## View がユーザーアクションを受け取る

JavaScript 側の MVC は、サーバー側の MVC と少し異なる（図 1-11）。厳密には、Controller に相当するものがなかったり、View が Controller の役割を兼ねていたりする場合があるからだ。ここでは JavaScript でも MVC と仮に呼ぶが、厳密には違うため、「MV\*」や「M-V-whatever（MVW）」と呼ばれたりする。JavaScript で MVC モデルを実現するには、JavaScript フレームワークを使うのが一般的だ。

JavaScript での MVC は、多くの場合、View に相当する部分がユーザーアクションを受け取る。ユーザーアクションとは、マウスをクリックしたり、文字を入力したりする行為だ。ただし、ユーザーのアクションとは違うイベントがトリガーになる場合もある。いずれにせよ、何かしらの処理のトリガーを HTML のイベントとしてキャッチする必要があるため、Controller の役割の一部を View 側で担うケースが多い。

View は処理のトリガーをキャッチした後、Model に引き渡す。Model 側では、場合によっては、Web サーバーに対して、何かしらのデータ操作をリクエストする。ここでは、API として、前述の AJAX で説明した XMLHttpRequest を使う。

Model での処理結果は View に戻される。View では、処理結果に基づいて、レンダリング処理を行う。レンダリング処理とは、DOM（Document Object Model）操作や CSS 操作を指す。DOM 操作は特定のコンテンツを追加、変更し



**図 1-11** JavaScript側のMVC

M-V-\*やM-V-whatever（MVW）と呼ばれることがある

たり削除したりする処理を、CSS 操作はスタイリングの変更やアニメーション（移動や回転など）の処理を実行する。

## Single Page Application がもたらす変化

Web システムのアーキテクチャーで、大きな変化の一つといえるのが「Single Page Application」（以下、SPA）の登場だ。ここ数年、HTML/CSS/JavaScript は旧来の Web ページだけでなく、アプリケーションにも使われるようになってきた。スマートフォンのアプリをイメージするとよいだろう。Web ブラウザーでアクセスするコンテンツでも、スマートフォンのアプリのような見た目とユーザーインターフェースを持つものがある。

アプリケーションでは、さまざまな画面が用意され、必要に応じて表示画面が切り替わる。この画面遷移を、これまでの Web ページと同じように別の HTML を表示するページ遷移で実現してしまうと、表示に時間が掛かるなどユーザー体験（UX）を損なってしまう。当然、ページ遷移では、フェードやスライドといった画面を切り替える際の表示効果も使えない。

そこで登場したのが SPA だ。SPA では基本的に、表示コンテンツを切り替えるだけで、ページ遷移を発生させない（図 1-12）。つまり、毎回サーバー側で新

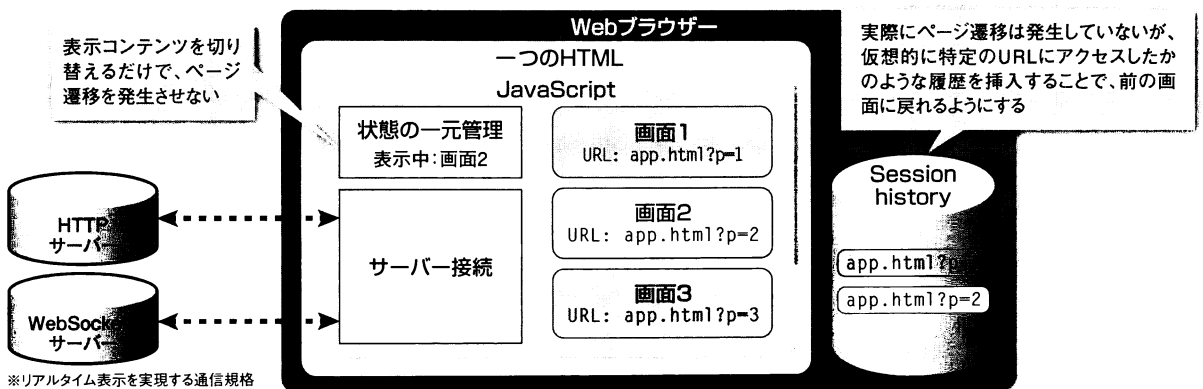


図 1-12 Single Page Application (SPA) のイメージ図

たに HTML を生成するのではなく、一つの HTML をベースに、一部のコンテンツだけを DOM 操作によって切り替えていく。

画面が少なければ、すべての表示コンテンツを一つの HTML 内にマークアップ（記述）しておき、クライアント内だけで表示する領域を切り替えることもできる。ただ実際には、数多くの画面が存在し、ユーザーごとに見せるコンテンツが異なるアプリケーションが多い。そのため、必要な都度、サーバーから表示コンテンツとなるデータを取得するのが一般的だ。

SPA では基本的に、HTTP を使ってサーバーから表示コンテンツを取得する。Web システムは RESTful な設計が一般的だ。しかし、画面が切り替わる際に、ページ遷移と変わらないほどにユーザーを待たせるようでは、ユーザーにとってメリットがない。

そこで最近では、通信の遅延を低減するために、HTTP に比べて通信を効率化したプロトコルである SPDY が候補に挙がるようになった。今後は、SPDY をベースに標準化された HTTP/2 を使うのが当たり前となるだろう。

もし Web アプリケーションがサーバーと常時接続する必要があるなら、双方向通信が可能なプロトコルである WebSocket も有効だ。WebSocket は Web ブラウザーからのリクエストなしにデータをやり取りできるため、プロトコル上のレスポンス向上が期待できる。

## アプリ全体をスクリプトで管理しやすい

SPA は Web アプリケーション開発において、どのようなメリットがあるのだろうか。まず、SPA はページ遷移が発生しないため、Web アプリケーション全体をスクリプトでコントロールしやすくなる。具体的には、何かしらの状態をスクリプトで把握可能になる。

もし画面遷移をページ遷移で実現してしまうと、ページ遷移が発生するたびに JavaScript がリセットされるため、JavaScript からだけではそれまでの状態を把握しにくい。まさに、旧来のようにサーバーとのセッションを Cookie などで維持し、サーバーでクライアントの状態を管理する必要がある。もちろん、今の Web ブラウザーはストレージ機能も提供しているため、状態をストレージに

保存し、JavaScript 側だけで状態を管理する手段も採れる。だがやはり、画面が遷移する都度、ストレージから情報を読み取っているのは、パフォーマンスに影響する。

SPA を採用するもう一つのメリットは、WebSocket を使ってサーバーと常時接続する際、画面遷移のたびにコネクションを接続し直さなくて済むこと。もし画面遷移の都度、WebSocket のコネクションを接続しているのは、ユーザー体験が悪くなるだけでなく、WebSocket サーバーにも無駄な負荷をかける。

### 画面が遷移した履歴を残す仕掛け

「ページ遷移がないのなら、URL も変わらないのではないか」と疑問に思う読者もいるだろう。まさしくその通りで、SPA では、URL は常に同じものになる。Web アプリケーションによっては、特定の画面を URL 指定で直接表示させたい場合もあるはずだ。また、Web ブラウザーの戻るボタンを使って、前の画面に戻りたい場合もあるだろう。

実は、SPA ではこれらの解決策を用意している。HTML5 には、「Session

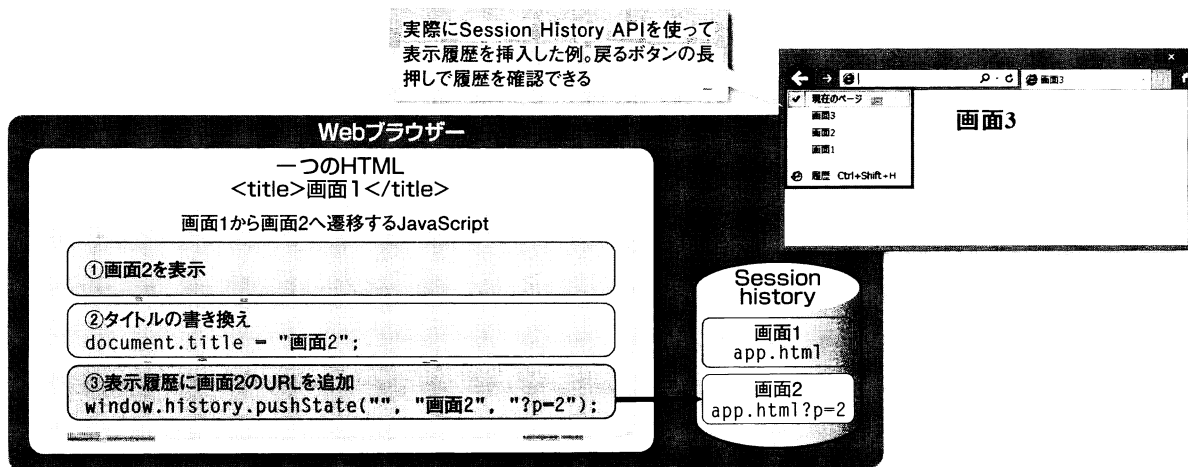


図 1-13 Session History APIで履歴を挿入する手順



history API」と呼ばれる API がある（図 1-13）。この API を使うと、JavaScript からページ表示履歴を制御できる。実際にページ遷移が発生していなくても、仮想的に特定の URL にアクセスしたかのような履歴を挿入できるのだ。

例えば、3 枚の画面が存在する Web アプリケーションを考えてみてほしい。この Web アプリケーションの URL は、`http://example.jp/app.html` とする。

ユーザーは、まず `app.html` にアクセスし、画面 1 を表示する。次に画面を操作して画面 2 を表示し、さらに画面 3 を表示したとしよう。実際には画面が遷移しても、`app.html` にアクセスしたままだ。ページ表示履歴には、画面 2 と画面 3 が表示された履歴は残らない。

履歴を残すには、画面遷移の際に、JavaScript から Session history API を使い、仮想的に画面 2 と画面 3 の履歴を挿入する。その際、各画面の適切なタイトルを、HTML の title 要素に設定しておく。こうすることで、ページ表示履歴のタイトルとして採用される。

履歴を挿入するときは、画面ごとに異なる URL を添える。図 1-12 では、画面 2 に対して `app.html?p=2` を、画面 3 に対して `app.html?p=3` を挿入している。どの Web ブラウザーでも、戻るボタンを長押しすると履歴を確認できる。

もし画面 3 を表示した状態で、ユーザーが Web ブラウザーの戻るボタンを押すと、JavaScript はそのイベントをキャッチする。そこで JavaScript は、イベント（`app.html?p=2` に戻るという操作）に基づいて、画面 2 を表示する。ユーザーが、Web ブラウザーのアドレスバーからダイレクトに `app.html?p=2` を指定してアクセスした場合は、そのパラメーターを JavaScript から取得して、画面 2 を表示する。

## 新しいアーキテクチャーが最適とは限らない

本パートでは Web システムのアーキテクチャーの変化を解説したが、必ずしも新しいアーキテクチャーが最適とは限らない。SPA は万能ではなく、単なるページ遷移で実現しても問題ないケースもある。

大切なのは、さまざまなアーキテクチャーが選択できることだ。こういったシステムにどういったアーキテクチャーがベストなのか、という一般解はない。そ

のため、システムの特性に合わせて最適なアーキテクチャーを選択し、サーバー側とクライアント側それぞれに適切な技術を選択するスキルが、今後ますます重要になってくる。

### 1-2 のポイント

- 旧来からあるAJAXは今でも重要な手法の一つとして使われ続けている。
- クライアント側でもMVCモデルが採用され始めた。
- 1枚のページだけで画面遷移を実現するSingle Page Applicationが登場。

## 1-3 MVC ライブラリの活用

# フレームワークの特徴を理解 保守も考え慎重に選ぶ

Web アプリケーション開発が高度化かつ複雑化するにつれ、JavaScript フレームワークや、別のプログラミング言語を採用するケースが増えてきた。

フレームワークと JavaScript の代替となる「AltJS」について紹介する。

高度で複雑な Web アプリケーションが増えるなか、JavaScript のコードの規模は肥大化している。現場では、スピーディーに Web システムを立ち上げるために、世間で注目されている JavaScript フレームワークを使って開発することが多い。

前パートは JavaScript における MVC（厳密には MV\*）の概念を紹介したが、本パートでは、それを実現するためのフレームワークについて紹介する。フレームワークによって提供する機能は異なる。その上、軽量なものから重厚なものまでさまざまある。ここでは、注目されているフレームワークのうち、「Backbone.js」と「AngularJS」を取り上げる。

最近では、JavaScript の代替となるプログラミング言語も注目されている。Web ブラウザーで JavaScript 以外の言語が動くのかと疑問を持つかもしれないが、そのからくりは本パートの後半で説明する。

### 慣れたライブラリを使える Backbone.js

まずは、JavaScript フレームワークのうち「Backbone.js」から見ていこう。Backbone.js は、MV\* に特化した軽量なフレームワークだ（図 1-14）。Backbone.js は「Underscore.js」という JavaScript ライブラリに大きく依存しており、これと併用する必要がある。さらに、DOM 操作に関しては JavaScript ライブラリ「jQuery」と併用するケースが多い。

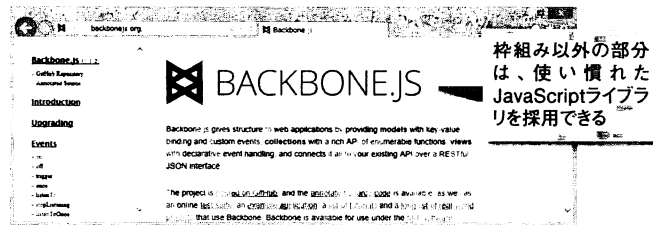


図1-14 Backbone.jsのWebサイト

<http://underscorejs.org/>

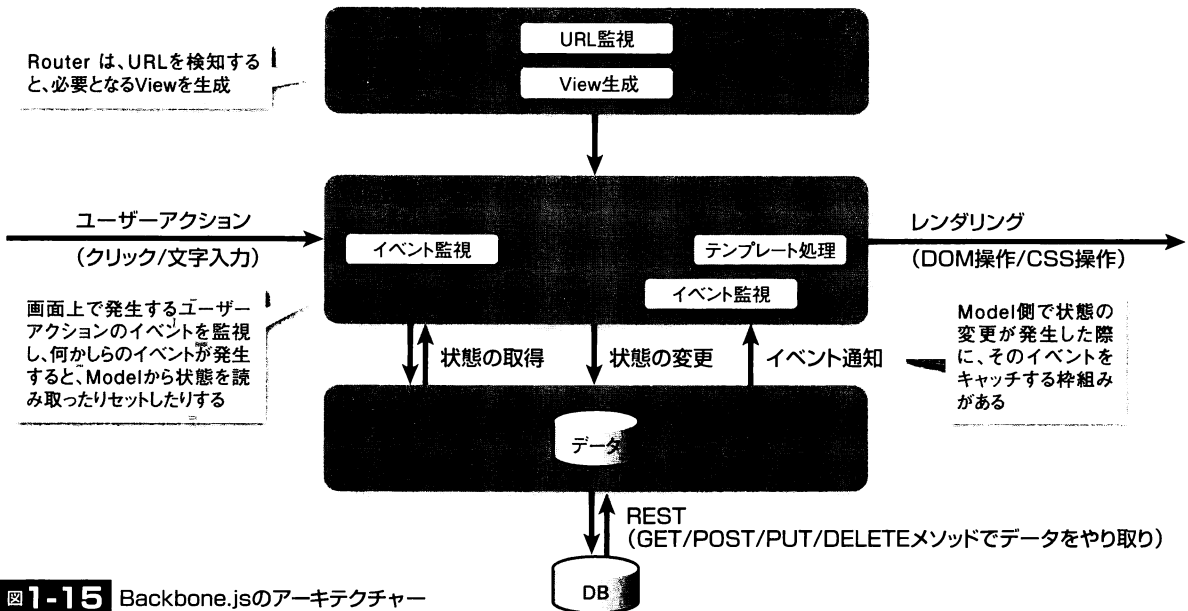


図1-15 Backbone.jsのアーキテクチャー

このように Backbone.js は軽量な半面、Backbone.js だけですべてをカバーできない。枠組み以外の部分に使い慣れた JavaScript ライブラリを採用できる点がメリットとなる場合もある。

Backbone.js の重要なモジュールは大きく分けて、View、Model、Collection、Router の四つである。これらは JavaScript オブジェクトとして扱われる (図

1-15)。図には Collection がないが、Model の集合のことだ。

まず、Router が URL をハンドリングする。ページ遷移なしに画面を切り替える SPA (Single Page Application) の場合は、Router が表示内容の切り替えをコントロールする。一見すると Router は、Model と View を橋渡しする Controller のような役割を担っているようだが、実際にはそうした機能はない。通常 Router は、URL を検知したら、必要となる View を生成する。View の生成時には、それに関連する Model を結び付ける。

View は、画面上で発生するさまざまなユーザーアクションのイベントを監視する。何かしらのイベントが発生すると、Model から状態を読み取ったり、セットしたりする。

View には、Model 側で状態の変更が発生した際に、Model で発生するイベントをキャッチする枠組みが用意されている。これによって、View から Model の内部に介入する必要がなくなる。このイベントの枠組みは、View と Model を疎結合とするために重要だ。

Model は、外部サーバーと REST でやり取りする機能もサポートしている。そのため、外部サーバーのデータベースにレコードを追加、変更、削除する処理を Model に組み込める。

## Underscore.js のテンプレートで開発を効率化

何かしらのユーザーアクションが発生し、必要な処理が終わったら、View 側でレンダリング処理 (DOM 操作や CSS 操作など) を行う。HTML 上の DOM をダイレクトに操作できるが、一般的には、Underscore.js が提供するテンプレートの枠組みを使う。

テンプレートとは、事前に用意した HTML のことだ。毎回、JavaScript から挿入したい HTML コンテンツを作り込むのは面倒なため、テンプレートの仕組みは開発の効率化には欠かせない。テンプレートには変数も埋め込めるため、Model で管理されているデータをそのままテンプレートにバインドできる。

テンプレートそのものは、スクリプトの中に変数として保持しておけるが、実際には HTML 側に記述しておくケースが多い。ただし、body 要素の中にコンテ

ンツとして記述しておくのではなく、script 要素の中にインラインで記述する。

Backbone.js は、軽量ながらも、MV\* としての強力な枠組みを持ち、サーバー側との連携までが一通り備わっている。SPA を開発する上では、強力なツールになる。

## テンプレートエンジンに近い AngularJS

続いて紹介する AngularJS は、米 Google が開発したオープンソースの JavaScript フレームワークだ (図 1-16)。MVC アーキテクチャーを採用しているが、実際に使ってみると、テンプレートエンジンに近いと感じる。サーバー側のシステム開発の経験があるエンジニアであれば、PHP の「Smarty」といったテンプレートエンジンを想像すると分かりやすい。事前に用意した HTML に変数などを記述しておけば、テンプレートエンジン側でそれらの変数を置き換えてくれる。

AngularJS のアーキテクチャーは、厳密ではないが、図 1-17 のようなイメージになる。Route とは、SPA でよく使われる URL のハンドリング機能で、Backbone.js の Router に近い。Route は URL に応じて Controller を振り分ける。表示画面ごとに Controller を用意すれば、SPA を効率よく開発できるようになる。

AngularJS には、ビジネスロジックを受け持つ Service と呼ばれる枠組みがある。AngularJS が機能として初めから用意した Service があるほか、自分でも作成できる。Service は、複数の Controller で利用できるモジュールのような役割

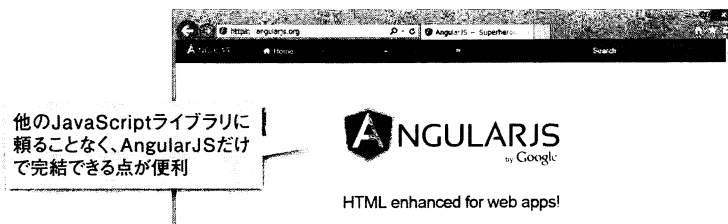


図 1-16 AngularJSのWebサイト

<https://angularjs.org/>

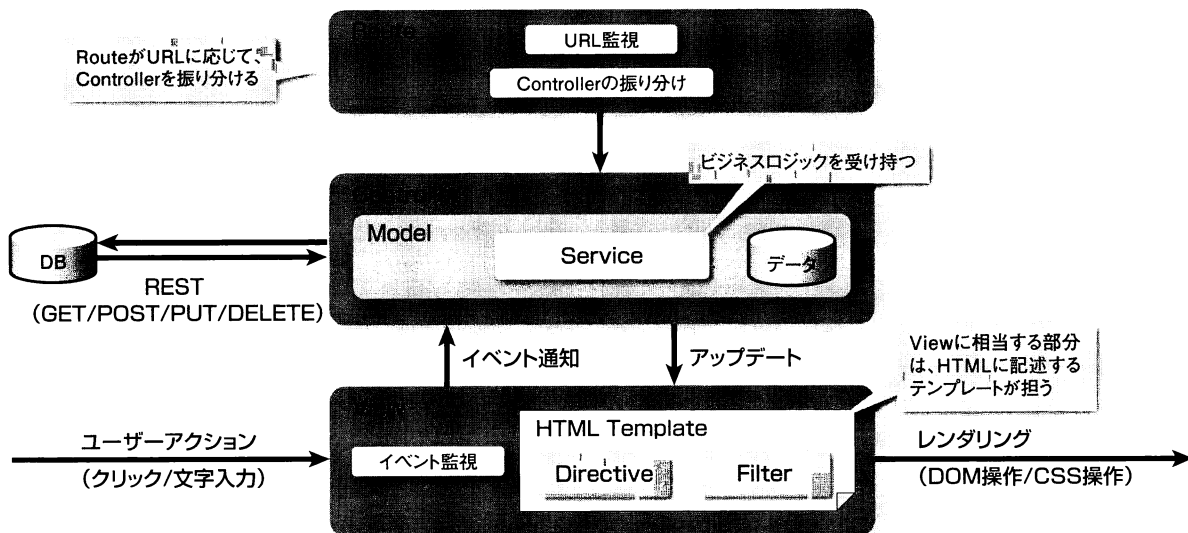


図 1-17 AngularJSのアーキテクチャー

を果たしている。

Controller は、View からユーザーアクションのイベント通知を受け取ると、Service に引き渡してデータを操作する。AngularJS では、HTML に記述するテンプレートが View に相当する。そのため、直接 HTML の DOM を操作することはない。Controller 側でデータを操作すれば、それがテンプレートに記述された通りにレンダリングされる。

## コンセプトは「HTML を拡張する」こと

AngularJS は、レンダリングに関する記述を、HTML の body タグの中にコンテンツとして記述できる点が大きな特徴だ。それゆえに AngularJS は、「HTML を拡張する」というコンセプトを強く持ったフレームワークといえる。これに対し、前述の Backbone.js は、テンプレートそのものをスクリプトに記述したりするなど、HTML よりもスクリプトを前提とした設計コンセプトといえる。

本書では基本的にコードの解説はしないが、AngularJS の便利さを理解してもらうために、少しだけ紹介しよう。図 1-18 は、実際に AngularJS の公式サイトに掲載されたチュートリアルサンプルを簡素化したものだ。HTML を見ると、body タグの中にレンダリングに関する記述があり、AngularJS 向けに記述されたテンプレートそのものになっている。また、値をバインドするための変数に相当する記述が書かれている。

AngularJS の楽なところは、スクリプト側で Model としてデータを定義するだけで、自動的にバインドしてくれる点だ。基本的に View に相当する処理は、AngularJS が勝手にやってくれる。つまり、AngularJS を使うと、Model に相当するロジックやデータの管理に集中できる。

もちろん、図 1-18 のようなリストだけでなく、フォーム関連の表示も充実している。さらに、クリックや文字入力といったイベントのハンドリングや、Model 側にセットされたデータをフィルターする機能もある。

AngularJS は、ここで紹介した機能の他にも、さまざまな機能が用意されている。多くのシーンで、他の JavaScript ライブラリに頼らず、AngularJS だけで完結できる点が便利だ。このため AngularJS は、「フルスタックフレームワーク」とも呼ばれている。

### HTML

```
<html lang="en" ng-app="myApp">
<head>
  <meta charset="utf-8">
  <title>AngularJS Sample</title>
  <script src="js/angular.min.js"></script>
  <script src="js/controllers.js"></script>
</head>
<body ng-controller="ListCtrl">
  <ul>
    <li ng-repeat="phone in phones">
      {{phone.name}}
    </li>
  </ul>
</body>
</html>
```

### JavaScript

```
var phonecatApp = angular.module('myApp', []);

phonecatApp.controller('ListCtrl', function ($scope) {
  $scope.phones = [
    {'name': 'Nexus S'},
    {'name': 'Motorola XOOM? with Wi-Fi'},
    {'name': 'MOTOROLA XOOM?' }
  ];
});
```

● サンプルコードの表示結果

レンダリングに関する記述は、HTMLのbodyタグの中にコンテンツとして記述できる

図 1-18 AngularJS のサンプルコードと表示結果



## フレームワークに確固たる定番はない

本パートでは二つのフレームワークを紹介したが、実は、JavaScriptの世界には多くのフレームワークが存在する。Backbone.js や AngularJS 以外では、「Ember.js」「KnockoutJS」「React」などが世界的に有名だ。このほかにも新たなフレームワークが次々と誕生している。

この状況は、ある意味、確固たる地位を築いている定番のフレームワークがないともいえる。これは、Web 標準と Web ブラウザーの進化によるところが大きい。Web ブラウザーが実装する API は増え続けているだけでなく、パフォーマンスも改善している。そして、Web ブラウザーが動作する PC やスマートフォンの進化も著しい。これにキャッチアップしていくと、どうしても、今注目されているアーキテクチャーが将来も最良の選択とは限らなくなるのだ。

また、それぞれのフレームワークを使いこなすための学習コストも無視できない。ましてや、チームで開発を行う場合は、メンバー全員がフレームワークを使いこなせなければならない。そのため、プロジェクトでどのフレームワークを使うべきかを判断するのは、かなり難しいだろう。

あくまでも個人的な見方ではあるが、受託で企業向けの Web システムを開発する場合は、選択したフレームワークが、その後のメンテナンスにどれほどインパクトを与えるのかを考慮することが重要だ。今話題になっている流行りのフレームワークが、ある時点で下火になってしまう可能性もある。数年後に、そのフレームワークを使いこなせるエンジニアをどれだけ用意できるかも心配だ。

通常、企業システムは何年も使われるため、その間さまざまな運用作業や機能追加作業を請け負うことになる。それを考えると、ある程度の実績を持つフレームワークを採用するほうがよい。

## フレームワーク選びのポイント

ではフレームワークはどのように選ぶとよいのだろうか。実は、どのフレームワークが最適かは一概にはいえない。プロジェクトに参加するエンジニアのスキルやプロジェクトの内容にも依存するからだ。

実際、Web 開発の現場によって判断基準はバラバラだ。また、当初は良かったと思って選定したフレームワークが、いざプロジェクトを進行してみると思った以上の効果が得られていない、ということもよく起こる。

唯一言えるのは、どのフレームワークを採用したとしても、必ず使いづらい点があり、使いこなしていくうちに不満は出てくるということだ。こうした中でポイントとなるのは、「技術的に不明な点が出てきたときに、簡単に情報が得られるか」である。

ユーザーが多いフレームワークほど、情報は多い。ユーザーが多いかどうかを把握するには、いつ誕生したかを確認するとよい。何年もバージョンアップを繰り返しながら、現在も利用され続けているものは、間違いなく多くのユーザーが



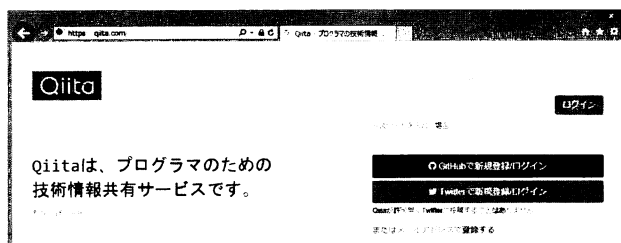
図 1-19 Stack Overflowのタグ検索

<http://stackoverflow.com/>

表 1-1 Stack Overflowでの質問数

AngularJSに関する質問が最も多い。2016年6月時点の数値

| タグ          | 質問数     |
|-------------|---------|
| angularjs   | 1149527 |
| backbone.js | 19788   |
| ember.js    | 19107   |
| knockout.js | 16329   |
| reactjs     | 17012   |



**図 1-20** 技術情報をブログ記事形式で投稿できるQiita

技術情報を日本語で得たいときに役立つ。https://qiita.com/

いる。また、ユーザーが多いと、バグの発見も早く、安心して使える傾向にある。

次に注目すべきは、ネットでの情報の充実度だ。公式サイトのチュートリアルや API リファレンスの充実度はもちろんのこと、ブログ記事や Q & A サイトでの質問と回答の数も参考になる。特にトラブルシューティングでは、Q & A サイトの情報が解決に役立つことが多い。

Q & A サイトはいくつかあるが、世界的に有名なのは「Stack Overflow」だろう (図 1-19)。Stack Overflow では質問にタグが付けてあるため、このタグから、どれくらいの質問が飛び交っているのかを調べられる。有名なフレームワークのタグが付けられた質問の数を調べると、圧倒的に AngularJS の質問が多いことが分かる (表 1-1)。

このほか、人気度を調べる方法としては、Google の検索数も参考になる。日本語の技術情報であれば、「Qiita」も参考になるだろう (図 1-20)。Qiita は、さまざまな技術情報をブログ記事形式で投稿できるサイトだ。Stack Overflow ほどの情報はないが、JavaScript フレームワークに関する記事は多い。また、初心者向けのチュートリアル記事から、かなりマニアックなトピックまで幅広く掲載されている。

## AltJS で JavaScript の難点を解消

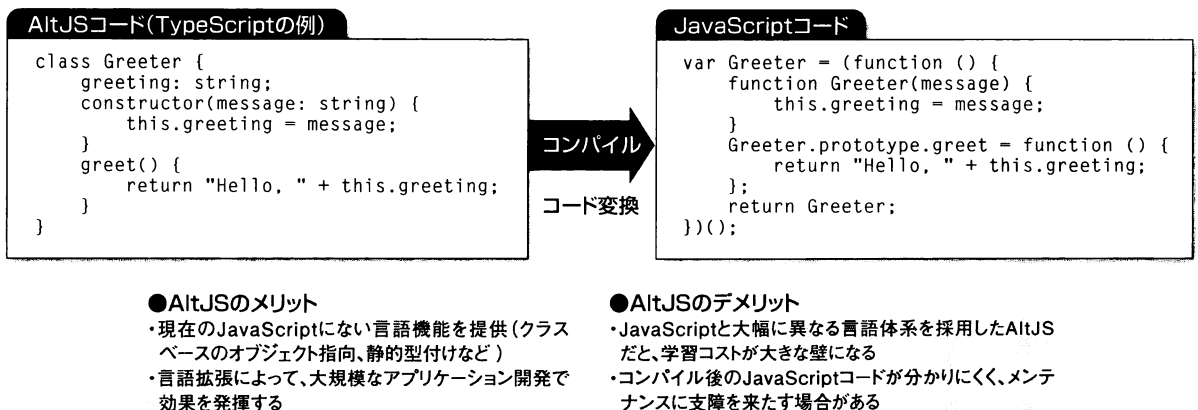
最後に、最近の Web 開発事情を語るうえで外せない「AltJS」(Alternative JavaScript) について説明する。AltJS は、JavaScript の代替となるプログラミ

ング言語の総称だ。ネット上では、AltJS はホットな話題の一つなので、聞いたことがある読者も多いだろう。

JavaScript は、プログラマーによっては歓迎されない言語だ。クラスを用いる一般的なオブジェクト指向プログラミング言語に慣れたプログラマーから見ると、JavaScript はクラスを持たないため扱いづらい（プロトタイプ型オブジェクト指向プログラミング言語と呼ばれる）。また、JavaScript は変数の型を意識せずにコードを書けるのだが、これがきっかけでバグを引き起こす場合もある。そのほか、JavaScript にはクセのようなものもある。さらに厄介なのは、コードの書き方によってパフォーマンスも変わる場合がある点だ。

これらを解決する試みとして生まれたのが、AltJS である（図 1-21）。Web ブラウザーは基本的には JavaScript しか扱えない。そのため AltJS で記述したコードは、コンパイルという作業が必要になる。ここでいうコンパイルは、C 言語のようなビルドやコンパイルとは意味が違い、AltJS のコードを JavaScript のコードに変換する行為を指す。

有名な AltJS としては、「TypeScript」「CoffeeScript」「Dart」「Haxe」などがある。とりわけ、TypeScript の人気が高い。TypeScript は米 Microsoft が開発した言語で、オープンソースとして公開されている。特徴は、JavaScript に慣れ



**図 1-21** JavaScriptの代替となるAltJS

JavaScriptの代替となるプログラミング言語を総称してAltJSという。AltJSで記述したコードは、JavaScriptコードに変換して利用する

たプログラマーなら学習コストが低い点と、コンパイル後に出力された JavaScript コードが読みやすい点だ。

AltJS が流行っているとはいえ、企業システムに AltJS を採用するかどうかについては、慎重になったほうがよいだろう。プログラミング言語そのものが変わってしまうので、安易に AltJS を採用したり、採用する AltJS をプロジェクトごとに変更したりすると、その後の保守に支障を来たしてしまう。もし採用するならば、開発メンバー全員がその AltJS を熟知しなければならないため、開発メンバーの同意を得るべきだ。

### **まずはキャンペーンサイトやモック制作で試す**

JavaScript フレームワークや AltJS の採用は、開発手法や開発メンバーのスキルなどの変更を求めることになる。安易に採用すると、かえって弊害を生んでしまう。一方、プロジェクトにかかわる開発メンバーが、JavaScript フレームワークや AltJS を使いこなせれば、得られる恩恵は大きい。もっとも、そこに至るまでには試行錯誤は避けられないだろう。

これらを用いる場合、いきなり長期間使われる企業システムに採用するのではなく、期間限定で使われるようなキャンペーン用アプリケーションや、システム提案時のモック造りなどで試してみるのがよい。このような取り組みを重ねながらノウハウをため、将来的に長期間使われる企業システムなどに採用できれば、多いにメリットを感じられるはずだ。

### 1-3 のポイント

- JavaScriptフレームワークは、Backbone.js とAngularJSの人気の高い。
- JavaScriptの代替言語としてTypeScriptに注目が集まる。
- フレームワークやAltJSには決定版がないため、慎重に採用すべき。

## 1-4 モバイルアプリ開発

# Web 技術でアプリを開発 コード共通化でコスト減

Android と iOS に二分されたスマートフォン市場。

HTML5 を使ってソースコードを共通化し、

開発コストを抑える「ハイブリッドアプリ」開発について紹介する。

モバイルアプリとは、Android が搭載されたスマートフォンや、iOS が搭載された iPhone などです。本書は Web がテーマなのに、なぜアプリが関係するのか不思議に思う読者もいるだろう。

アプリ開発では、Android 向けアプリであれば Java、iOS 向けアプリであれば Objective-C や Swift と呼ばれるプログラミング言語を使う。これらの言語で開発されたアプリを「ネイティブアプリ」と呼ぶ。

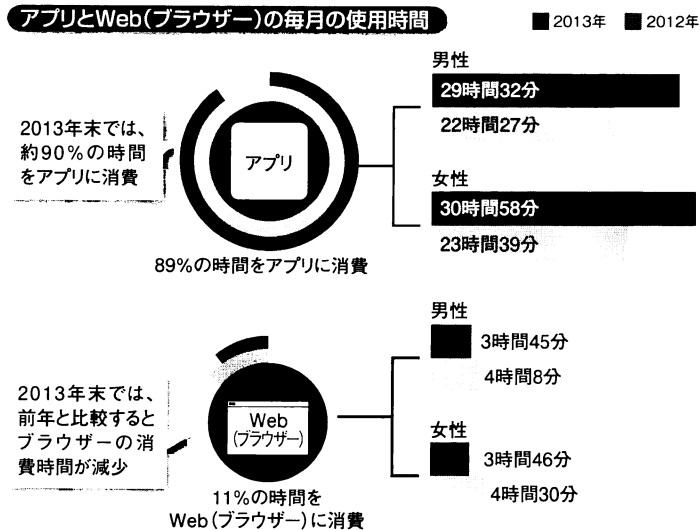
一方、Web ブラウザーで表示させて使うアプリは「Web アプリ」と呼ばれる。Web アプリは、実質的には、単なる Web ページと同じだ。JavaScript や CSS を駆使し、あたかもネイティブアプリであるかのようなユーザーインターフェースや機能を実現する。

近年、Web アプリでネイティブアプリに近いことができるように、ネイティブアプリを Web 技術だけで開発するという考え方が生まれた。つまり、HTML、CSS、JavaScript でネイティブアプリを開発するわけだ。このような Web 技術を使って開発するアプリを「ハイブリッドアプリ」と呼ぶ。

本パートでは、ハイブリッドアプリの仕組みと、実際のハイブリッドアプリの開発方法について紹介する。

### Web アプリは利用度が低い

まず、図 1-22 の調査結果を見てほしい。これは米 Nielsen が米国のスマートフォンユーザーに、アプリと Web ブラウザーの利用時間を調査した結果だ。2013 年



**図 1-22** 米国でのアプリとブラウザーの利用時間

米Nielsenが発表した調査結果を基に作成。<http://www.nielsen.com/us/en/insights/reports/2014/an-era-of-growth-the-cross-platform-report.html>

末では、90%弱の時間をアプリに費やし、Web ブラウザーの利用はたかだか10%強という結果だった。さらに、前年の2012年末の時点と比較すると、Web ブラウザーの消費時間が減少していることが分かる。この調査はやや古いですが、現在でもこの傾向は続いているだろう。

この調査結果は、Web 開発者にはショッキングだ。調査結果が出た頃は、「HTML5 の登場によってさまざまなアプリは Web 技術で作れるようになるだろう」と開発者が期待していたからだ。結局、スマートフォンでは、すべてを Web ブラウザーで実現することはかなわなかった。

Web 技術への誤解もあり、開発者には HTML5 への期待が高まっていた。しかし、ネイティブアプリにできて、Web アプリにできないことはたくさんある。とりわけ Web アプリは、「通知」系と「外部デバイスとの連携」が非常に弱い。

例えば、ネイティブアプリでは当たり前のように使われる「プッシュ通知」。スマートフォンの画面上や待ち受け画面に、サーバー側から通知する機能だ。だ



が、これは OS に大きく依存する機能のため、Web ブラウザーで動作する Web アプリでは扱えない。また、Bluetooth、USB、NFC (Near Field Communication) といった外部デバイスとの連携に必要な通信系の API も、Web ブラウザーには用意されていない。特に、プッシュ通知が使えないのは、アプリビジネスにかかわる多くの開発者にとって大問題だ。

もう一つ大きな問題がある。それはアプリマーケットだ。ネイティブアプリであれば、アプリを提供する仕組みとして「Google Play」や「App Store」といったアプリマーケットがある。加えて、料金回収の仕組みまで用意されている。売上げの何割かはマーケット運営事業者に支払うものの、マーケティングコストや料金回収のシステム作りなどを考えると、スタートアップ企業にとっては非常に便利な環境が用意されている。

一方、Web アプリは通常の Web サイトと同じため、自社サイトにユーザーを誘導しないと何も始まらない。さらに、課金決済の仕組みも自前で用意しなければならない。アプリマーケットに売上げの一部を献上する必要があるとはいえ、敷居が高いのが実情だ。

## Web アプリはパフォーマンスにも難点

Web アプリはパフォーマンスについても問題がある。ネイティブアプリは下層の OS に最適化されているため、ハードウェアの性能を最大限に生かせる。しかし Web アプリは、Web ブラウザーというアプリケーション基盤が OS との間に入っている分、ネイティブアプリに比べてパフォーマンスが悪くなる。特に、アニメーションを多用し、ユーザー操作に即反応するゲームなどのアプリでは致命的だ。また、SNS に見られるような無限スクロールを伴うアプリも、Web アプリでの開発は難しい。

パフォーマンスに関しては、Web 技術を用いるハイブリッドアプリでも同様の傾向がある。お馴染みの Facebook のアプリは当初、ハイブリッドアプリだった。実際に目にする部分、操作する部分の多くは、HTML で作られていた。しかし、2012 年には、ハイブリッドアプリからネイティブアプリに移行した。国内ではあまり利用度が高くないが、海外では有名な SNS の LinkedIn も、2013 年にはハ

イブリッドアプリからネイティブアプリに移行した。

実は当時、Web ブラウザー上で、Web アプリがどれほどメモリーを消費しているかを知る手段がなかった。さらに、JavaScript によるメモリーリークの発見が困難だったため、長時間安定して動作する Web アプリの開発が難しかった。

冒頭に紹介したようなスマートフォンユーザーの Web ブラウザー利用時間の減少、そして、ハイブリッドアプリからネイティブアプリへの回帰といったトレンドから、一時は「Web は死んだ」と言われることが増えた。

こうした状況はスマートフォンのスペック向上で変化しつつある。今後も、ネイティブアプリのパフォーマンスに勝ることはないが、多くのアプリはハイブリッドアプリでも問題なく使えるようになっている。

## ブラウザー機能をネイティブアプリから利用

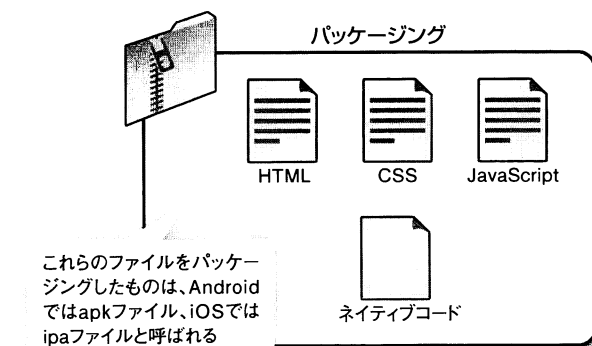
Web アプリには課題があるものの、同じ技術で複数の OS に対応するアプリを開発できるのは非常に魅力的だ。

先ほど説明したパフォーマンスの話は、ゲームなどの派手で話題性の多いアプリで大きな問題となるが、実際多くのアプリは動きが派手ではない。むしろ、ユーザー操作に対してさほど要求が厳しくないものが多い。こういったアプリでは、ハイブリッドアプリ開発の採用が増えている。

では、ハイブリッドアプリは、どうやって Web アプリをネイティブアプリとして動作させるのだろうか。実は、Android も iOS も、Web ブラウザーの機能をネイティブアプリから利用できる仕組みを提供している。この仕組みを、Android では「WebView」、iOS では「UIWebView」や「WKWebView」という。ここでは、これらをまとめて WebView と呼ぶ。

WebView は、URL を指定して該当のページを表示するだけでなく、アプリと同じフォルダー内の HTML を読み込んで表示できる。極端な話だが、URL を指定して WebView を呼び出し、インターネット上の Web ページを表示するだけの、実体としては空っぽのアプリも作れる。もちろん、これではアプリマーケットの審査が通らないため意味はないが、イメージは湧くだろう。

ハイブリッドアプリ自体には、HTML ファイル、CSS ファイル、JavaScript ファ



**図 1-23** ハイブリッドアプリに必要なファイル

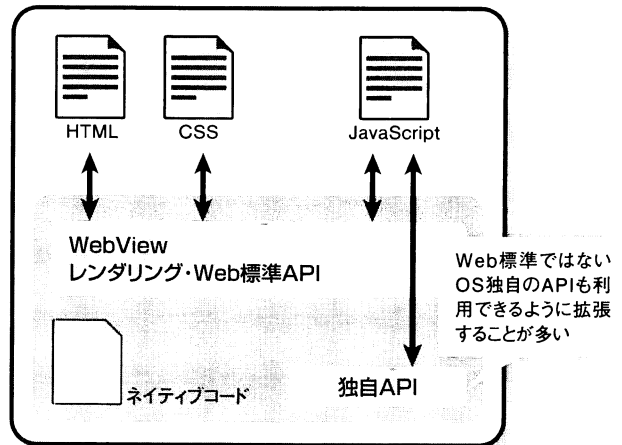
イルなど、Web アプリとして必要なファイル一式を同梱する（図 1-23）。これだけではネイティブアプリとして動作しないため、WebView 上でアプリを実行するためにネイティブコードも必要である。これらを一つのファイルとしてパッケージングする。

パッケージングされたファイルは、Android であれば apk ファイル、iOS であれば ipa ファイルとなる。つまり、最終的にはネイティブアプリと同様に、OS が指定する開発環境でビルドする必要がある。結果的に OS から見れば、通常のネイティブアプリと同じになり、アプリマーケットを通して配布できるようになる。

## OS 独自の API も利用できるように拡張

では、ハイブリッドアプリが動く仕組みを簡単に見てみよう（図 1-24）。まずネイティブコードで、OS が提供する WebView を呼び出し、同梱の HTML ファイル、または事前に定義された URL を WebView に読み込ませる。この時点で、Web ブラウザーで HTML ファイルを開いた状態と同じ。単に HTML を表示するだけなら、ネイティブコードの役割はたったこれだけだ。

しかし、このままでは WebView、つまり Web ブラウザーが提供する Web 標準の API しか使えない。これでは、アプリマーケットで配布できることしかメリットはなく、ハイブリッドアプリ化するメリットは大幅に薄れてしまう。そのため



**図1-24** ハイブリッドアプリの動作の仕組み

実際には、Web 標準ではない OS 独自の API も、JavaScript から利用できるように機能を拡張することが多い。

この場合、ネイティブコードは、JavaScript とネイティブアプリ向けの API との間の橋渡しをするブリッジの役割を果たす。本来、WebView では使えない API を、ネイティブコードによって JavaScript コードから利用できるようにするのだ。こうして、JavaScript だけでネイティブアプリと変わらない機能を利用できるようになる。

ただしこの API は、ネイティブコードの独自の API となるため、他の環境との互換性は全くない。とはいえ、この API 以外の部分はすべて標準の HTML、CSS、JavaScript で作られるため、他の環境、または他の OS への移植は、ネイティブアプリに比べるとはるかに楽になる。

## クラウドベースのハイブリッドアプリ開発環境

ハイブリッドアプリの開発では、前述のネイティブコードを毎回自分で作るのは大変なので、開発環境を利用するのが一般的だ。すでに数多くのハイブリッドアプリ開発環境が提供されている。

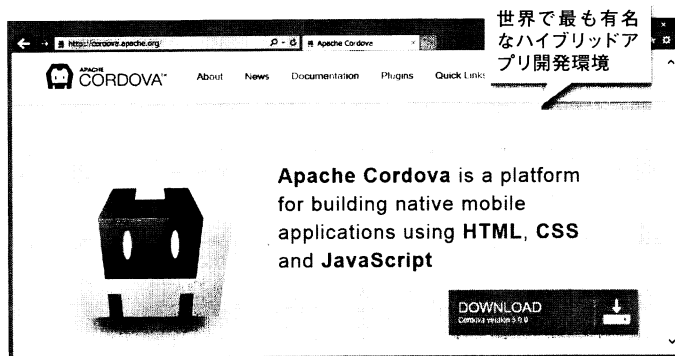


図 1-25 Apache Cordova

世界で最も有名なハイブリッドアプリ開発環境は「Apache Cordova」だ（図 1-25）。もともと、カナダの Nitobi Software が「PhoneGap」という名称でオープンソースソフトウェアとして公開したものだ。後に米 Adobe Systems が Nitobi Software を買収したため、PhoneGap は Adobe Systems の商標になった。しかし、ソフトウェア資産は今なおオープンソースとして提供中で、現在は Apache Cordova という名称に変更されている。

Apache Cordova では、コマンドラインでプロジェクト（ソフトウェア作成に必要なファイルなどを管理する単位）を生成すると、必要なファイルがすべて作られる。あとは、デフォルトの HTML ファイルや JavaScript ファイルを編集してアプリを開発する。

もちろん、Apache Cordova には、独自の API が多数用意されている。しかし WebView は、Web 標準の API をサポートしているものの、独自 API をサポートしているわけではない。そこで Apache Cordova は、JavaScript コードから独自 API にアクセスできるようにするために、cordova.js という JavaScript ライブラリを用意している。実際にアプリを開発する際には、HTML に cordova.js を script タグで読み込んだ上で、アプリを開発する。ハイブリッドアプリ開発環境を提供するほかのツールやサービスでも、こういった独自 API にアクセスするための JavaScript ライブラリを通してアプリを開発することになる。

開発したアプリは、最終的にはスマートフォン OS ごとにビルドする必要がある

る。ただし、Apache Cordova にはビルドの仕組みがない。このため、Android 向けアプリ開発であれば「Android SDK」、iOS 向けアプリ開発であれば「Xcode」など、一通りの開発環境を構築しておく必要がある。

しかし、できれば Web アプリ開発に必要な最小の環境でハイブリッドアプリを開発できるようにしたいところだ。この場合は、ビルド環境をクラウドで提供するサービスを利用するとよいだろう。以下に、無料のサービスをいくつか紹介する。さまざまな有料の付加サービスがあるので、まずは試してみて、自社プロジェクトに適したサービスを選ぶとよい。

### ■ PhoneGap Build

Adobe Systems が提供するクラウドベースのビルド環境（図 1-26）。Web 開発者にはおなじみの Web オーサリングツール「Dreamweaver」と連携してアプリを開発することもできる。

### ■ Monaca

「Monaca」はアシアルが提供するハイブリッドアプリ開発向けの製品・サービスである（図 1-27）。Cordova をベースに、さまざまな機能やサービスを付加している。クラウドベースの「Monaca クラウド」は、一般的な IDE が用意する機能を、Web ブラウザーだけで実現する（図 1-28）。コードはすべてクラウド上で

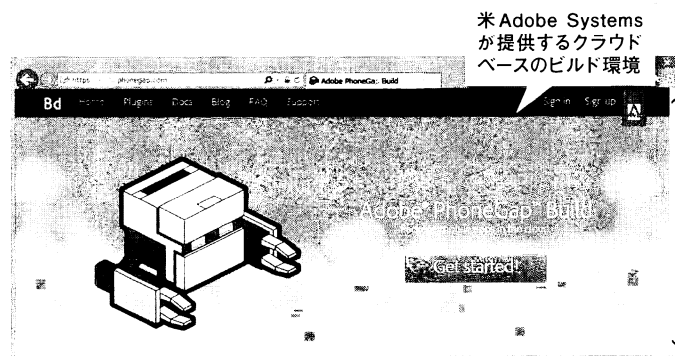


図 1-26 Phone Gap Build

アシアルが提供する  
Cordovaベースのハ  
イブリッドアプリ開  
発環境サービス



図 1-27 Monaca

アプリ開発をWebブ  
라우저だけで完結  
できる

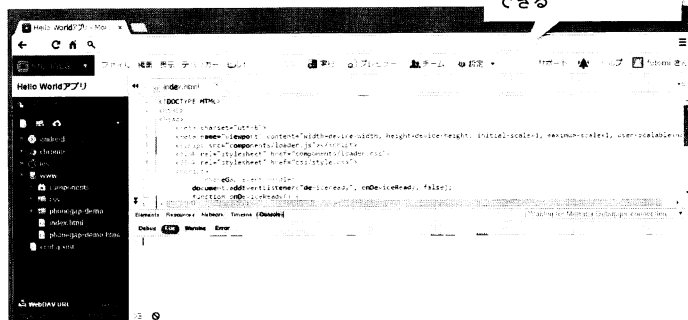


図 1-28 Monacaクラウド

管理され、そのままビルドも可能。アプリ開発すべてを Web ブラウザーだけで完結できる。

## ■アプリカン

ニューフォリアが提供するハイブリッドアプリ開発環境サービス（図 1-29）。アプリカンとは、Cordovaではなく、独自のハイブリッドアプリエンジンを利用するが、多くのAPIはCordovaと互換性がある。また、Ridgeが提供する位置連動プッシュサービス「popinfo」、インターメディアプランニングがNTTドコモの

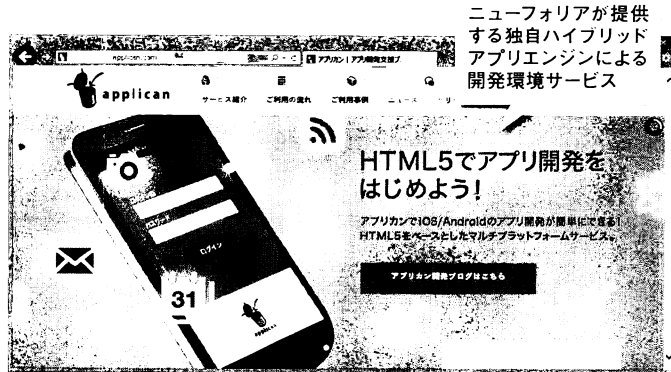


図 1-29 アプリカン

技術開発成果を使って提供するジオフェンシングサービス「39Geopla」など、Android や iOS 向けに提供された各社の SDK を組み込んで、JavaScript から利用できるようにしている点も特徴だ。実際に開発した Web アプリをクラウドにアップロードするだけで、ビルドせずに実機で確認できる。

## ファイルを用意しシミュレーターで動作確認

ハイブリッドアプリ開発の流れは次のようになる。通常は、何かしらハイブリッドアプリ開発サービスが提供するテンプレート（HTML、CSS、JavaScript ファイル）があるため、それを利用する。もちろん、自分で用意したファイルでも構わない。

ハイブリッドアプリ開発サービスでは、初期に起動する HTML のファイル名が決められている。もしくはファイル名を設定できる。初期起動用のファイル名で HTML ファイルを用意すれば、あとは通常の Web ページを作る手順と同じだ。

どのサービスでも、ハイブリッドアプリ開発環境が提供する独自 API を簡単に使えるようにするための、JavaScript ライブラリを用意している。それを HTML に組み込めば、ハイブリッドアプリ開発サービスが提供する独自 API を利用できる。すでに紹介した MVC フレームワークも当然使える。画面構成が複雑なアプ



りなどでは重宝するだろう。

ある程度、Web アプリが形になったら動作を確認する。この機能はハイブリッドアプリ開発サービスによって異なるため、ここではアプリカン为例に説明する。

アプリカンでは、web という名前のフォルダーを作り、その中に Web アプリを構成するファイルを入れておく。このフォルダーを web.zip というファイル名で zip 圧縮する。Web ブラウザーから管理メニューにログインし、そこから web.zip をアップロードする。

次に、事前にスマートフォンやタブレットにインストールしておいたアプリカンシミュレーターを起動する（図 1-30）。画面には、クラウドにアップロードされたアプリの一覧が表示される。該当のアプリを選択すると、アプリが起動し動作を確認できる。シミュレーターにはログ表示機能もあり、JavaScript エラーもチェックできる。

一通り、シミュレーターで動作を確認したらビルドする。Android 向けと iOS 向けに、それぞれ Web ブラウザー上でビルドを実行できる。ビルドが完了したら、Android であれば apk ファイル、iOS であれば ipa ファイルを Web ブラウザーからダウンロードする。その後、各アプリマーケットに申請する。

以上が一通りの流れだが、基本的にはネイティブアプリの開発環境は一切不要



図 1-30 アプリカンシミュレーター

だ。ほとんどは Web ブラウザーを通して、クラウド側で処理してくれる。

### ハイブリッド開発なら独自 API での機能を要確認

本パートはハイブリッドアプリ開発について説明したが、現場ではハイブリッドアプリ開発を採用すべきか、ネイティブアプリにすべきかは悩むところだろう。

アニメーションやユーザー操作の反応を、ハードウェアのスペックの限界まで高めたいなら、ネイティブアプリとして開発するべきだろう。特にゲームアプリは、ネイティブアプリで開発するケースが多い。ただし、OS ごとに開発が必要となるため、それなりの開発コストを覚悟しなければならない。それ以外のアプリであれば、おおむねハイブリッドアプリで問題ないはずだ。企業向けシステムに使うような管理系クライアントなら、ハイブリッドアプリ開発が最適だ。

近年、企業向けシステムも、旧来のように PC 向けのクライアントソフトだけでなく、スマートフォンやタブレットに対応したクライアントアプリが求められるようになった。企業によっては、社員が個人で所有するスマートフォンでもシステムを利用できるようにする流れもある。これは BYOD (Bring your own device) と呼ばれる。セキュリティの問題もあり、まだ一般的ではないが、用途によっては BYOD を採用する事例も増えてくるだろう。この場合、多種多様な機種や複数の OS に対応しなくてはならない。こうした場合にこそ、ハイブリッドアプリ開発が適している。

ハイブリッドアプリ開発では、Web 標準にない機能がどのくらい必要かを事前に把握しておくことが肝要だ。Web 標準にない機能を使いたい場合は、ハイブリッドアプリ開発サービスが独自 API として提供しているかを調べておく必要がある。もし独自 API として提供していないのであれば、ハイブリッドアプリ開発サービスの利用をあきらめて、ネイティブアプリとして開発するしかない。企業システムのクライアントアプリとして、ハイブリッドアプリ開発が可能なプロジェクトだと分かれば、その恩恵は計り知れないだろう。

### 1-4 のポイント

- ハイブリッドアプリ開発はコード資産を共通化でき、開発コスト削減が可能。
- 企業システムのクライアントアプリには、ハイブリッドアプリ開発が適する。
- プロジェクトで必要な機能を把握し、ハイブリッドアプリで実現できるかどうかを検討。

## 1-5 企業システムを変える HTML5

---

# マルチデバイスに対応し 運用費の削減にも貢献

企業システムでは、マルチデバイスに対応し、かつ OS のバージョンアップの妨げにならない Web アプリケーションの開発が求められるようになってきている。HTML5 による Web ブラウザーベースのフロントエンドが注目される中、その効果や使い勝手を解説する。

企業システムでは、入出力端末として Windows PC を採用し、クライアントソフトウェアを Windows デスクトップアプリとして開発することが多い。また、企業で使う Web アプリケーションといえば、米 Microsoft 独自の機能を駆使し、Internet Explorer (IE) 上で動作するものが一般的だった。数年前までは Web 標準の技術の進歩が遅れ気味で、できることが限られていたため、やむを得ない選択といえる。

一方、HTML5 をはじめとした Web 標準の技術は年々進化し、多くの Web ブラウザーに実装されるようになった。そのため、企業システムのフロントエンドを開発するための技術として、HTML5 が注目されている。

以下では、このように進化した Web ブラウザーを企業システムのクライアントとして活用する上で、どのような Web 標準の API が使えるのかを中心に解説する。

### Web ブラウザーだからインストールが不要

まずは、企業システムのフロントエンドに Web ブラウザーを使うメリットを考えてみる。最も分かりやすいメリットは、インストール作業が不要な点だろう。

通常、Windows PC を購入すれば、あらかじめ IE が組み込まれている。システムリリース直前だけでなく、PC のリブレースや追加などの際にも、インストール作業が不要だ。さらに、HTML5 ベースで開発されたフロントエンドは基本的

に Flash などのプラグインを使わないため、ユーザーに何かしらの追加インストールの作業を強いることもない。

また、クラウドベースの Web コンテンツとして作られたフロントエンドであれば、バージョンアップのためのインストールという概念すらなくなる。このため、運用の側面からも、Web ベースでフロントエンドを開発するメリットは大きいだろう。

## スクリーンに合わせて表示を最適化する RWD

フロントエンドの開発に HTML5 を採用すれば、コードの流用性が増すことは間違いない。場合によっては、全く同じコードを、あらゆる端末に向けて利用できるだろう。

その際、同じ HTML マークアップで、PC にもスマートフォンにも Web サイトを表示できるように、「Responsive Web Design (RWD)」を採用することが多い(図 1-31)。通常、Web コンテンツは、スクリーンのサイズによってレイアウトを変えなければならない。RWD では、スタイルシートをスクリーンのサイズに応じて切り替えることで、コンテンツのレイアウトをダイナミックに変える。

RWD は、Web サイトに限らず、Web アプリケーションでも使われる。例えば



**図 1-31** あらゆるデバイスに対応する Responsive Web Design

米 Microsoft が提唱する「ユニバーサル アプリ プラットフォーム」で開発した画面例

Microsoft は、一つのアプリを PC、タブレット、スマートフォン、Xbox といったさまざまなデバイスで動作させる仕組みとして、「ユニバーサルアプリプラットフォーム（Universal app platform）<sup>注</sup>」を提唱している。

### 標準技術を積極的に実装した「Edge」

これまでは、企業システムで HTML5 を採用しようとする、古い Windows PC にプリインストールされている IE が問題になっていた。旧来の IE は、Web 標準の API の実装が最も遅れていたからだ。

そのため、Flash、Silverlight といったプラグインや、IE のみで使える Microsoft 独自の機能を駆使する必要がある、他の環境に流用できなかった。これでは、Web ブラウザーをフロントエンドに採用したメリットが半減してしまう。

しかし最近是这样した事情にも変化が起きている。Microsoft は Web 標準の機能を IE に積極的に実装するようになり、IE は Google Chrome や Mozilla Firefox に勝るとも劣らないレベルの Web ブラウザーへと進化しているのだ。さらに、2015 年 7 月にリリースした Windows 10 では、標準ブラウザは IE ではなく「Edge」と呼ばれる Web ブラウザーになった（図 1-32）。

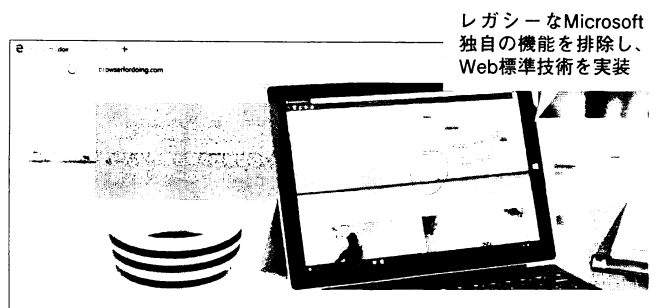


図 1-32 Windows 10の標準ブラウザ「Edge」

注)

A first look at the Windows 10 universal app platform (<http://blogs.windows.com/buildingapps/2015/03/02/a-first-look-at-the-windows-10-universal-app-platform/>)

Edge は、これまでのレガシーな Microsoft 独自の機能を排除し、最先端の Web 標準技術を積極的に実装するブラウザである。もちろん、従来の IE 11 も利用できるが、これから新たにシステムを開発するのであれば、Edge を視野に入れたほうがよいだろう。

## Web ブラウザーを採用するデメリットもある

Web ブラウザーが進化しているとはいえ、企業システムのフロントエンドに Web ブラウザーを採用することにはデメリットもある。それは、Windows デスクトップアプリと比べて、できることが限られる点だ。

例えば、日本語入力に必要な IME の入力文字種の強制が挙げられる。企業システムでは、入力フィールドに合わせてカナや英数字などに、IME のモードを強制的に切り替える機能がよく使われている。Web ブラウザーだとこのような制御に限界がある。

このほか、古い Windows OS だと対応する IE のバージョンが古いため、HTML5 ベースの機能がほとんど使えない。企業システムがフロントエンドに求める要件次第ではあるが、筆者の個人的な感覚では、IE 9 でも HTML5 ベースでフロントエンドを開発するのに不足を感じる。Chrome や Firefox に近いレベルの API 実装を期待するのであれば IE 11 が適する。またはいっそうのこと Edge にフォーカスするほうがよいだろう。

先ほど IME の入力モードの切り替えに触れたが、筆者は、会計システムといった帳票入力でもない限り、IME の入力モードを強制的に切り替えるのはやめるべきと考えている。通常、Web ページをブラウザで閲覧する際には、IME の入力モードが強制的に切り替わることはない（できない）のが一般的だ。そのためユーザーは、Web ブラウザーを使っているにもかかわらず、IME の入力モードが強制的に切り替わると、混乱する恐れがある。

また、そもそも文字種の変換は、ある程度クラウド側で対処できるはずだ。例えば、全角文字で保存しなければならないときに、それをユーザーに強要するのはナンセンスだ。半角で入力された文字は、クラウド側で全角文字に変換できる。

クラウド側で変換したくない事情があるなら、フロントエンド側で JavaScript

を使って変換できるだろう。システムの事情をユーザーに強要するのは、できる限り避ける設計を心掛けたい。

## Web ブラウザーの API 実装状況を確認

HTML5 の標準化は進んだものの、ターゲットとなる Web ブラウザーがそれを実装していなければ意味がない。対象となる Web ブラウザーが、どのような API を実装しているかを事前にチェックしておこう。

自分で実際に試すのもよいが、Web ブラウザーごとの実装状況をレポートしてくれる便利なサイトがある。どの API が利用できるかを手っ取り早く調べるには、「Can I use」がオススメだ。今、自分で使っている手元の Web ブラウザーがどれほどの API を実装しているかを網羅的に見たいなら、「HTML5test.com」が適している。

### ■ Can I use (<http://caniuse.com/>)

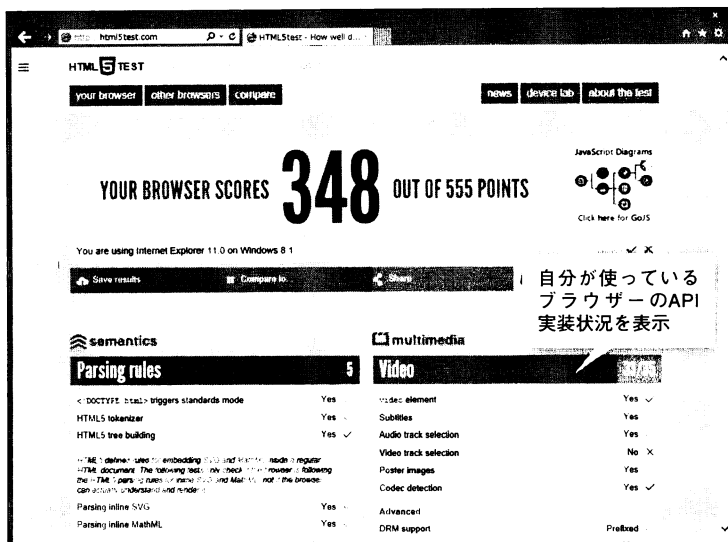
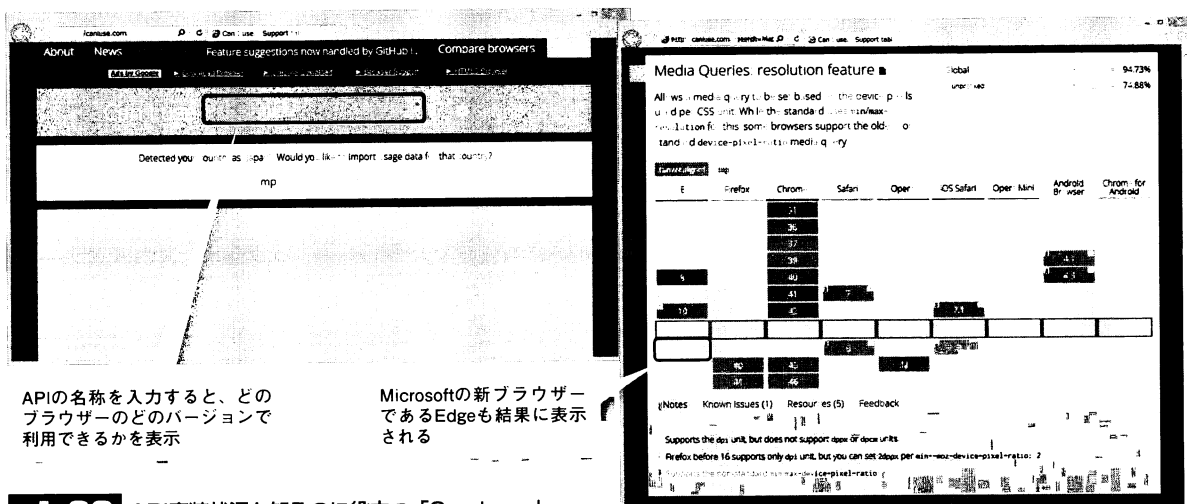
API 名称を入力すると、どの Web ブラウザーのどのバージョンでその API を利用できるのかが表示される (図 1-33)。利用に当たっての留意事項までレポートしてくれる。PC 版ブラウザだけでなく、スマートフォン向けブラウザも網羅している。また、Windows 10 の標準ブラウザとなる Edge の実装状況も分かる。

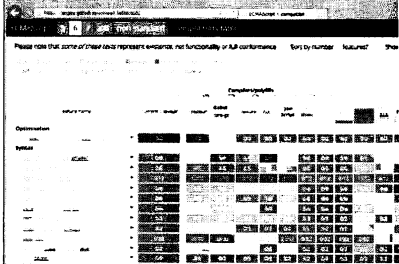
### ■ HTML5test.com (<http://html5test.com/>)

利用中の Web ブラウザーがどの API を実装しているかを表示する (図 1-34)。世界中の開発者が試した結果を見ることもできる。PC やモバイル向けブラウザのみならず、テレビやゲームコンソールに内蔵されたブラウザの結果まで分かる。

実際に Web ベースでフロントエンドを開発する上では、HTML5 をはじめとした Web 技術の API だけでなく、プログラミング言語である「ECMAScript」のサポート状況も知っておく必要がある。それには、「ECMAScript compatibility table」(<http://kangax.github.io/compat-table/es6/>) が役立つ (図 1-35)。







ECMAScriptのバージョンごとに、ブラウザーでの実装状況を表示

| Feature      | IE | Firefox | Chrome | Safari | Opera | Android | Windows Phone | BlackBerry | Kindle | Tablet |
|--------------|----|---------|--------|--------|-------|---------|---------------|------------|--------|--------|
| ECMAScript 5 | 10 | 10      | 10     | 10     | 10    | 10      | 10            | 10         | 10     | 10     |
| ECMAScript 6 | 11 | 10      | 10     | 10     | 10    | 10      | 10            | 10         | 10     | 10     |
| ECMAScript 7 | 11 | 10      | 10     | 10     | 10    | 10      | 10            | 10         | 10     | 10     |
| ECMAScript 8 | 11 | 10      | 10     | 10     | 10    | 10      | 10            | 10         | 10     | 10     |
| ECMAScript 9 | 11 | 10      | 10     | 10     | 10    | 10      | 10            | 10         | 10     | 10     |

図 1-35 ECMAScript compatibility table

ECMAScript 5 については、IE 9 をはじめ多くの Web ブラウザーで実装がほぼ終わっている。今後、気になるのは、最新バージョンである ECMAScript 6 の実装状況になってくる。

## 企業システムで注目の HTML5 関連の API

HTML5 をはじめとした Web 標準の API は、数え切れないほどある。ここでは、企業システムのフロントエンドに使えるような、「フォーム」「マルチメディア」「グラフィックス」「ストレージ」関連の API の概要と、使い道について解説する。どれも、IE 11 および Edge に実装されている。

### ■ フォーム

企業システムのフロントエンドでは入力機構が最も気になるだろう。HTML5 では、HTML4.1 や XHTML に比べて、フォーム関連の機能が大幅にグレードアップした。具体的には、テキスト入力フィールド、テキストエリア、セレクトメニュー、ラジオボタンに加え、検索、電話番号、URL、メールアドレスの入力フィールド、数値入力フィールド、スライダー（数値の範囲）、プログレスバーなどが追加された。これらはいずれも IE 11 や Edge で利用できる。Chrome や Firefox であれば、それ以外のフォームコントロールも利用可能だ。

ただし、いずれのコントロールも、IE 11 や Edge では、単なるテキスト入力フィールドにしか見えない。ところが、これをスマートフォンで見ると事情が変

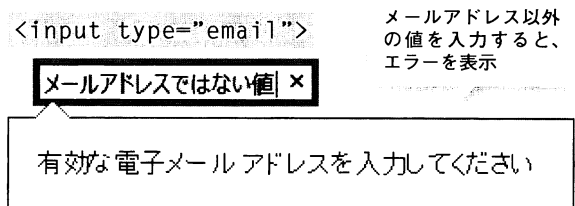
わってくる。例えば、数値入力のコントロールの場合、Android 標準ブラウザであれば、入力パッドが数値モードになる（図 1-36）。

数あるコントロールのうち、URL、メールアドレスについては、期待通りの値が入力されていないと、フォームを送信する際に Web ブラウザーがエラーを表



**図 1-36** 数値入力欄では入力パッドが数値モードになる

Android標準ブラウザで表示した場合



**図 1-37** メールアドレスの入力欄ではエラーを表示

```
<input type="text" pattern="¥d{3}¥-¥d{4}" required title="(例) 123-0001">
```

この例では、ハイフン付きの郵便番号を入力しないと、エラーが表示される

1230001 x

次の形式を使用してください: (例) 123-0001

**図 1-38** 入力値のパターンを指定できる

示する (図 1-37)。

コントロールの種類に応じた入力制限だけでなく、入力必須、値の最大値と最小値などもマークアップで指定できる。制約に反した値が指定された状態でフォームを送信しようとする、同様にエラーが表示される。さらに強力なのは、入力値のパターンを指定できる点だろう。正規表現を使って、入力可能な値のパターンを指定できる。例えば、正規表現でハイフン付きの郵便番号を入力させることを想定したテキスト入力フィールドの場合、パターンに一致しない値を指定するとエラーが表示される (図 1-38)。

HTML5 では、入力チェック用の「Form validation API」が規定されており、IE 11 や Edge では、この API を使って JavaScript から入力値が期待通りのものかをチェックできる。Web ブラウザーが用意したエラー表示を抑制して、自分でエラー表示を作り込めるのだ。

## ■マルチメディア

HTML5 では、video タグと audio タグが規定され、ビデオやオーディオを再生できるようになった。しかし、近年の Web ブラウザーは、単にメディアファイルを再生するだけでなく、ストリーミング配信やデジタル著作権管理 (DRM) もサポートしている。

ストリーミングは、「Media Source Extensions (MSE)」と呼ばれる仕様として W3C で標準化された。すでに、IE 11、Edge、Chrome、Firefox、Opera が

サポートしている。ただし、IE 11 は Windows8 以降でのみ利用可能だ。

MSE を使うことで、クライアントのネットワーク状況に応じて動的にビットレートを変えたり、途中で別のビデオコンテンツに切り替えたりできるようになる。また、ユーザーにビデオコンテンツをファイルとして保存されたくない場合にも MSE は有効だ。

DRM は、「Encrypted Media Extensions (EME)」という名前で標準化された。すでに、IE 11、Edge、Chrome、Firefox、Opera がサポートしている。EME は、DRM そのものをサポートするのではなく、Web ブラウザーがサポートする DRM とビデオをつなぐためのフレームワークである。そのため Web ブラウザーによって、対応する DRM の種類が異なる点に注意が必要だ。

MSE も EME も、ビデオ配信サーバーを自社で用意するのは現実的ではない。さまざまなクラウドサービスがあるが、IE 11 や Edge であれば、Azure Media Services と組み合わせるのがよいだろう。

## ■グラフィックス

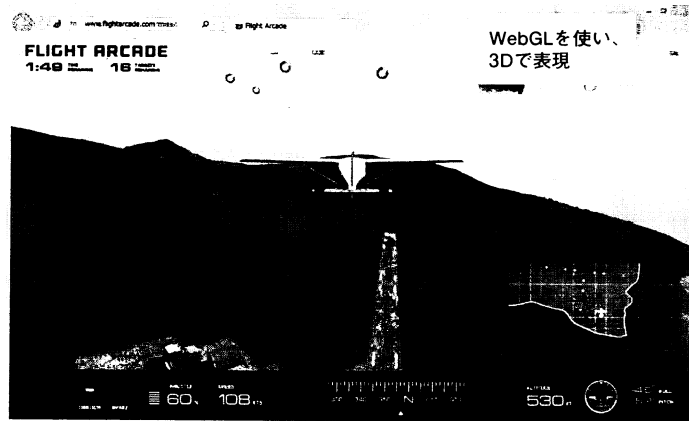
HTML5 の 2D グラフィックスを実現する「Canvas」は、すでに IE 9 および他の主要な Web ブラウザーに実装されている。3D グラフィックスを実現する「WebGL」も、IE 11 も含め、主要な Web ブラウザーには実装されている。

Microsoft は、IE や Edge で新たにサポートした API を見せるために、「Test Drive」というデモサイトを用意している。ここには、WebGL を使ったいくつかのデモが公開されている。中でも、「Flight Arcade」と呼ばれるフライトシミュレーターは圧巻だ (図 1-39)。

企業システムのフロントエンドでここまで凝ったコンテンツを作ることはないだろうが、3D 表現はさまざまなシーンで求められるだろう。

## ■ストレージ

ストレージの仕組みは、企業システムのフロントエンドアプリケーションには不可欠なものだろう。HTML5 には Web ブラウザー側にデータを保存する仕組みとして、「Web Storage」「Indexed Database API」「File API: Directories and System」「Web SQL Database」の四つがある。



**図 1-39** WebGLを使ったフライトシミュレーター「Flight Arcade」

MicrosoftのWebサイト「Test Drive Sites and Demos」(<http://dev.modern.ie/testdrive/>)に掲載されている

Web Storage は、キーバリュー型の比較的小さなデータを保存する仕組みだ。Indexed Database API はオブジェクトストレージの一つで、さまざまなデータ構造をオブジェクト形式で保存できる。テキストデータだけでなく、バイナリーデータも保存できるため、画像などのファイルのデータを保存するためにも使われる。File API: Directories and System は、Web ブラウザーが管理する仮想的なストレージ領域に、ディレクトリー階層を作ったり、その中にファイルを保存したりする仕組みを提供する。Web SQL Database は、名前の通り、SQL を使ってデータを扱う仕組みを提供する。

このうち、File API: Directories and System と Web SQL Database は、W3C では廃止扱いになっているが、Chrome、Safari、Opera には今でも実装が残されており、今後も使い続けられる。Web Storage と Indexed Database API は、近年のメジャーな Web ブラウザーすべてに実装されている。今後は、これら二つを用途に応じて使い分けることになる。

Web ブラウザー側に保存されたデータを暗号化する仕組みも用意されている。「Web Cryptography API」は、暗号論的疑似乱数列の生成、暗号学的ハッシュ値の生成、公開鍵と秘密鍵の生成、デジタル署名の生成と検証、暗号化と復号と

いった基本的な暗号処理系の機能を提供する。IE 11、Edge、Chrome、Firefox、Opera は実装済みだ。

## 標準技術を使ったフロントエンド開発にかじを切る

これまでの解説を通して、企業システムのフロントエンドを HTML5 ベースで開発できることは実感できたのではないだろうか。もちろん、デスクトップアプリや ActiveX といった企業独自技術を使った場合と比べると、かゆいところに手が届かない部分もあるだろう。しかし、HTML5 ベースでフロントエンドを開発するメリットは、そういったデメリットを上回る。

近年、企業側もマルチデバイスを求めるようになった。また、IT の重要性を理解した企業であれば、OS のアップデートも比較的早く、最新の Web 技術を駆使する環境が整い始めている。これまでの企業システムに見られがちな、OS と Web ブラウザーのバージョンが固定されることを前提としたシステム開発は、徐々に時代遅れになっていくことは間違いない。標準化技術を使って、マルチデバイス、マルチブラウザーをある程度許容し、OS のバージョンアップの足かせにならないフロントエンド開発が望まれている。

### 1-5 のポイント

---

- HTML5によって配布やバージョンアップの手間が省け、運用コスト削減に貢献する。
- HTML5はデスクトップアプリに比べ、機能面ではおよばないが、多くのシーンでは十分な機能が備わっている。
- 顧客企業が望むマルチデバイス対応や、OSの環境に依存しないフロントエンドの開発にはHTML5が有望。



## 1-6 HTML5 の未来

# 独自の機能拡張も可能に 永久にベータ版として進化

HTML5 をはじめとした Web 技術はどこへ向かうのだろうか。

そして、今後も使い続けられるのだろうか。

HTML5 の今を分析し未来を考察する。

HTML5 は、PC のデスクトップアプリやスマートフォンのネイティブアプリと比べて機能面で不足があるものの、企業システムで採用する技術として機は熟してきた。とはいえ、企業システムはライフサイクルが非常に長いいため、これまでと異なる技術を採用することに対しては慎重にならざるを得ないだろう。

とりわけ、新たに採用した技術が将来どうなるのかについては気になるところだ。HTML5 をはじめとした Web 技術の標準化のトレンドと、今後の姿を考察する。

### HTML 標準化の二つのモデル

2014 年、標準化団体の W3C によって HTML5 仕様が勧告となり、大きな話題になった。W3C に限らず、多くの標準化団体では、「スナップショットモデル」と呼ばれる方式を採用している。

スナップショットモデルでは、ある時点で完成を宣言し、その仕様の内容を固定する。W3C の場合、最初にワーキングドラフトという形で素案の提案があり、その後議論と改良を繰り返し、草案から勧告候補、勧告案、そして勧告に至る。勧告になるためには、少なくとも二つ以上の Web ブラウザーに実装され、すでに使える状態にあることが条件となる。

このスナップショットモデルの特徴は、その後に機能追加などを実施する際にはバージョン番号を付け、新たな仕様とする点だ。HTML5 の 5 は、まさにこのバージョン番号である。ところが、Web ブラウザー上で JavaScript から利用で

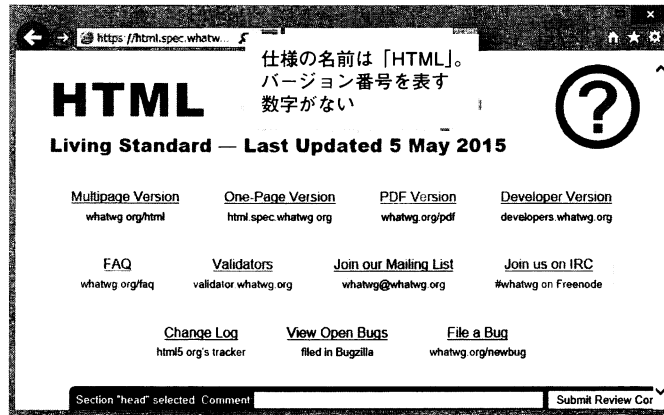


図 1-40 WHATWGのHTML仕様

ブラウザベンダーが主体となって設立したWHATWGは、HTML5仕様のベースとなった仕様を更新し続けている。https://html.spec.whatwg.org/multipage/

きる API については、このスナップショットモデルから脱却した団体がある。ブラウザベンダーで構成する業界団体「WHATWG (Web Hypertext Application Technology Working Group)」だ。

WHATWG は 2004 年にノルウェーの Opera Software、米 Mozilla、米 Apple が設立した。実は、W3C の HTML5 仕様は、元をたどれば WHATWG が作成した仕様である。W3C では HTML5 仕様は完成し次のバージョンを策定しているが、WHATWG では今なお、HTML5 仕様のベースとなった仕様を更新し続けている。その仕様の名前は「HTML」(図 1-40)。バージョン番号を表す数字の 5 がない。

このような標準化の方式を、「リビングスタンダード (Living Standard)」という。分かりやすいいえば、「永久にベータ版」ということだ。仕様に問題が見つかった場合の修正や、新たな機能の追加は随時行われる。W3C の HTML5 仕様は、ある意味、WHATWG の HTML 仕様の特定の時期におけるスナップショットなのだ。もちろん、W3C で独自に機能の追加や削除、修正が行われているため、まったく同じではない。

WHATWG では、まだ W3C の標準化トラックに乗っていない機能なども先行

して標準化している。実験的にブラウザーベンダーがその機能を実装することすらある。また、WHATWG で策定された機能は、その後に、W3C の標準化トラックに乗り、勧告に至るものが多い。

## HTML5 の次はどうなるのか

W3C の HTML5 仕様が勧告になったからといって、これで Web の進化が終わりというわけではない。W3C はすでに HTML5.1 仕様の策定に取り掛かっている (図 1-41)。これは、おそらく WHATWG の HTML 仕様をキャッチアップしたものになる。HTML5.1 が勧告に至れば、続いて HTML5.2 の策定が始まる。

Web アプリケーションを開発する上では、HTML5 仕様だけでなく、その他のさまざまな API も必要だ。こういった API についても、次々に新たな機能が提案され、標準化トラックに乗ってきている。

今、W3C で話題になっているのは、「Extensible Web」という概念である。これは、開発者が独自の機能を作り込める、つまり仕様を拡張可能にするという考え方だ。

これまで API の標準化では、ユースケース (利用シーンなど) を想定して、

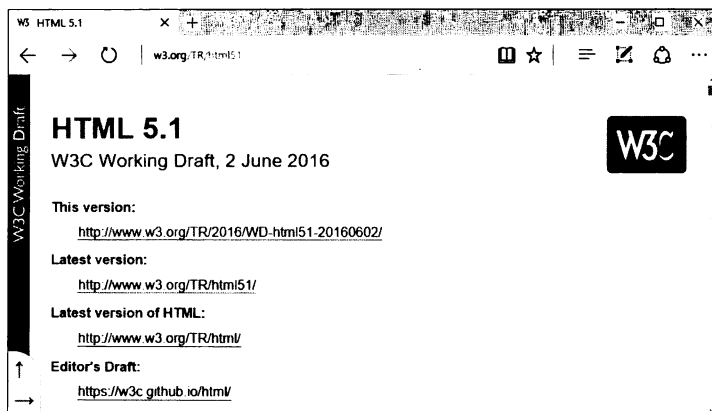


図 1-41 W3C が策定中の HTML5.1 仕様草案

<http://www.w3.org/TR/html51/whatwg.org/multipage/>

開発者ができる限り簡単に特定の機能を利用できるように API が設計されてきた。ただし、ユースケースというものは、すでに存在しているもの、または現時点で想像したものにも過ぎない。実際には、時代が進むにつれ、当初は想定できなかったユースケースが現れることもある。

こうした事態に対応すべく、API を低レイヤーで設計し、機能を拡張しやすくする流れが出てきた。例えば、HTML にはタグが規定されている。これまでは仕様で定義されたタグしか使えなかったが、API が低レイヤーで設計されていることで、開発者は自分で独自の機能を持ったタグを作れるようになる。すでに「Custom Elements」や「Shadow DOM」といった拡張可能な仕様が作成されており、一部の Web ブラウザーには実装されている。

このように、これまで Web アプリ開発者が関わるのができなかったレイヤーまで API を用意して、自由な開発環境を提供する動きが出てきた。筆者は、この流れは必然と考えている。以前は JavaScript でやることといえば、Web ページを多少便利にする程度だった。しかし近年は、PC のデスクトップアプリやスマートフォンのネイティブアプリと同じようなことを Web ブラウザー上で実現したいというニーズが高まっている。このニーズに応えるための技術が生み出されているわけだ。

今後、JavaScript を使った Web アプリケーションは、OS が持つ機能を提供できるようになっていき、ますます高度化、複雑化していくことになるだろう。

## HTML5 の実装が進む Web ブラウザー

Web ブラウザーの HTML5 対応を取り巻く状況は大きく様変わりした。特に変化が大きいのは Internet Explorer (IE) の HTML5 への対応だ。Opera、Mozilla、Apple、Google は以前と変わらず HTML5 を推進し、新たな API を積極的に実装している。ここに米 Microsoft も加わった。

特に、Microsoft は積極的に情報を公開するようになった。まず、Windows 8 以降、かなり早い時期からプレビュー版を開発者向けに提供するようになり、開発者はいち早く IE と Edge の API 実装状況を把握できるようになった。Web アプリ開発者にとって最も重宝するのは、API 実装状況を掲載している「Platform

Status」だろう（図 1-42）。

Chrome のベースとなる Chromium も同様のページ「Chromium Dashboard」を公開している（図 1-43）。Chrome は、他の Web ブラウザーと比べても早期に Web 標準の API を実装し始める場合が多い。今後、こういった機能が実装されるかを知るために見ておくとよいだろう。

## Web ランタイムも大きく進化

スマートフォンでは、ハイブリッドアプリ開発において、Web ランタイム（WebView）上でコンテンツを実行する際に、HTML5 をふんだんに使うことになる。WebView は Web ブラウザーの機能をネイティブアプリから利用できるようにする仕組みだ。

これまで WebView は、OS のバージョンで機能が固定されていたのが難点だった。iOS は、ある程度は過去の iPhone や iPad でも OS のバージョンアップをサポートしており、多くの市場の端末は、最新にバージョンアップされる傾向がある。しかし Android OS は、通信キャリアや端末メーカーが Android のバージョンアップに積極的でないこともあり、古いバージョンのものが一定期間残ってしまう。

しかし、この問題も今後は解決に向かいそうだ。Google は、Android 5.0（Lollipop）以降、WebView を Android OS から切り離し、単独で Google play ストア経由でアップデートできる仕組みに変更した（図 1-44）。旧来の WebView は、米 Apple が Safari に採用している WebKit と呼ばれるオープンソースの Web ブラウザーがベースになっていた。一方 Chrome は、Chromium と呼ばれるオープンソースがベースになっている。Chromium はもともと WebKit から派生したプロジェクトだが、今は WebKit とは別物といってもよいほどに進化を遂げている。

新たな WebView は、Chrome と同じ Chromium ベースになっており、旧来の WebView と比べてパフォーマンスに優れ、さらに Chrome でサポートされる新たな API も、追隨して利用できる。つまり、Android OS のバージョンアップがなくても、最新の Chrome の機能を WebView 上で利用できるようになる。

このほか、WebView に相当する Web ランタイムをアプリに同梱してしまう方

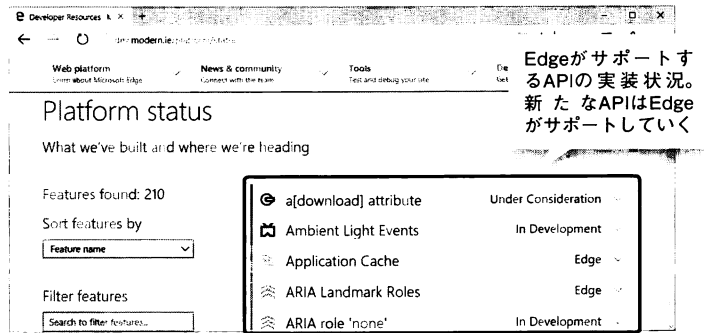


図 1-42 EdgeのAPIサポート状況を示すPlatform Status

<http://dev.modern.ie/platform/status/>

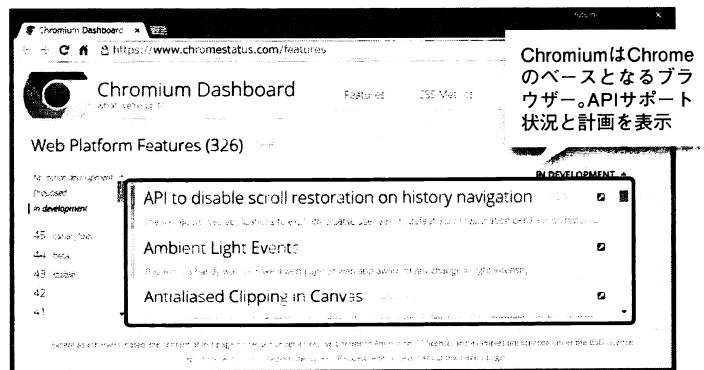


図 1-43 Chromiumの「Chromium Dashboard」

<https://www.chromestatus.com/features>

法がある。「Crosswalk」と呼ばれるプロジェクトは、ハイブリッドアプリ開発環境向けに、Chromium をベースに Web ランタイムを提供している (図 1-45)。もちろん、Web ランタイムとなる Crosswalk が一つのアプリ内に含まれるため、アプリのサイズは大きくなるが、Android OS のバージョンに関わらず統一されたランタイム環境を手に入れられる。「1-4 モバイルアプリ開発」で紹介した Apache Cordova は、この Crosswalk をサポートしている。



WebViewをAndroid OSから切り離して、単独でGoogle playストア経由でアップデートできるようにした

図 1-44 Google playストアでのAndroidシステムのWebView



図 1-45 Webランタイムを提供するCrosswalk

<https://crosswalk-project.org/>

## アプリ基盤として成長する HTML5

これまでは、Web ブラウザー上で動作する Web アプリと、ハイブリッドアプリ開発における WebView 上で動作するスマートフォンアプリという側面で、HTML5 を紹介してきた。実は、HTML5 はそれ以外の領域にも入り始めている。

まず、HTML5 ベースのアプリ基盤を備えている OS から見ていこう。Mozilla

が開発した Firefox OS は、Web ブラウザーである Firefox をベースにアプリ基盤を構成している。Firefox OS 上で動作するアプリは、すべて HTML5 ベースで開発しなければならないが、Web 標準の API だけではスマートフォンアプリに求められる機能が不足している。そのため、Firefox OS は、さまざまな機能を拡張している（図 1-46）。Bluetooth、TCP Socket、Telephony（電話発着信用の API）など、通常の Web ブラウザーでは実装されていない API が多数用意されている。

Chrome OS も同様だ。日本ではまだ Chrome OS はほとんど普及していないが、北米ではそれなりの存在感が出てきている。Chrome OS のアプリも、すべて HTML5 ベースで開発するわけだが、さまざまな機能が拡張されている。アプリケーション基盤としては十分な機能を持つ。

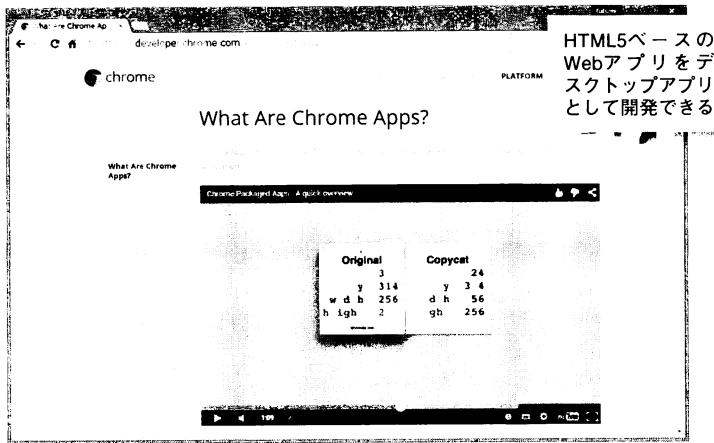
Windows 8 から導入された Windows ストアアプリも、HTML5 ベースのみで開発可能だ。Web ランタイムは IE 10 または 11 ベースだが、Web ブラウザーとしての IE では使えない機能が相当盛り込まれている。JavaScript のみで、Windows が提供する機能のほとんどを利用できる。



図 1-46 Firefox の WebAPI

<https://developer.mozilla.org/ja/docs/WebAPI>





**図 1-47** HTML5ベースのアプリをパッケージ化するChrome Apps

<https://developer.chrome.com/apps/>

PC のデスクトップアプリにも HTML5 は浸透し始めている。Google の「Google Chrome Apps」は、HTML5 ベースで開発した Web アプリをパッケージ化して、インストール型アプリに仕立て上げるプラットフォームだ（図 1-47）。Chrome Apps を使うためには Chrome をインストールする必要があるが、Chrome Apps の利用時は Web ブラウザーとしての Chrome のウィンドウは表示されず、単独のデスクトップアプリとして表示される。もちろん、Chrome Apps をインストールすると、Windows のスタートメニューにも登録され、ユーザーから見れば、ネイティブのデスクトップアプリと変わらない。企業システムのフロントエンドとして、Web ブラウザーを利用するのがためらわれる場合には、このようにデスクトップアプリとして仕立て上げるのもよいだろう。

## コンパイル型のアプリ開発でパフォーマンス改善

HTML5 がさまざまな領域に浸透し、高度化するにつれ、パフォーマンスの問題が顕著になってくる。すでに、各ブラウザーベンダーは、JavaScript エンジンやレンダリングエンジンをチューニングし、高速化に取り組んでいる。しかし、

OS ネイティブのアプリと比較すると劣るのが実情だ。

解決策の一つとして有望なのが、Web アプリケーションの一部を C/C++ で開発してコンパイル型のアプリケーション開発を可能にする仕組みである。これに最も早く取り組んだ Google は、「Native Client (NaCl)」「Portable Native Client (PNaCl)」を発表している。NaCl は CPU に依存するため、CPU ごとにコンパイルが必要だが、PNaCl は「LLVM 中間言語コード」を採用することで、CPU に依存せず使い勝手がよい。

これらはいずれも、コンパイル型の仕組みによって、JavaScript というスクリプト言語のパフォーマンスにおけるデメリットを補ってくれる。しかし残念ながら、NaCl や PNaCl はあまり普及しているとはいえない。やはり、Chrome でしか利用できない点が普及の妨げになったといえるだろう。

注目は、Google と Microsoft、Mozilla、WebKit プロジェクトのエンジニアたちが 2015 年 6 月に発表した、「WebAssembly」と呼ばれるコンパイル型の仕組みだ (図 1-48)。WebAssembly は HTML5 を置き換えるわけではない。ほとんどを WebAssembly で開発することも可能かもしれないが、実際には JavaScript から呼び出せるライブラリとして利用されることが多いだろう。

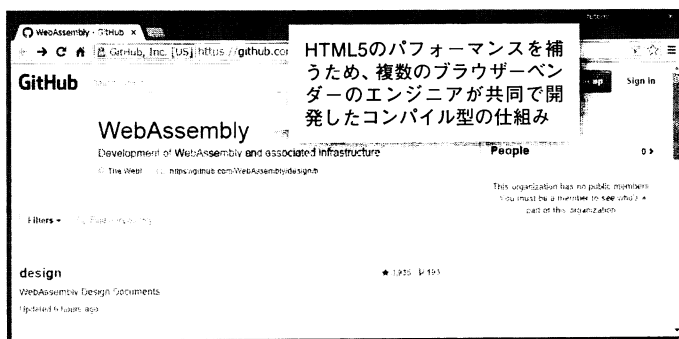


図 1-48 GitHub上のWebAssemblyのページ

<https://github.com/WebAssembly>

## 「どこまでできるのか」を知ることが重要

よく HTML5 とネイティブアプリは対立軸で語られることがあるが、もはやそうした次元ではないことを理解していただけたらだろうか。

HTML5 をはじめとした Web 技術は、ネイティブアプリにもデスクトップアプリにも使われ、さらには PC やスマートフォンだけでなく、テレビ、自動車のIVI（カーナビなど）、デジタルサイネージ、IoT の分野にまで幅を広げようとしている。あらゆる ICT の分野に、ブラウザーのベースとなる Web ランタイムが介在する時代は、すぐそこにまで迫ってきている。

こういった Web 技術を採用する利点の一つは、同じ技術でさまざまな分野にビジネスを広げられる点といえる。そのために必要なのは、現時点で HTML5 をはじめとした Web 技術で「どこまでできるのか」を具体的にイメージできること。つまり「目利き」となる知識を蓄えることだ。これまで解説してきたとおり、HTML5 で何でもできるわけではない。

## HTML5 は息の長い技術になる

次に、技術的ノウハウについて触れよう。特に、システム開発受託を中心とする会社において、フレームワークやライブラリといった汎用的な部品となるコード資産をどれだけ保有しているかが重要なことは言うまでもない。おそらく読者が所属する会社にも、秘伝のタレともいえるような独自のライブラリがいくつも存在するだろう。こういった資産によって、短期間に高品質なシステム提供が可能となる。今後はそこに JavaScript のコードも加わることになるだろう。

とはいえ、何でもかんでも自前でコードを作る必要がないのも、Web の世界の大きな特徴だ。これまで紹介してきたとおり、数多くの優秀な MVC フレームワークが存在している。また、画面表示に必要なユーザーインターフェースに関する JavaScript ライブラリも数多く存在する。

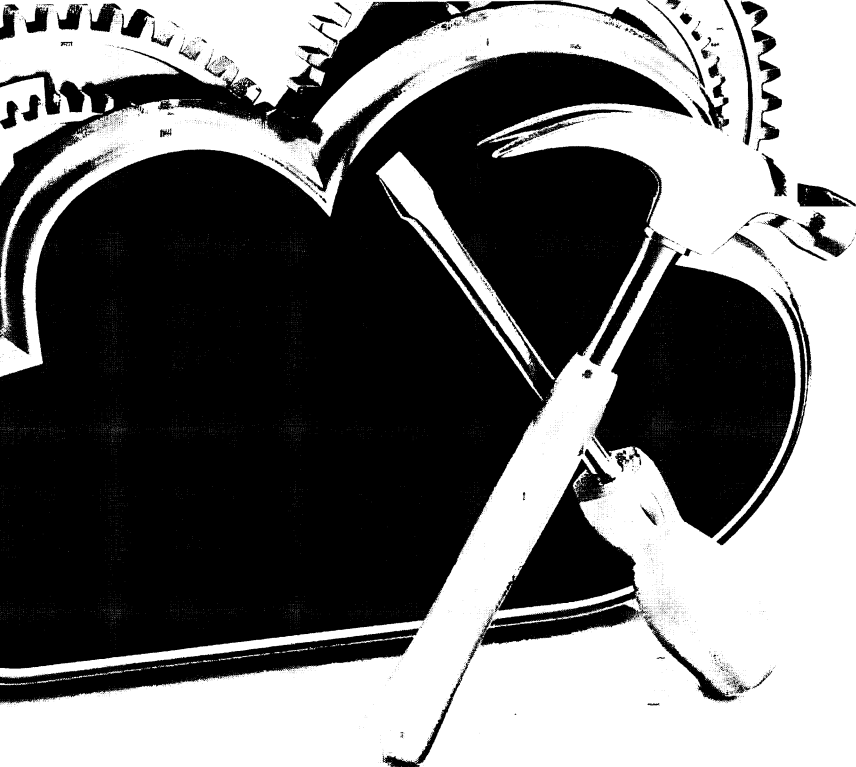
こういった部品をゼロから開発するのは、コストが増えるだけでなく、品質を維持するのも難しいだろう。世界的に有名となった JavaScript ライブラリは、世界中の開発者からのフィードバックを受けて日々改良され続けている。たかだか

1社の力で、同じレベルの品質を持った JavaScript ライブラリを短期間で開発するのは至難の業だ。できる限り車輪の再発明を避け、既存の JavaScript ライブラリを使いこなすノウハウや、細かい点のカスタマイズのノウハウを蓄積するほうが効率的だ。

今後、HTML5をはじめとした Web 技術は、ICT の世界にいる以上、避けては通れない。企業独自の技術と比べて息が長い技術となるだろう。

### 1-6 のポイント

- HTML5は永久にベータ版であり進化し続ける。
- HTML5のAPIはどんどん高度化し、ネイティブアプリに近づく。
- あらゆるICTの分野に、ブラウザのベースとなるWebランタイムが介在する時代になる。



## 第 2 章

---

### HTML5 の基本を学ぶ

|                          |     |
|--------------------------|-----|
| 2-1 マークアップとスタイリング .....  | 84  |
| 2-2 入力とユーザーインタフェース ..... | 97  |
| 2-3 マルチメディア .....        | 110 |
| 2-4 ハードウェアへのアクセス .....   | 123 |
| 2-5 ネットワーク .....         | 136 |
| 2-6 標準化提案中の API .....    | 149 |

## 2-1 マークアップとスタイリング

# 大きく変わった要素の意味 現状に即した使い方へ変化

企業システムでも、もはや避けては通れない必須知識となった HTML5。Web 技術の基礎となるマークアップとスタイリングは、HTML5 でどのように変わったのだろうか。まずは、HTML5 時代のマークアップとスタイリングの基本を解説する。

2014 年、HTML5 仕様が標準化団体 W3C での「勧告」となり、話題になった。もちろん、それ以前から HTML5 は Web 業界で注目されており、実践でも使われてきた。

世間一般でいわれる「HTML5」には、勧告に至った HTML5 仕様だけでなく、Web ブラウザー上で JavaScript から利用できる API や、スタイリングのための CSS (Cascading Style Sheets) といった技術も含まれる。このため、HTML5 は Web 技術全般の進化という意味合いで話題になってきた。

とはいえ、Web 開発を専門としていない読者にとって、HTML5 が旧来の Web と何が違うのかを説明するのは難しいだろう。HTML5 で何ができるようになったのかを具体的に理解していなければ、当然 HTML5 を説明できない。第 2 章では、「なんとなく Web が進化した」というレベルの理解から、具体的に何ができるのかを理解できるレベルへ到達するための道しるべとして、HTML5 の基礎知識を解説していく。まずは、HTML5 の基礎であり、多くの人に最もなじみがある HTML マークアップと CSS を取り上げる。

### HTML5 で強化された「セマンティクス」

まずは HTML5 でのマークアップの変化について見ていこう。HTML は、ドキュメントのコンテンツに対して意味付けを行うマークアップ言語である。例えば、

```
<h1>...</h1>
```

と書けば「見出し」という意味を持つ。

もともと HTML は、論文などのドキュメントを電子化するために考案されたという経緯もあり、近年のようなエンターテインメントや商業向けのコンテンツを考慮して作られてはいなかった。そのため、コンテンツのマークアップではボキャブラリーが不足していた。

そこで HTML5 では、ブログやショッピングサイト、SNS といった、近年では当たり前でコンテンツで適切な意味付けができる要素（タグ）を数多く追加した。図 2-1 は、ブログサイトの典型的なブログ記事ページの文書構造だ。これまではセマンティクス（意味）を持たない div 要素を使って構造化してきたが、HTML5 では article 要素、section 要素、nav 要素、aside 要素、header 要素、footer 要素などが追加された。このようなセマンティクスの強化が HTML5 の大きな特徴である。

セマンティクスが強化されるとどのようなメリットがあるのだろうか。そもそも、CSS でスタイリングしてしまえば、HTML に意味を持たせなくても、どの部分がどんな意味を持つのかは、人にはすぐに理解できる。それに関わらず、HTML5 でセマンティクスが重要視されたのは、Web コンテンツは人が見るだけでなく、コンピュータも扱うようになったからだ。

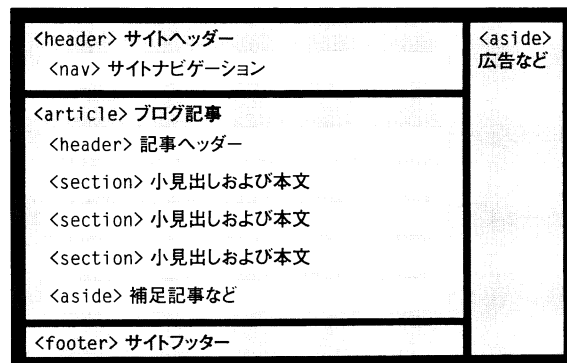


図 2-1 HTML5 を使ったブログ記事ページの文書構造の例

例えば、コンテンツが検索結果の上位に出るように最適化する「SEO」(Search Engine Optimization) という技術がある。検索結果の表示では、検索エンジンなどのロボットが世界中の Web コンテンツをクロールするのだが、当然、こういったロボットは Web ブラウザーで表示された結果を見ているわけではない。ロボットが理解できるのは、HTML 上に記述された本文とタグだ。ロボットは、テキスト情報の重み付けをタグなどから判別する。

こうした状況に合わせ、Web コンテンツ制作者は HTML を書くときの作法に注意する必要がある。例えば、画像の使い方だ。Web サイトによっては、ページのコンテンツのほとんどを画像で作り込んだサイトもあるが、基本的に、ロボットは画像の内容を理解できない。このため、写真や図などは画像を使うしかないが、単にフォントが気に入らないという理由だけで、見出しや本文まで画像にってしまうのはやめるべきだろう。どうしても文章を画像にしなければならない事情があるなら、必ず、img 要素の alt 属性に、そのテキスト情報を入れるべきだ。

```

```

このように、できる限り文字情報を HTML 内に注記する。こういったガイドラインのような規定も、HTML5 仕様には詳細に書かれている。

## アクセシビリティを考慮してタグを使う

アクセシビリティという観点でも、セマンティクスは重要だ。例えば、目が見えない人が Web コンテンツにアクセスする際には、スクリーンリーダー（音声読み上げソフト）を使う。もちろん、スクリーンリーダーの種類によって、セマンティクスをどこまで考慮するかには違いがあるが、HTML 上でタグを適切に使うことで、将来的にはタグの意味に応じた読み上げ方をすることも考えられるだろう。

アクセシビリティに関わる HTML5 仕様の大きな特徴の一つに「WAI-ARIA」との連携が挙げられる。WAI-ARIA とは、Web アプリケーションに対して、タグ以上の意味をコンテンツに与えるための枠組みだ。例えば Web アプリケーショ



ンでは、ボタンを表すために必ずしも button 要素や input 要素を使うとは限らない。場合によっては、div 要素や span 要素を使ってボタンを表すこともある。しかし、スクリーンリーダーは div 要素や span 要素をボタンだと解釈できない。スクリーンリーダーがボタンと解釈できない以上、スクリーンリーダーの利用者はボタンを押せない。

このような課題を解決するために、WAI-ARIA は属性という形でタグに意味を付け加える。

```
<span role="button" aria-pressed="false">ボタン</span>
```

この例は、span 要素をボタンとして意味付けているが、さらに状態も指定している。ここではボタンは押されていない状態であることを示している。

WAI-ARIA は、ユーザー操作に関する意味付けだけでなく、文書構造に関する意味付けなども定義している。次の例は、div 要素が、該当のドキュメントの中でメインとなるコンテンツであることを表している。

```
<div role="main">メインコンテンツ</div>
```

WAI-ARIA は、アクセシビリティにおいては非常に重要だが、Web コンテンツ制作者に普及しているとはいえない。しかし、HTML5 では、あらゆる要素に対して WAI-ARIA を関連付けているため、WAI-ARIA を使わなくても、HTML5 の要素で表せるセマンティクスの範囲内であれば、スクリーンリーダーに意味を正確に伝えられる。制作者は、できる限り HTML5 仕様に規定された通りに、適切に要素を選定して HTML コンテンツを作るべきだ。

アクセシビリティは目に見えるものではないため、学習コストや制作コストを考えると、なかなか制作の現場では浸透しないのが実情だ。だが発注側の意識は異なる。筆者は近年、大手企業が発注する Web サイトの制作では、セマンティクスやアクセシビリティの意識が高まっていることを実感する。納品検査の際に、HTML のマークアップまでチェックし、適切にセマンティクスを考慮しているかを確かめるケースが多くなっている。もはや、目に見えないからといって、

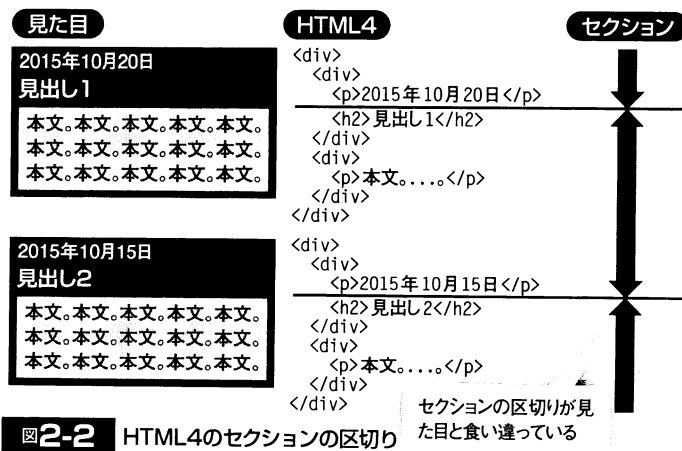
セマンティクスをないがしろにはできない状況にある。

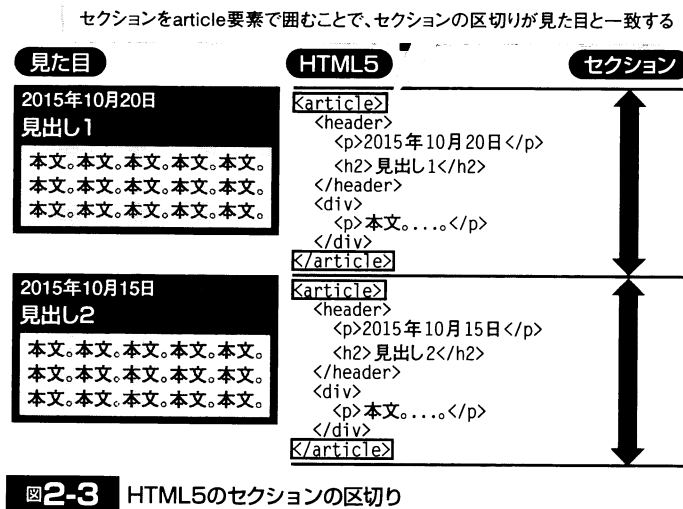
## セクションの概念で区切りが明確に

次に、HTML5 に新たに導入されたセクションという概念について解説しよう。セクションとは、日本語で言い換えると、「章」や「節」のことである。HTML4 や XHTML1.0 では h1 ～ h6 要素が規定されており、見出しとして使われてきた。この見出しによって、セクションが始まると解釈されてきた。正確にいうと、そう解釈するしかなかったのだ。

論文のような文書構造なら大きな問題はないのだが、ブログ記事など、近年ではごく一般的な Web ページだと支障が出る。図 2-2 は、ブログ記事を想定したものだが、見た目では二つのセクションが存在することが分かるだろう。

ところが、セマンティクス上は、セクションの区切りが見た目と食い違っている。この二つのセクションはそれぞれ div 要素で囲まれているが、div 要素そのものにはセクションという意味はない。そのため、セクションの区切りを推測することはできても、それが本当に正しいセクションの区切りかは、はっきりしないのだ。もしプログラムを使って HTML からセクションを切り出そうとしたら、かなり苦勞することは想像に難くない。





では、HTML5でマークアップするとどう変わるだろうか（図2-3）。見た目は同じだが、セクションを article 要素で囲んでいることに注目してほしい。article 要素は、セクションというセマンティクスに加え、独立して成り立つコンテンツというセマンティクスを持つ。主にブログ記事を囲むのに使われる。このように、明示的にセクションというセマンティクスを持った article 要素を使うことで、機械的にセクションを抽出できるようになる。

セクションというセマンティクスを持った要素は他にも規定されている。ナビゲーションを表す nav 要素、本題と関係が薄いコンテンツを表す aside 要素（補足記事、広告など）、これら以外の特別な意味を持たない一般的なセクションを表す section 要素だ。コンテンツを作る際は、これらの意味を理解して使うようにしたい。

## 要素の意味を変更しスタイリングを排除

HTML5では、HTML4に規定された旧来の要素の意味を、一部変更している。ここでは、意味が変更された要素をいくつか紹介する。

small 要素は、以前は小さい文字で表示するスタイリング要素であり、セマンティクスを持ち合わせていなかった。HTML5 では、著作権表記や免責事項といった細目というセマンティクスが与えられた。em 要素と strong 要素は、以前はいずれも強調というセマンティクスを持っていたが、HTML5 ではそれぞれ別々の意味が与えられ、em 要素は強調、strong 要素は重要性を表すことになった。

このように、HTML5 はこれまでスタイリング目的にしか使われなかった要素を廃止したり、新たな意味（セマンティクス）を与えたりした。そして、重複する意味を持った要素には、一方を廃止するか別の意味を与えた。HTML5 仕様で規定された要素はすべて、何かしらの意味を持ち、スタイリングという役割を一切排除したのだ。

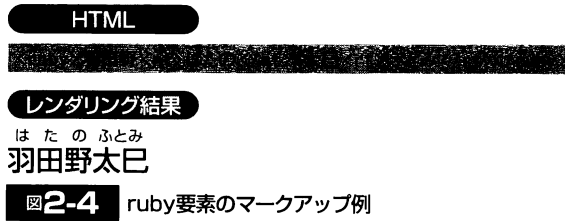
## 国際化する HTML5 マークアップ

基本的に HTML は、英語圏を中心に考案されてきた。HTML5 でも、英語圏の事情が影響している部分がある。

分かりやすいのは i 要素だ。HTML4 での i 要素は、イタリック体で表示するためのスタイリング要素だった。しかし、HTML5 では、分類名称、技術用語、別の言語の慣用句的なフレーズ、船の名前に使われることになった。日本人から見ると、意味が分からない規定だと思うだろう。実は、英語圏の印刷慣例では、イタリック体で表記する対象は決まっている。その慣例に沿った形で i 要素を使うことになったのだ。

一方で、英語圏だけでなく、他の言語でも役に立つような改良も加わっている。まずは中国語だ。u 要素は、HTML4 では下線を引くためのスタイリング要素だった。そのため、u 要素は HTML5 仕様の策定段階で、いったんは廃止扱いになった。ところが、中国語圏では、特定の種類の単語（人名や地名や組織名といった固有名詞）を他の部分と区別するために下線を引く慣例があり、それを u 要素に割り当てることになった。

ただし実際には、どの言語でも使えるよう、そのセマンティクスは一般化され、前後の文章と区別しないと分かりにくい表記や、本来の表記ではないことを表す（意図的にスペルミスの単語を記載する）場合に使うことになった。日本語でも、



固有名詞がひらがなの場合、前後の文字もひらがなだと読みにくいことがあるだろう。こういう場合に、それをu要素で囲み、CSSを使ってスタイリングするとよいだろう。見た目はCSSを使って、下線ではなく別のスタイルを当てればよい。

次に日本語だ。HTML5には、新たにruby要素が加わった（図2-4）。名前の通り「ルビ（ふりがな）」だ。実は、Internet Explorerはかなり以前からルビには対応していたが、米Microsoftの独自規格であったことと、他のWebブラウザは対応していなかったことから、あまり普及していなかった。だが現在は、標準化された要素であり、多くのメジャーなWebブラウザがruby要素を実装したため、今後は利用度が上がっていくだろう。

このほか、アラビア語などの右から左に向かって流れる言語に必要な要素なども、新たに追加している。

## 充実したマルチメディア関連

HTML5のマークアップで最も注目されたのは、マルチメディア関連の要素だ。HTML4までは、ビデオの再生やオーディオの再生、グラフィックスの動的生成は、Flashなどのプラグインに頼るしかなかった。HTML5では、ビデオ再生ならvideo要素を、オーディオ再生ならaudio要素を、グラフィックスの動的生成はcanvas要素を使えるようになった。

さらに、ビデオに関しては、字幕情報を再生中のビデオに連動して表示するtrack要素も加わった（図2-5）。これにより、プログラミングなしに字幕を入れられるようになった。字幕そのものは、WebVTTと呼ばれるフォーマットで書かれたテキストファイルとして用意する。track要素がそれを読み込んで、ビデ



図2-5 track要素で字幕を表示するデモ

HTML5 Rocks「track要素の基礎」(<http://www.html5rocks.com/ja/tutorials/track/basics/>) から引用。このデモはChromeでのみ動作

オと同期して、ビデオ画面内に表示する仕組みだ。

## CSS を使って画面サイズの変形などが可能に

HTML5 という言葉が大きく流行りだしたころ、CSS3 という用語もセットで聞くようになった。実際は、CSS3 という一つの仕様書が存在するわけではない。CSS2.1 以降は、機能ごとに仕様書が分離されて新たな機能が追加されていった。それらを総合したものを CSS3 と呼ぶようになった。

単体の仕様書を例に挙げると、背景やボーダーを扱う仕様書は「CSS Backgrounds and Borders Module Level 3」という。このように、CSS2.1 の機能を拡張したものについては、機能を表す名称の後ろに Level 3 が付いている。ただし、実際に話題になったものは、CSS2.1 にはなかった全く新しい機能だ。ここでは、そのいくつかを紹介しよう。

### ■ Media Queries

近年、一つの HTML で、画面のサイズに応じてページコンテンツのレイアウトを変えて、デバイスに最適な形で見せる仕組みが一般的になってきた。これを RWD (Responsive Web Design) という。この RWD を実現する要となる機能

が「Media Queries」だ。

Media Queries は、ディスプレイ領域の幅や、オリエンテーション（スマートフォンを縦にした状態と横にした状態）といった端末環境を Web ブラウザーが判定し、サイズに応じて適用するスタイルを切り替える仕組みだ。特に、スマートフォンやタブレットといった PC 以外のデバイス向けのコンテンツでは、Media Queries は必須といってもよいだろう。

すでに、近年のスマートフォン向け Web ブラウザーは Media Query をサポートしている。また、米 Google が RWD を推奨していることもあり、今後の Web サイト制作では不可欠な機能だ。W3C では、Media Queries はすでに勧告になっている。

## ■ CSS Transforms

「CSS Transforms」は、HTML のコンテンツを所定の角度に回転した状態に見せたり、拡大したり縮小したりして見せる機能だ。このほか、移動と剪断（せんだん）が可能だ。図 2-6 の左は、正方形となるよう div 要素にスタイルを当て、それを 45 度だけ回転したものである。分かりやすくするために、本来の位置を薄い正方形として描いてある。

これだけでも面白いのだが、注目を浴びたのは 3D の機能だ。CSS Transforms

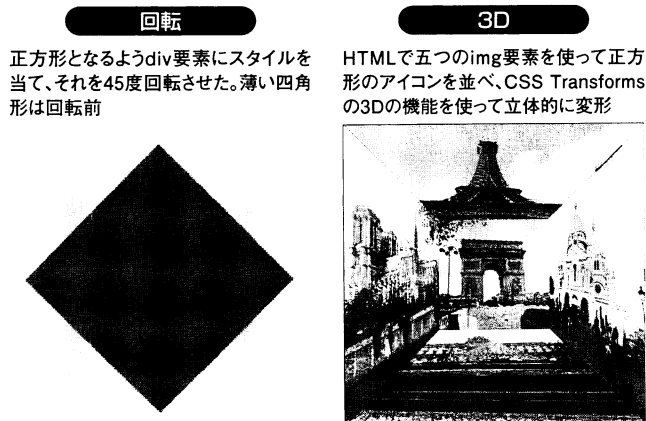
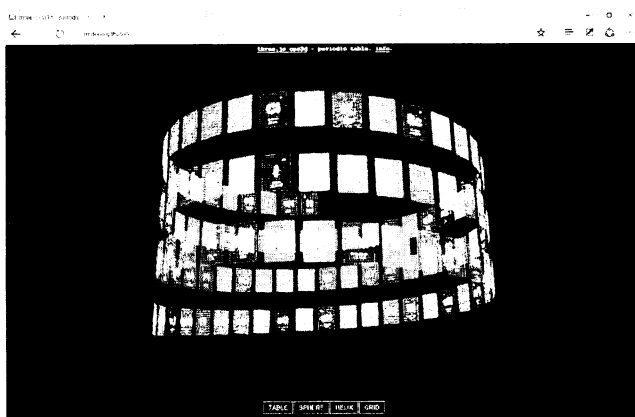


図 2-6 CSS Transforms の適用例



**図2-7** CSS Transformsを駆使したデモ

three.jsと呼ばれるJavaScriptライブラリのデモ。http://mrdoob.github.io/three.js/examples/css3d\_periodictable.html

は、二次元の変形だけでなく、三次元の変形もサポートしている。図2-6の右は、HTMLでは五つのimg要素を使って、正方形のアイコンを並べただけだ。それを、CSS Transformsの3Dの機能を使って立体的に変形し、あたかも奥行きのある立体空間に見せている。

図2-7は、CSS Transformsを駆使して作られている。実際は、three.jsと呼ばれるJavaScriptライブラリのデモだ。こういったJavaScriptライブラリを使えば、複雑な描画も比較的簡単に実現できる。

## ■アニメーション

CSSにはアニメーションに関する機能が二つ用意されている。「CSS Transitions」と「CSS Animations」である。

CSS Transitionsは、特定のHTMLコンテンツを、ある特定のスタイリングの状態から別のスタイリングの状態へ、指定の時間をかけて変化させられる。例えば、背景色を青から赤へ1秒間かけて徐々に変化させるといった効果や、コンテンツを左から右へ移動したり、回転したりするなどの、さまざまなアニメーションの手段として使える。



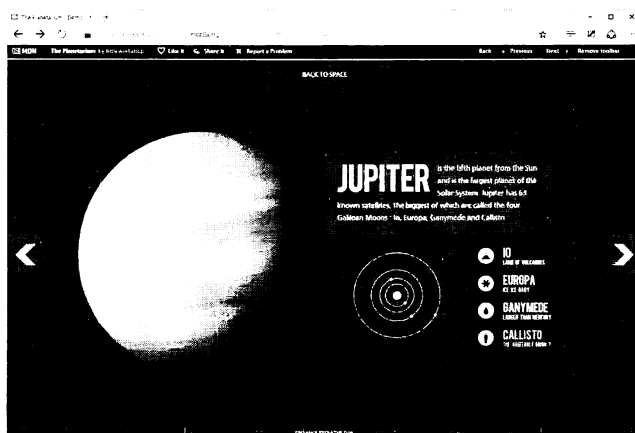


図2-8 CSSのアニメーションと3Dを組み合わせたデモ

画面を操作すると3D効果で画面が遷移する。<https://developer.mozilla.org/en-US/demos/detail/the-planetarium/launch>

CSS Animations は、キーフレームを定義することで、より詳細なアニメーション効果を定義できる。CSS Transitions では最初と最後の状態を指定して、その間を所定の時間をかけて変化させるだけだったが、CSS Animations は所定の時間内の特定の位置での状態を細かく定義できる。

CSS Transitions と CSS Animations は、前述の CSS Transforms と組み合わせて使うことで、よりダイナミックなアニメーションを実現できるようになる。図 2-8 のデモは、太陽系の惑星を紹介しているのだが、画面を操作すると 3D 効果で画面が遷移する。また、惑星の軌道の動きをアニメーションで表現している。

これまで紹介してきたアニメーションや 3D は、ゲームなどのエンターテインメント向けのコンテンツでよく使われている。ただし、ゲームだけではなく、企業システムでも応用シーンは多いと考えられる。例えば、企業システムに蓄積されたさまざまなデータをグラフなどで表すときに活用できる。さらに、教育分野でも役に立つ。教科書のような誌面や、教師が黒板に描くものだけでは理解しにくい分野では、立体的で、ユーザーが操作して動かせるコンテンツが有効である。

### 2-1 のポイント

---

- HTML5ではセマンティクスが強化された。
- Webコンテンツは人だけでなくロボットにも理解される必要がある。
- CSSのスタイリング技法の向上は企業システムにも寄与する。

## 2-2 入力とユーザーインターフェース

---

# UIを工夫し操作性を向上 必要なAPIは出そろおう

HTML5 では、数多くの API を使って便利なユーザーインターフェース (UI) を実現できるようになった。UI は操作性にかかわる重要なファクターである。入力と UI に関する API について解説する。

企業システムの操作画面は、文字入力や選択が非常に多い。このため、フロントエンドでの入力の仕組みや UI が、使い勝手を大きく左右する。以下では、HTML5 に伴って Web ブラウザーに実装された入力と UI に関する機能について解説する。

入力と UI に関する機能の中には、特定の Web ブラウザーにしか実装されていないものがある。実装されていない Web ブラウザーでそれらの機能を使いたい場合には、機能をエミュレートする JavaScript ライブラリーを活用するとよい。この JavaScript ライブラリーは「Polyfill」と呼ばれる。ここでは、代表的な入力と UI の機能を、一部 Polyfill も交えて紹介する。

### ■ダイアログボックス

ダイアログボックスは、JavaScript の `alert()` 関数や `confirm()` 関数から呼び出せる簡素なものが一般的だ (図2-9)。ただし、自由にデザインできないなど、使い勝手が悪い。また、ダイアログボックスが表示されている間は、JavaScript の処理が止まってしまうといった難点もある。

自由にカスタマイズできるダイアログボックスは、HTML5.1 仕様で `dialog` 要素 (タグ) として策定された。マークアップで構造を作り、CSS でスタイルを指定できる。デザインだけでなく、ボタンの数や表記なども自由だ。

`dialog` 要素は、すでに Chrome が実装している。ほかのブラウザーも対象にしたい場合は、`dialog` 要素をエミュレートする Polyfill<sup>ie21</sup> を使うとよい。

図 2-10 は、筆者が Polyfill を使って簡単なダイアログボックスを作り、Edge

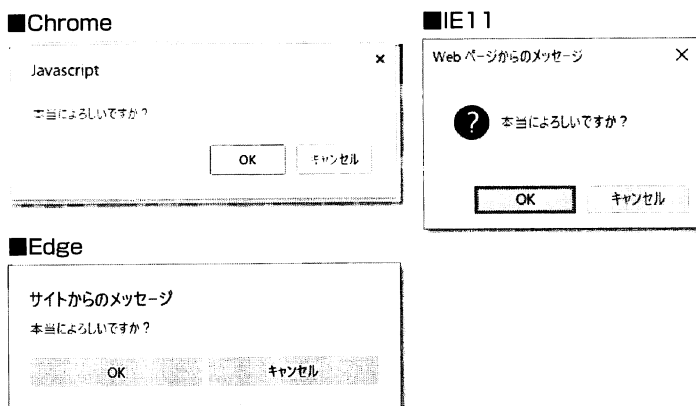


図2-9 各ブラウザで表示できるダイアログボックス

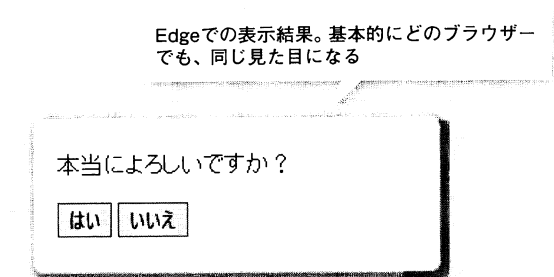


図2-10 Polyfillを使って作成したダイアログボックス

で表示した結果だ。基本的にどの Web ブラウザーでも、同じ見た目のダイアログボックスになる。

## ■テキスト選択

マウスを使ってテキストを選択状態にし、選択したテキストをコピーする UI が必要になる場合がある。意図的に JavaScript から所定のテキストの範囲を選択状

### 注2-1 dialog要素をエミュレートするPolyfill

GitHub GoogleChrome/dialog-polyfill (<https://github.com/GoogleChrome/dialog-polyfill>) を参照

態にしたいときもあるだろう。このテキスト選択の API は、W3C で「Selection API」として策定中だ。Internet Explorer (IE) や Chrome などメジャーな Web ブラウザーはすでに実装している。

Firefox で 2 カ所を選択状態にした例を図 2-11 に示す。このように、Selection API では、タグをまたいだテキストも選択できる。ただし現状では、二つ以上のテキスト範囲を同時に選択状態にできるのは Firefox のみだ。

### ■編集可能なコンテンツ

文字を入力編集可能な領域を作成するには、input 要素を使ったテキスト入力フィールドや、textarea 要素を使ったテキストエリアを利用する。入力値をサーバーに送信したいなら、これらのフォームコントロールを使ったほうがよい。

吾輩は猫である。名前はまだ無い。

### ■マークアップ

<p>吾輩は猫である。<mark>名前はまだ無い。</mark></p>

**図2-11** Firefoxでテキストを選択した状態

W3Cで策定中のSelection APIを利用した例

|       | プランA | プランB | プランC |
|-------|------|------|------|
| 2012年 | 4    | 0    | 0    |
| 2012年 | 5    | 0    | 0    |
| 2012年 | 1    | 0    | 0    |
| 合計    | 10   | 0    | 0    |

セルに数値を入力すると、自動的に合計を計算

**図2-12** 各セルを編集可能なテーブルの例

実は、HTML 上の多くの要素は、編集可能な領域を作成できる。HTML5 仕様の場合、編集可能にするための「contenteditable 属性」がほぼすべての要素で規定されている。

```
<span contenteditable="true">編集可能
</span>
```

ローカルで処理するだけなら、わざわざフォームコントロールを使うまでもない。また、フォーカス（入力や操作を受け付ける状態）が外れれば、通常のコンテンツとしてレンダリングされる。図 2-12 は、テーブルの各セルの値を編集可能にしたサンプルだ。セルに数値を入力すると、自動的に合計が計算される。これは、JavaScript からフォーカスが外れた時点で入力値を読み取って合計を計算している。

### ■ IME の変換候補の取得

IME を使って日本語を入力する際、変換値の候補が表示され、その中から一つを選んで入力を確定する。この一連の流れをリアルタイムに JavaScript から把握できる。これは「Composition Event」と呼ばれ、W3C UI Events 仕様で規定されている。

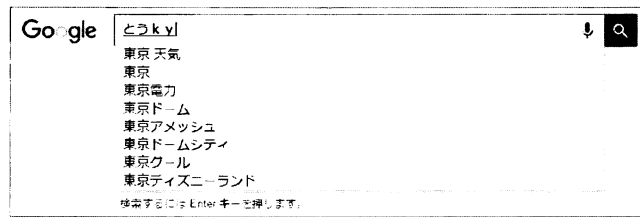
#### ■ IME で変換中の状態

A screenshot of a web form. At the top, there is a text input field containing the text '東京都'. Below the input field, there is a list of suggestions, each preceded by a number from 1 to 7. The suggestions are: 1 東京と, 2 東京都, 3 東響と, 4 桃郷と, 5 とうきょうと, 6 トウキョウト, and 7 トウキョウト. To the right of the last suggestion, there is a right-pointing arrow icon.

#### ■ 変換の流れ

A screenshot of a browser's developer console. At the top, it shows the selected frame as '<top frame>' and a checkbox for 'Preserve log'. Below this, a list of log entries shows the sequence of 'composition' events. The events are: 'compositionstart:', 'compositionupdate: t', 'compositionupdate: と', 'compositionupdate: とう', 'compositionupdate: とう k', 'compositionupdate: とう k y', 'compositionupdate: とうきょ', 'compositionupdate: とうきょう', 'compositionupdate: とうきょう t', 'compositionupdate: とうきょうと', 'compositionupdate: 東京都', and 'compositionend: 東京都'. The first five events are preceded by a circular arrow icon, and the last two are preceded by a circular arrow icon with a dot.

図 2-13 IME で変換する流れを JavaScript で把握



**図2-14** Googleサジェストでの変換候補の表示

例えば「東京都」と入力したい場合、「とうきょうと」と入力して変換を繰り返す（図 2-13）。最後にエンターキーを押して確定すると、テキストフィールドにセットされる。一連の流れを、JavaScript からイベントとして取得し、その時点の入力変換状況を把握する。入力変換状況をコンソールに表示した結果は図 2-13 の右ようになる。

Composition Event を使うと、Google 検索のように、入力を確定する前にキーワード候補を表示できる（図 2-14）。企業システムのような入力が多いサービスでは、ユーザーの手間を省く機能として大いに役に立つだろう。Composition Event は、IE や Edge などメジャーな Web ブラウザーすべてに実装されている。

### ■クリップボード

カットやコピー、ペーストといった操作は、作業効率化には欠かせない。もちろん、Web ブラウザー上でもクリップボードを使ってテキストをコピーし、テキストエリア内にペーストできる。しかし、Web アプリケーションによっては、その操作そのものを JavaScript を使ってコントロールしたい場合もあるだろう。

W3C では、「Clipboard API and events」仕様が策定されており、メジャーな Web ブラウザーにはその一部が実装されている。この API が用意している機能は、カット、コピー、ペーストのイベントの取得だ。ただし、実際に何がカットまたはコピーされたかは把握できない。できるのは、ペーストされた値を把握することだ。

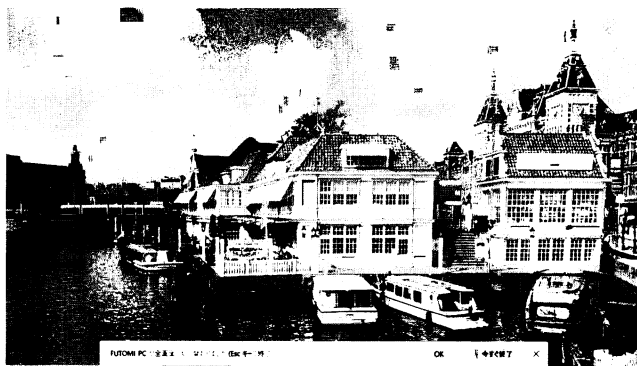
この API の活用ポイントは、OS が提供するクリップボードの機能を JavaScript で横取りするところにある。例えば、Web ブラウザーに表示されて

いるテキストをコピーするケースを考えてみよう。本来なら、そのテキストは OS のクリップボードに格納される。この API を使うと、JavaScript によってアクションそのものを横取りし、クリップボードに自由に値を挿入できる。もちろんその値は、他のソフトウェア上にもペースト可能だ。ただし、ユーザーのコピー操作なしに、JavaScript からクリップボードにデータを送ることはできないので、注意してほしい。

### ■フルスクリーン表示

Web コンテンツは、基本的に Web ブラウザーのウィンドウの中に表示される。コンテンツによっては、Web ブラウザーのツールバーやステータスバーが邪魔になることがある。

特に、スマートフォンやタブレット向けの Web ブラウザーでは、画面が小さいにもかかわらず、Web ブラウザーのツールバーが大きな面積を占め、大事なコンテンツを見せる領域がさらに狭くなる。この課題を解決するのが、WHATWG が策定している「Fullscreen API」仕様だ。この API は、Web ブラウザーに表示されたコンテンツをフルスクリーン表示するもので、メジャーな Web ブラウザーはすべて実装している。



エスケープ (Esc) キーでフルスクリーンモードを解除できる

図2-15 Edgeのフルスクリーンモード



読者の中には、PC 版の Web ブラウザーであれば、API を使わなくても、Web ブラウザーが用意しているフルスクリーン表示モードを使えばよいと考える人がいるだろう。この API のメリットは、HTML コンテンツの特定の領域だけをフルスクリーン表示できるところにある。

Fullscreen API は、ゲームなどのコンテンツを想定して考え出されたものだが、企業向けシステムでも役に立つときがある。例えば、窓口で顧客にタブレットを使って商品紹介を見せるときなどだ。

図 2-15 は、ページ上の掲載された画像の部分だけをフルスクリーン表示した結果だ。JavaScript から所定のメソッドを呼び出すだけで、特定のコンテンツだけをフルスクリーン表示できる。また、モードが変わった際にイベントも取得でき、さらに、どのコンテンツ（要素）がフルスクリーン表示されているかも判定できる。モードに応じてスタイルを変更することも可能だ。フルスクリーンモードは、どの Web ブラウザーでもエスケープ（Esc）キーを押せば解除できる。

#### ■スクリーンオリエンテーション

スマートフォンやタブレットには、オリエンテーションという概念がある。デバイスを縦に使うか横に使うかで、モードが切り替わることだ。Web ブラウザーも例外ではない。基本的には、Web ブラウザーが制御してどちらのモードでもそれなりに見やすい形で表示してくれる。

ただし、この仕組みも完全とは言えない。また、コンテンツ提供者としては、より細かい制御を入れたいところだろう。そのためには JavaScript からオリエンテーションの状態をリアルタイムに把握する必要がある。これを実現するのが、「W3C Screen Orientation API」だ。

この API は非常にシンプルで、現在のオリエンテーションモードを返すプロパティと、オリエンテーションが変わった際のイベントが規定されている。これに加え、前述のフルスクリーンモードの場合、オリエンテーションをロックする機能を利用できる。これは、デバイスの向きにかかわらず、強制的にポートレートモード（縦）またはランドスケープモード（横）に固定できるというものだ。現在、仕様に合わせて実装されているのは、Android 版の Chrome のみ。iOS の Safari には実装されていない。

### ■マウスポインターのロック

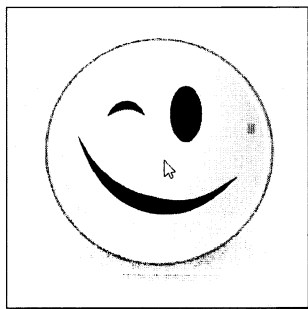
マウスポインターのロックとは、マウスを使えないようにするという意味ではなく、ポインターを非表示にして動きのみを検知する仕組みのことだ。もともとはプレイヤーの視点でゲーム内の空間を表示するファーストパーソン・シューティング・ゲーム（本人視点シューティングゲーム）のために考え出された。

マウスの動きに応じて自分の向きを変えたり移動したりするには、マウスの移動範囲がディスプレイの領域に制限されては困る。この制限を解除するのがマウスポインターのロックである。仕組みは「W3C Pointer Lock」仕様で規定されており、すでに Edge、Chrome、Firefox、Opera サポートしている。

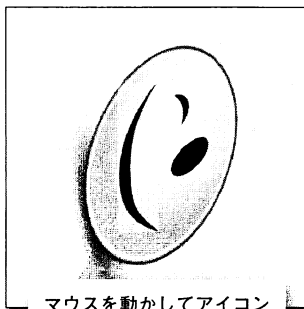
マウスポインターのロックは、さまざまなシーンで使えると考えられる。ゲームなら、ファーストパーソン・シューティング・ゲームでなくても、「Minecraft（マインクラフト）」のような 3D 空間を扱うゲームなども対象になる。企業向けシステムなら、例えば不動産業界などで物件の室内のレイアウトを立体的に見せるアプリケーションが想定される。このほか、教育分野でも有効だろう。

次ページの図 2-16 は筆者が Chrome 向けに作った簡単なサンプルだ。平面のアイコン画像をマウスの動きに合わせて立体的に回転させる。左がポインターをロックする前の状態、右がロックした後にマウスを動かしてアイコンを回転させた状態だ。このサンプルの操作感は紙上では分かりにくいですが、ゲームだけでなく

■ポインターのロック前



■ポインターのロック後



マウスを動かしてアイコン  
を回転させた状態

図 2-16 マウスポインターをロックしてアイコンを回転

3D を扱うコンテンツであれば、多くのシーンで応用できる一例である。

### ■ゲームコントローラー

先述の Pointer Lock は 3D コンテンツを扱う際に便利だと説明したが、実際に触ってみると、かなり使い勝手が悪い。やはり Xbox や PlayStation のようなコンソールゲーム機のコントローラーのほうが圧倒的に使いやすい。このようなときに活用したいのが「W3C Gamepad」仕様だ。汎用的なゲームコントローラーを扱うための API を規定している（図 2-17）。PC の USB ポートに接続して利用するタイプのコントローラーであれば、問題なく利用できる。

Gamepad 仕様では、ボタンの押下のみならず、スティックの傾き方向や傾き度合いもリアルタイムに把握できる。現時点では、Edge、Chrome、Firefox が Gamepad 仕様で規定された API を実装している。

ゲームコントローラーと聞くと、どうしても遊びのイメージが強いが、筆者は 3D コンテンツを操作する上で、ゲームコントローラーほど効率的な入力デバイスはないと考える。

スライドを使ったプレゼンテーションでも役立つ。一般的に講演に使うスライドは、PowerPoint などのプレゼンテーションソフトを使うことが多い。スライドの移動は矢印キーまたはエンターキーを使う。しかし、それ以外の細かい操作は、立ちながらの講演では非常にやりにくい。筆者は講演で、スライドを Web コン

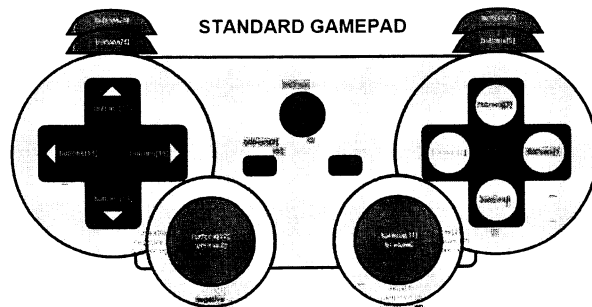


図 2-17 Gamepad 仕様が想定するゲームコントローラー

出典は W3C Gamepad 仕様 (<http://www.w3.org/TR/gamepad/>)

テンツとして作成し、ゲームコントローラーでスライドを操作することがある。使うのは、Gamepad 仕様の API を活用した自作の JavaScript フレームワークだ。

開発するのは面倒だが、実際の講演では非常に便利だと実感した。いずれ、企業向けシステムでもゲームコントローラーが入力デバイスとしての地位を確立する時代が来ると考えている。

## ■バイブレーション

Web アプリケーションにおいて、何かしらのアクションをユーザーに伝えるためのフィードバックは非常に重要だ。メッセージの表示は当然だが、ときにはオーディオを再生することもあるだろう。ただし、状況によっては、オーディオはユーザーにとって迷惑になる場合もある。そこで役立つのがバイブレーションだ。スマートフォンであれば、通常は手に持って操作しているので、バイブレーションは確実にユーザーへフィードバックを返せる。

「W3C Vibration API」はバイブレーションを JavaScript から制御するための API である。この API は非常にシンプルなため、ここでは少しだけ JavaScript コードを紹介しよう。

```
navigator.vibrate(1000);
```

このコードでは 1 秒間だけデバイスが振動する。このように単にブルルっと所定の時間だけ単調なバイブレーションを実現するだけでなく、さまざまなパターンを命令できる。

```
navigator.vibrate([1000, 500, 1000]);
```

このコードは、1 秒間のバイブレーションを 2 回発生させる。2 回のバイブレーションの間の休止は 0.5 秒だ。このように、バイブレーションと休止を組み合わせ、さまざまなパターンのバイブレーションを実現できる。現時点で、Android 4.4 以降の標準ブラウザ、Android 版 Chrome、Firefox が Vibration API を実装している。

## ■ドラッグ&ドロップ

ドラッグ&ドロップは、HTML5 が話題になり始めた当初から注目された API だ。今となっては、新しさを感じないだろう。Web ブラウザー上に表示されたページの中にあるコンテンツを、ブラウザ内でドラッグ&ドロップできたとしても、あまりインパクトはない。だが、デスクトップ上のファイルを Web ブラウザー内にドロップして、それをサーバーにアップロードできればどうだろうか。

従来デスクトップやフォルダー内のファイルをアップロードするには、input 要

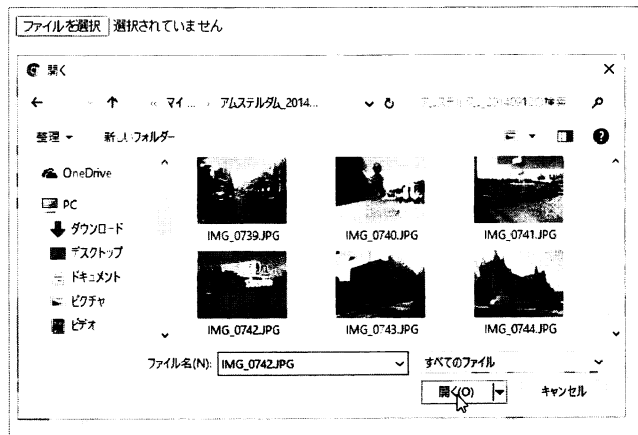
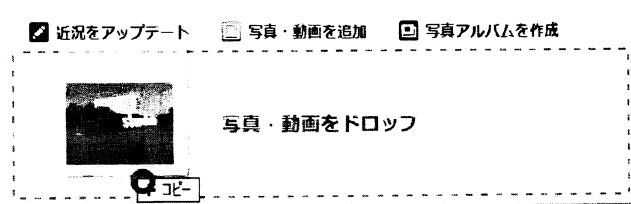


図2-18 従来のinput要素を使ったファイル選択

## Facebookのファイルアップロード画面



デスクトップ上のファイルをブラウザ内にドロップしてサーバーにアップロード

図2-19 ドラッグ&ドロップでファイルアップロード

素を使って、ファイル選択ダイアログからファイルを選択させる方法しかなかった(図 2-18)。ファイル選択ダイアログからファイルを選択するのは非常に面倒だ。一方、Facebook などのコンシューマー向けサービスでは、デスクトップ上のファイルを Web ブラウザーにドロップしてサーバーにアップロードできる(図 2-19)。デスクトップやエクスプローラーからファイルをドラッグ&ドロップできるほうが、圧倒的に効率的だろう。

デスクトップ上のファイルをドラッグ&ドロップしてアップロードする機能は、PC 向けの主要な Web ブラウザーはすべてサポートしている。

## 多数の API が今後実装されていく

企業システムでは、入力デバイスやユーザーインタフェースが利用者の作業効率に直結するため、こだわりたいところだ。ほかにも、紹介しきれなかった UI や入力に関する API が数多く存在する。例えば、日付や時間、色選択などのフォームコントロール、フォームの入力チェックメカニズム、表示画像を画面解像度に合わせて切り替える仕組み、マウスで右クリックした際に表示されるコンテキストメニューをカスタマイズする API、独自の機能を持たせた要素(タグ)を自由に作れる「Custom Elements」、マウスとタッチとペンからの入力を包括的に扱う「Pointer Events」などだ。

これらの紹介できなかった API は、まだすべての Web ブラウザーに実装されているわけではないが、いずれ利用できるようになるだろう。

### 2-2 のポイント

---

- 企業システムでは、入力デバイスとUIが作業効率に直結する。
- 必要なAPIはIE 11やEdgeでそろってきた。
- Polyfillを使うことで解決できるAPIもある。

## 2-3 マルチメディア

# 重要性増すビジュアル表現 ブラウザーの対応が進む

ビデオやオーディオの再生、グラフィックス、アニメーションは、企業システムのフロントエンドではあまり使われてこなかった。しかし現在、あらゆる Web ブラウザーでマルチメディア機能が充実し、環境が整ってきた。マルチメディア関連の API を解説する。

HTML5 の大きな特徴の一つは、マルチメディア系の API の充実だろう。アニメーションやビデオ、オーディオ再生はこれまで、すべてプラグインに頼らざるを得なかった。一方、近年の Web ブラウザーはプラグインを排除する方向に向っており、とりわけスマートフォンやタブレットでは、プラグインは完全に排除された。

このような状況の中、マルチメディアコンテンツを扱うには、HTML5 関連の Web 標準の API が欠かせない。ビデオ、オーディオ、グラフィックスそれぞれの API を紹介する。

### ■ Canvas

Canvas は 2D のビットマップグラフィックスのための API だ。Canvas は、HTML5 黎明期から Internet Explorer (IE) を除くメジャーブラウザーに実装されていたこともあり、HTML5 関連の API の中で最も認知度が高い。Firefox 1.5、Opera 9.0、Safari 1.3、Chrome 1.0 の時代から実装されていた。IE に Canvas が実装されたのは、IE 9 になってからだ。

Canvas は図形を表現するだけでなく、ピクセルごとに操作できる。お絵かきアプリやチャート、グラフ描画などに使われている。例えば、Mozilla Developer Network が公開している「Ghostwriter Art Studio」は、Canvas を利用して絵を描くデモである（図 2-20）。JavaScript ライブラリー「Chart.js」を使ったグラフのサンプルは、Canvas を使って作られている（図 2-21）。





図2-20 Canvasを利用した「Ghostwriter Art Studio」

Mozilla Developer NetworkのWebサイトにあるデモ。 <https://developer.mozilla.org/en-US/demos/detail/ghostwriter-art-studio/launch>

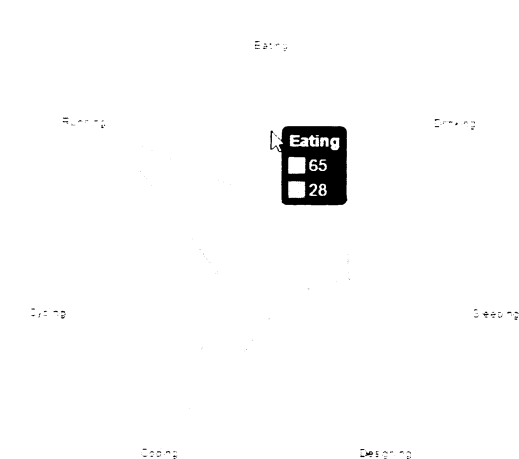


図2-21 Canvasを使って作成したグラフ

JavaScriptライブラリー「Chart.js」を使ったサンプル。 <http://www.chartjs.org/>に掲載されたドキュメントから引用

Canvas では、直線、円弧、矩形、テキストなどを自由に描ける。枠線の色や、塗りつぶしの色なども個別に指定できる。ただし Canvas は、あくまでもビットマップグラフィックスだ。描いた直線や円弧といった図形は、単なるピクセルご

とに色が塗られたにすぎない。つまり、後から図形を認識できない。

その代わり、描いた図形からピクセルごとに色情報を抜き出せる。さらに、ピクセルごとに色を指定して、その色を塗ることもできる。このピクセル操作は、主に写真やビデオのコマを Canvas に貼り付けて、その映像を解析する際にも使われる。

Canvas は一枚の絵を描くだけの機能に特化しているため、アニメーションの機能はない。だが、使い方次第ではアニメーションも実現できる。その方法とは、一コマごとリアルタイムに書き換えるというもの。一コマの描画速度は、近年のスマートフォンならかなり高速だ。描画領域がよほど大きくない限りは、滑らかにアニメーションを実現できる。

Canvas は 2D ゆえに、3D グラフィックスに必要な機能は一切ない。コンピュータグラフィックスでは図形の変形に座標変換がよく使われるが、Canvas には移動、拡大・縮小、回転、せん断が用意されている。しかし、Canvas には奥行き (z 軸) の概念がないため、遠近法に関わる変形はできない。計算によって無理やり 3D っぽく見せるコンテンツを作れるが、それは以降で紹介する「WebGL」に任せるのがよいだろう。

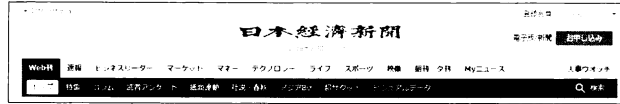
## ■ SVG

SVG は 2D のベクターグラフィックス仕様である。Canvas がピクセルで図を表現するのに対して、SVG は座標や線などの数値で表現する。SVG の特徴はこのベクターである点と、XML ベースのマークアップ言語という点にある。

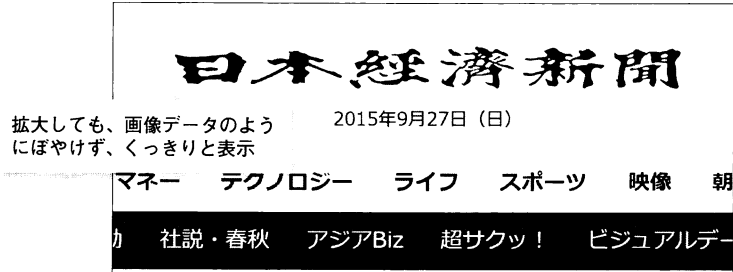
実は、SVG が W3C の勧告になったのは 2001 年で、かなり古い技術だ。以前から IE を除くメジャーブラウザは SVG を実装してきた。しかし、かつて最もシェアが高かった IE が実装しなかったため、少なくとも Web 業界で使われることはほとんどなかった。また、ベースとなる言語が XML のため、通常の HTML の中に組み込んで表示できなかった点もネックだった。

HTML5 時代に入り、SVG は改めて注目され始めた。きっかけとは、HTML5 仕様では、HTML の中に SVG をインラインにマークアップできるようになったことだ。実際に、各メジャーブラウザがそれを実装した。また、IE 9 が SVG を実装したことによって、現在では、すべてのメジャーブラウザで SVG が使

## ■日本経済新聞 電子版のロゴ



## ■ブラウザの機能で拡大表示



## ■図2-22 SVGロゴは拡大してもくっきり表示できる

日本経済新聞社の「日本経済新聞 電子版」のロゴの例。SVGで作られているため、拡大しても表示がぼやけない

えるようになっている。

ベクターグラフィックスの特徴は、図を拡大してもきれいに見えること。そのため、チャートやグラフなどで拡大表示機能を持たせる際に重宝する。また、企業や商品のロゴやアイコンにも使われている。

実際に SVG を企業ロゴやアイコンに使っているサイトが数多くある。国内の企業では、日本経済新聞社の「日本経済新聞 電子版」のトップに掲載されているロゴが有名だ（図 2-22）。同サイトにアクセスして、Web ブラウザーの拡大機能を試してほしい。もしロゴが画像ならばぼやけてしまうはずだが、SVG ロゴならどれだけ拡大してもぼやけずに表示されるのが分かる。

海外企業では、米 Apple や米 Starbucks の Web サイトのロゴなどがある。Apple のサイトでは各製品のアイコンに、Starbucks のサイトでは左上の企業ロゴに SVG が使われている。

このほか、SVG は HTML と同様に、すべての描画オブジェクトが JavaScript から「ノード」という形で認識できる。このため、描画した後に色を変えたり移動したり、クリックを受け付けられたりするなど、HTML と同じように扱える。



**図2-23** JavaScriptライブラリー「three.js」

WebGLが簡単に扱えるようになる定番のライブラリー。<http://threejs.org/>



**図2-24** three.jsを使って作成したサンプル

仏Little Workshopが制作した「The new Renault Espace」(<http://www.littleworkshop.fr/renaultespace/>)の画面

## ■ WebGL

WebGL は Canvas の 3D 版だ。ただし、仕組みは全く異なり、OpenGL がベースである。グラフィックスの描画を GPU (Graphics Processing Unit) で処理するため、通常の HTML コンテンツのように CPU でレンダリング処理する場合と比べて、圧倒的にパフォーマンスが高い。

WebGL は、W3C とは別の標準化団体である Khronos Group が仕様を策定し

ている。OpenGL のプログラミング経験があるエンジニアであれば WebGL を扱うのは問題ないだろうが、Web エンジニアにとってはかなり難しいのが実情だ。そのため一般的には、WebGL を比較的容易に扱えるようにした JavaScript ライブラリーを使う。

定番のライブラリーは「three.js」だ (図 2-23)。three.js のサイトには、さまざまなサンプルが掲載されている。例えば、仏 Little Workshop の作品は、マウスを使って自動車をどのアングルからも自由に閲覧できる (図 2-24)。自動車メーカーの Web サイトでも、同様の Flash を使ったコンテンツがあるが、プラグインを使わなくても、WebGL を使って、かなり精細でなめらかな動きを持ったコンテンツが作れることが分かる。

## リフレッシュレートに合わせて描画

ここまで説明してきた、グラフィックスの技術を使いこなすには、ディスプレイのリフレッシュレートに関する知識も必要だ。

アニメーションを実現する場合、Canvas では 1 コマずつ領域全体を描画し直したり、WebGL では描画されたオブジェクトの座標を変えたりする。この場合、以前であれば、JavaScript のタイマー関数を使って、所定の間隔でコマ描画を処理してきた。一般的な PC のディスプレイはリフレッシュレートが 60Hz だ。つまり、16 ~ 17 ミリ秒ごとにコマ描画の処理を行うと、最大限になめらかなアニメーションが実現できる。

ところが、近年はデバイスが多様化したため、必ずしもディスプレイのリフレッシュレートが 60Hz とは限らない。また、タイマー関数を使うと、リフレッシュレートぴったりにタイミングを合わせて処理することはできず、必ずズレが生じる。リフレッシュレートより少しでも間隔が短ければ、一部のコマはディスプレイに反映されず、CPU に無駄な処理をさせるだけになってしまう。

このような問題を抱える中、登場したのが W3C の「Timing control for script-based animations」という仕様だ。ネットで検索する場合は、この仕様で規定している「requestAnimationFrame」というメソッドの名称を使ったほうがよい。

`requestAnimationFrame` は、タイマー関数の一種である。だが、時間間隔を指定しない。その代わり、セットした処理を、次にやってくるリフレッシュレートに合わせて実行してくれる。

現在、スマートフォンも含めて、近年の Web ブラウザーはすべて `requestAnimationFrame` をサポートしている。もはやアニメーションには欠かせない機能となっている。

## オリジナルのビデオプレーヤーも作れる

HTML5 が登場した頃、大きく話題となったのがビデオとオーディオの再生だ。かつて、ビデオとオーディオの再生には、Flash などのプラグインが必要だったが、HTML5 仕様で規定された `video` 要素と `audio` 要素を使えば、プラグインなしに再生できる。すでに YouTube や Vimeo といった動画配信サイトも、HTML5 ベースで動画を配信している。

とはいえ、大きく話題になった割には、一般的な Web サイトではあまり使われていないようだ。基本的に動画はファイルサイズが大きい。また、ネットワークトラフィックにも限界がある。そのため、無料で YouTube に上げたほうが理に適っている。

HTML5 仕様で規定された `video` 要素や `audio` 要素は、特に企業向けシステムで必要とされることが多い。あまり目にしないのは、一般に公開したくないコンテンツは、社内に閉じたサービスとして提供するからだろう。有料のサービスだ



図2-25 Internet Explorer 11のビデオプレーヤー



画像提供 : Google

と、一般公開はそもそも論外だ。そういう意味では、一般に知られていないところで、video 要素や audio 要素が使われているといえる。

HTML5 仕様では、タグだけでなく、JavaScript から制御するための API も規定された。あらゆる操作ができるように、膨大な量の API と説明が記載されている。実は、こうした API を使えば、JavaScript を使ってオリジナルのビデオプレーヤーも作れる。プレーヤーの見た目は、Web ブラウザーによって異なる。例えば、図 2-25 は IE 11 のビデオプレーヤーである。

一方、YouTube が提供する独自のプレーヤーは、どの Web ブラウザーでも同じユーザーインターフェースと機能を提供している（図 2-26）。このように、Web ブラウザーのデフォルトのプレーヤーには用意されていない機能を追加して、使い勝手の良いプレーヤーも作れる。

## 主流となった HTTP を使ったストリーミング

動画のストリーミングといえば、RTP (Realtime Transport Protocol) といったプロトコルを想像する人が多いだろう。ところが、RTP が許可されていないネットワークも多く、一般消費者向けのブラウザベースのサービスとして RTP を使うのは現実的ではない。そこで近年は、HTTP ベースのストリーミング技術「HTTP アダプティブストリーミング」が主流となっている。

スマートフォンでよく使われる「HTTP Live Streaming (HLS)」は Apple が開発した技術である。仕組みが簡単で、さらに Android でも利用できるとあつ

て、現在最も普及しているストリーミング技術といえる。

HLSに限らず、HTTP ベースのストリーミング技術は、動画データを数秒ごとに分断しておき、それを逐次取りに行く、という仕組みで動画を再生する。分断されたデータの所在は、何かしらテキストベースのメタデータファイルに記述する。Web ブラウザーは、そのメタデータを事前に取得した上で、必要なデータを逐次取りに行く。

HLS を利用するには、video 要素の src 属性に、メタデータファイルの URL を記述するだけでよい。

```
<video src="movie.m3u8"></video>
```

このように、HLS は仕組みが簡単で使いやすいが、いくつか問題点もある。

JavaScript からストリーミングの挙動をコントロールできない。ここでいうコントロールとは、データのダウンロードが再生に間に合わない場合に、低ビットレートのストリーミングにシームレスに切り替えるといった制御や、動的に別のストリームに切り替えるといった制御のことだ。

また、HLS は標準化の足並みがそろっていない。Apple が標準化団体の IETF (Internet Engineering Task Force) で標準化を進めようとしてはいるが、業界全体で取り組む流れはできていない。HTTP アダプティブストリーミングには、HLS のほかにも、米 Microsoft の「Smooth Streaming」、米 Adobe Systems の「HTTP Dynamic Streaming」などがあり、業界内で乱立している。

このような中、Motion Picture Experts Group (MPEG) が、HTTP アダプティブストリーミングの仕様として「MPEG-DASH」を策定した。この標準化作業には、ソフトウェアベンダーの Microsoft や Adobe、ネットワーク機器ベンダーの米 Cisco Systems、チップベンダーの米 Qualcomm、動画配信サービス事業者の米 Netflix などが加わっている。また、米 Google も、Chrome に MPEG-DASH を実装するなど、今後は MPEG-DASH が主流となると見られる。

一方、W3C は HTTP アダプティブストリーミングを JavaScript から扱うための API 開発に着手した。仕様の名称は「Media Source Extensions (MSE)」である。MSE は HLS と違い、マークアップによる方法はなく、完全にスクリプトベ

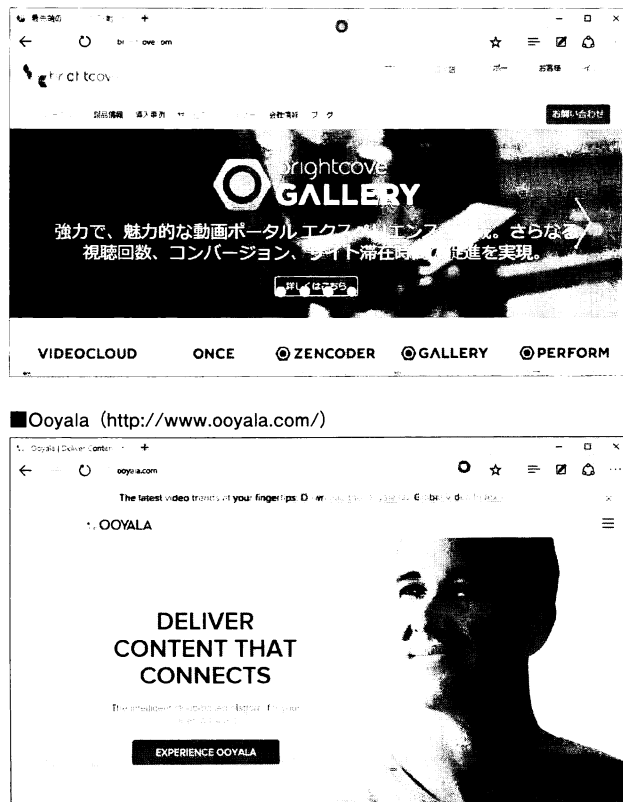


スだ。

すでに PC 版のメジャーブラウザはすべて MSE を実装している。スマートフォン向け Web ブラウザーでは、Android の標準 Web ブラウザーおよび Chrome がサポートしている。あとは iOS の Safari を残すのみだ。

## 課題となるコンテンツの著作権管理

動画を扱う際に課題となるのが、コンテンツの著作権保護である。W3C は、



**図 2-27** 動画配信プラットフォームを提供するサービス

配信サーバーやプレーヤー、分析機能などを提供する

前述の MSE と並行して、DRM (Digital Rights Management、デジタル著作権管理) を扱う API 仕様「Encrypted Media Extensions (EME)」の開発に着手した。EME は、DRM を標準化したものではなく、複数方式の DRM を扱えるようにするためのフレームワークといったほうがよいだろう。

重要なのは、各 Web ブラウザーが EME を実装しているかどうかではなく「ブラウザーがどの DRM をサポートしているか」だ。DRM にはさまざまな方式がある。Microsoft の「PlayReady」や Adobe の「Primetime」、Google の「Widevine」のほか家電メーカーが中心となって策定した「Marlin」などだ。

マルチブラウザーで DRM を使った動画配信をする場合、主要な DRM をサポートした配信サーバーが不可欠だ。だが、HTTP アダプティブストリーミングを扱う MSE や、DRM を扱う EME の API を使って、自社で動画配信の仕組みを開発するのはまれだろう。そもそも、現在主流となっている HLS を開発した Apple は、今のところ MSE や EME をサポートする気配がない。

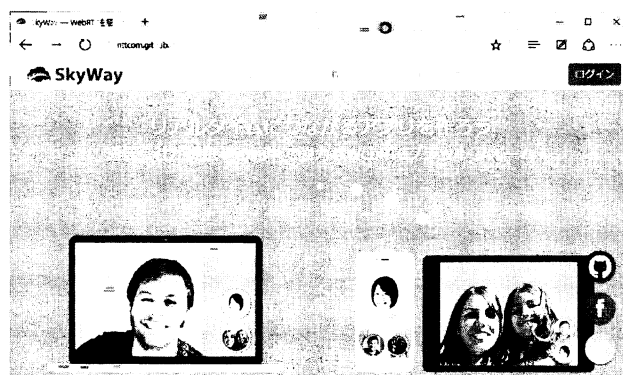
実際には、配信サーバーから Web ブラウザー上で動作するプレーヤーまで、まとめて提供するサービスを使うのが一般的である。オンライン動画プラットフォームとしては、米 Brightcove (ブライトコブ) や、米 Ooyala (ウーヤラ) の各種サービスが有名だ (図 2-27)。

## ビデオ通話を実現する WebRTC

Web ブラウザーでビデオ通話を手軽に実装するなら、WebRTC が向く。WebRTC は、PC やスマートフォンのカメラとマイクロフォンの映像と音声を取得し、送受信するための API である。専用のアプリをインストールしなくても、Web ブラウザーだけで Skype のようなビデオ通話システムが構築できる。

WebRTC の大きな特徴はピアツーピア接続だ。通話中にやり取りする映像や音声のストリームは、サーバーを中継せずに、ダイレクトに相手へ送れる。そのため、遅延が少なく使い勝手の良いビデオ通話が可能となる。もちろん、データの送受信もできる。

ピアツーピアといっても、相手を発見する仕組みや、制御信号を送受信する仕組みが別途必要となる。これらは主に、IP 電話で使われてきた技術を応用する



**図 2-28** WebRTCプラットフォーム「SkyWay」

NTTコミュニケーションズが無償提供する、WebRTC技術を活用するために開発したプラットフォーム。ソフトウェア開発支援サービスGitHub上でライブラリー・ソースコードも公開している。<https://nttcom.github.io/skyway/>

ため、各種サーバーが必要となるが、自前で構築するのは大変だ。そのため各種サーバー機能を提供するクラウドサービスがすでいくつか存在しており、国内事業者のサービスでは、NTTコミュニケーションズの「SkyWay」が有名である（図 2-28）。

WebRTC は、社内のビデオ会議システム以外の用途にも利用できる。例えば、一般消費者向けサービスを展開している事業者であれば、サポートセンターの受付手段の一つとして採用してもよいだろう。WebRTC を使うと、Web サイトを見ているユーザーがボタン一つでオペレータに接続できる。フリーダイヤルによる着信が減り、通信コスト削減に寄与するだろう。今どきの PC やスマートフォンであれば、カメラやマイクロフォンを内蔵しているため、すでに現実的なサービスになりつつある。

本パートで紹介したビジュアル表現は、企業システムのフロントエンドで重要性が高まっている。文字ばかりの情報では伝わりにくい場合、ビデオやチャートを駆使して、使いやすいシステムを構築するとよい。また、後半に紹介したストリーミング、DRM、WebRTC などは、クラウドサービスが数多く登場してきている。技術動向を把握しつつ、サービスを活用するのも一つの方法だろう。

### 2-3 のポイント

- HTML5でマルチメディアコンテンツを実現する環境は十分に整った。
- クラウドと連携しないと使えないAPIも数多くあり、高度化、複雑化している。
- クラウドサービス事業者と組んだシステム構築も視野に入れる必要がある。

## 2-4 ハードウェアへのアクセス

---

# 各種センサーを活用し Webの応用範囲を広げる

Web アプリケーションの応用範囲を広げるためには、ネイティブアプリで当然のように実現できていた、ハードウェアリソースへのアクセスは欠かせない。そういった API のうち、すでに Web ブラウザーに実装されている API を紹介する。

HTML5 時代に入り、JavaScript から利用できる API は大きく進化した。その代表例が、ハードウェアに関連するリソースにアクセスできるようになったことだ。

従来、Web ブラウザー上に表示するコンテンツは、あくまでも Web ページの延長でしかなかった。しかし、ハードウェアに関連するリソースにアクセスできるようになり、Web アプリケーションでネイティブアプリケーションに近いことができるようになった。

ハードウェアに関連するリソースとは、具体的には、デバイスに搭載された各種センサー類、ストレージ、カメラ、マイクロフォンなどだ。範囲を広げれば、CPU や USB などの外部端子も含まれる。Web アプリケーションからハードウェアにアクセスするために、すでに Web ブラウザーに実装された主な API を紹介する。

### 4 種類ストレージ系 API

Web ブラウザーにデータを保存する仕組みは、アプリケーションにとっては非常に重要だ。従来は、Web ブラウザーにデータを保存する仕組みとして Cookie が使われてきた。しかし Cookie は、保存できるデータの量が非常に小さい。加えて、該当サイトにアクセスするたびに、Web ブラウザーは Cookie のデータをサーバーに送信してしまう。

このため、純粋にデータを Web ブラウザーに保存させたい場合には、Cookie

### ■Web Storage

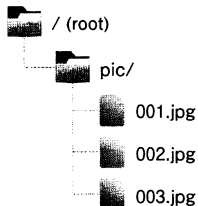
キーバリュースペア

| key   | value    |
|-------|----------|
| "001" | " 山田太郎 " |
| "002" | " 鈴木次郎 " |
| "003" | " 佐藤史郎 " |

基本的に保存できる値はテキスト情報のみで、保存できるデータのサイズはサイトごとに5M～10Mバイト程度

### ■File API: Directories and System

ファイルシステム



ディレクトリを作ったり、指定のディレクトリの中にファイルを保存したりできる

### ■Web SQL Database

テーブル

| key   | value    | email             | age |
|-------|----------|-------------------|-----|
| "001" | " 山田太郎 " | "taro@example.jp" | 25  |
| "002" | " 鈴木次郎 " | "jiro@example.jp" | 39  |
| "003" | " 佐藤史郎 " | "siro@example.jp" | 43  |

Web Storageでは対応していなかった大量のデータを保存するためのAPI。だが、標準化作業は中止されている

### ■Indexed Database API

オブジェクトストア

| key   | object                                                                               |
|-------|--------------------------------------------------------------------------------------|
| "001" | { "name": " 山田太郎 ", "email": "taro@example.jp", "age": 25, "pic": Blob( バイナリーデータ ) } |
| "002" | { "name": " 鈴木次郎 ", "email": "jiro@example.jp", "age": 39, "pic": Blob( バイナリーデータ ) } |
| "003" | { "name": " 佐藤史郎 ", "email": "siro@example.jp", "age": 43, "pic": Blob( バイナリーデータ ) } |

階層構造を持ったデータを格納し、それらのデータにインデックスを設定できる。大量のレコードを持ったデータを格納するのに役立つ

図2-29 ストレージ系APIの違い

は不向きだった。そこで登場したのが、ストレージ系の API である。

データを Web ブラウザーに保存する目的は、主にパフォーマンスの向上だ。変更が少ないデータを、サイトにアクセスするたびにサーバーからダウンロードするのは時間の無駄だ。最初に Web ブラウザーに保存しておけば、次回以降はサーバーからのダウンロードが省け、サーバー管理者にとっても、利用者にとっ

でも大きなメリットがある。

ストレージに関連する API は全部で四つある (図 2-29)。ここではそれぞれについて簡単に紹介しよう。

### ■ Web Storage

この API は、キー・バリュー・ペアを保存するためのものである。何かしらの一意的なキーに対して任意の値を保存し、その後はキーを使って値を取り出せる仕組みだ。最も古くから Web ブラウザーに実装されたストレージ系 API であり、Internet Explorer 8 から実装されている。

非常にシンプルな API で使いやすい半面、さまざまな制約がある。基本的に保存できる値はテキスト情報のみ。また、ほとんどの Web ブラウザーは、保存できるデータのサイズがサイトごとに 5M から 10M バイト程度と小さい。

### ■ Web SQL Database

この API は、Web Storage では対応していなかった大量のデータを保存する仕組みを提供するために考え出された。リレーショナルデータベースで使われる SQL 文を使って、レコードを登録したり検索したりできる。すでに、レンダリングエンジン「WebKit」をベースとした Safari や、オープンソースの Web ブラウザープロジェクト「Chromium」をベースとした Chrome と Opera が、Web SQL Database API をサポートしている。

しかし、SQL の互換性の問題をはじめ各種理由から、W3C での標準化作業が中止となった。現在は作業が打ち切られた文書である Group Note という扱いになっている。Safari や Chrome、Opera は、すでに実装したこの API を取りやめる予定はないが、これから新たに開発するアプリでは、次に紹介する「Indexed Database API」を使ったほうがよい。

### ■ Indexed Database API

この API は、オブジェクトストアと呼ばれる仕組みを提供する API である。簡単に言うと、JavaScript のデータオブジェクトをそのまま保存する仕組みだ。値には、階層構造を持ったデータを格納し、それらのデータにインデックスを設

定できる。つまり、データを格納した後、インデックスが設定された値を素早く検索可能だ。

こういった特徴から、Indexed Database API は、大量のレコードを持ったデータを格納するのに役立つ。例えば、日本全国の郵便番号と住所のデータをまるごと保存可能だ。こういった膨大なレコードから、特定の値を素早く取り出す仕組みが Web ブラウザーの中で実現できる。

Indexed Database API は、バイナリーデータもサポートしている。そのため、画像などのデータも保存でき、それを URL 形式で参照するとページ上に表示できる。

すでに、主要なブラウザーはすべて Indexed Database API を実装済みだ。

### ■ File API: Directories and System

この API は、JavaScript からディレクトリーやファイルとしてデータを保存する枠組みを提供する。ディレクトリーを作ったり、指定のディレクトリーの中にファイルを保存したりできるため、開発者が直感的に理解しやすい。Indexed Database API と同様に、バイナリーファイルの保存と読み出しをサポートしており、保存したファイルには URL でアクセスできる。

ただし、この API でデバイス上の任意のファイルシステムに、ディレクトリーを作ったりファイルを作ったりできるわけではない。あくまでも Web ブラウザーが用意した仮想的なファイルシステムが対象だ。

Chromium ベースの Chrome と Opera がこの API を実装している。だが、W3C では仕様策定作業は中止となり、現在は Group Note という扱いになっている。とはいえ、代替の API がないことから、利用できるブラウザーは限られるものの、ファイルの保存が必要なら、しばらくはこの API を使わざるを得ないだろう。

## GPS を活用して高精度な位置情報を取得

次に、デバイスのさまざまなセンサーを活用するための API を紹介しよう。

まずは位置情報を取得するための「Geolocation API」である。この API は、

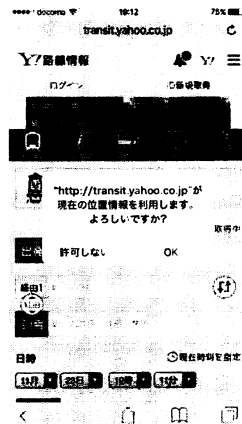


デバイスの位置を表す緯度と経度を取得する API で、ハードウェアアクセス系 API の中でも、最も普及しているものの一つだ。すでに多くの Web サイトで使われている。Geolocation API は、移動中であれば、移動速度や移動方向、GPS 高度といった情報まで取得できる。

例えば、Yahoo! JAPAN の Yahoo! 路線情報では、Geolocation API を使って現在位置を特定し、出発地点の住所をセットできる (図 2-30)。初めてこの機能を使うときは、利用許可のダイアログが表示される。2 回目以降は、ダイアログ表示なしに利用可能だ。

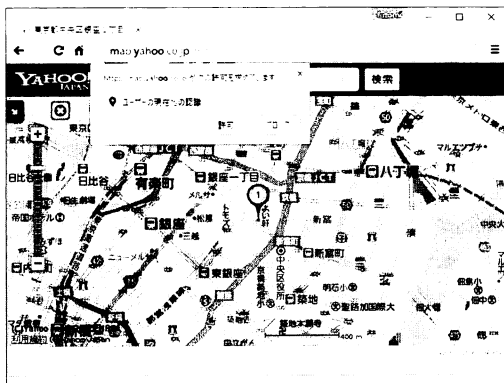
デバイスが GPS に対応していれば、精度が高い位置情報が得られる。GPS に対応していなければ、携帯基地局や無線 LAN、IP アドレスなどの情報を駆使して、位置情報を割り出す。例えば、Yahoo! 地図は GPS が搭載されていない PC でも現在位置を表示する機能を持つ (図 2-31)。

ただし IP アドレスで位置を判定した場合、位置情報の精度は低く、該当の都道府県の代表的な場所を指し示すことが多い。具体的な現在位置を示すケースでも、環境によっては実際よりも離れた位置が表示されることがある。精度に課題はあるものの、都道府県が判定できるだけでも便利なサービスは多い。場所に結



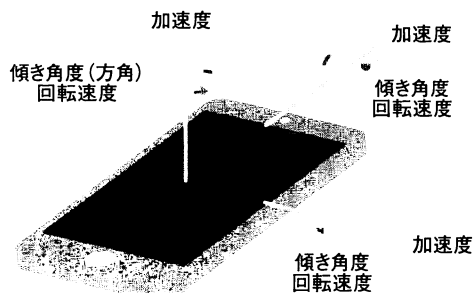
**図2-30** 位置情報を利用する許可を求めるダイアログ

Geolocation APIで位置情報を取得する際、初回のみ利用許可を問うダイアログが表示される。提供：Yahoo! 路線情報



**図2-31** PCでも現在地の地図を表示できる

GPSを持たないデバイスの場合、IPアドレスで位置情報を判定する。提供：Yahoo! 地図



**図2-32** デバイスの傾き角度、加速度、回転速度を取得

DeviceOrientation Eventを使うと、デバイスの3軸の傾き角度や加速度、回転速度をリアルタイムに取得できる

び付いたサービスを提供する場合には、必須の API である。

## コンパスと加速度センサーで傾きや回転を把握

コンパスと加速度センサーの結果を複合的に扱う API として「DeviceOrientation Event」がある。DeviceOrientation Event を使うと、開発者はセンサーを区別せずに、デバイスの3軸の傾き角度、加速度、回転速度をそ

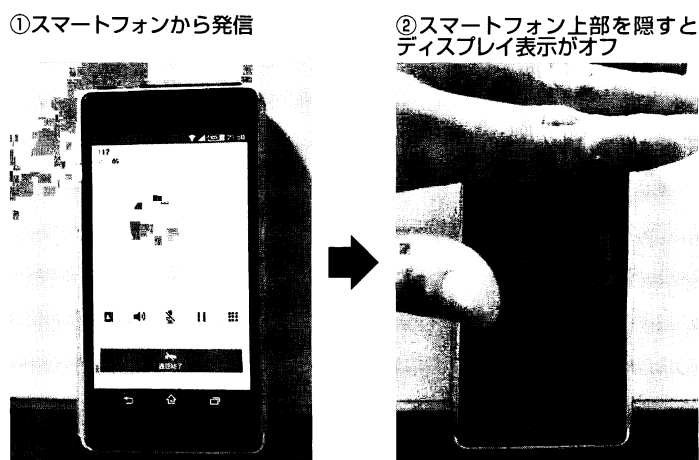
れぞれリアルタイムに取得できる（図 2-32）。

この API は、単純なコンパスアプリはもちろんのこと、スマートフォンを、任天堂のゲーム機「Wii U」のようなリモコンとして何かを操作するためのアプリや、傾きを利用したゲームアプリなどに使える。

### 近接センサーを活用してディスプレイ表示を制御

スマートフォンには、デバイスからの物体の距離を計測する近接センサーが搭載されている。例えば、スマートフォンで通話中に本体を耳に当てると、近接センサーがそれを検知してディスプレイ表示がオフになる。これは通話中に誤ってタッチパネルを触ってしまい、意図せぬ動作につながらないように配慮された仕組みである。

ほとんどのスマートフォンは、本体上部のインカメラの左右のいずれかに組み込まれている。通話料金が発生してしまうが、試しにスマートフォンから電話をかけて、発信後にスマートフォンの上部を手で隠してみしてほしい。ディスプレイ表示がオフになることが分かる（図2-33）。ただし、スマートフォンによっては、



**図 2-33** 近接センサーで通話中のディスプレイ表示をオフにする

スマートフォンから発信後、本体の上部を手で隠すと、ディスプレイ表示が消える。近接センサーは、スマートフォン上部のインカメラに組み込まれていることが多い

本体を寝かした状態では反応しない場合がある。

この近接センサーを扱うのが「Proximity Events」という API だ。この API は、物体を検知したらイベントを受け取れるようにする。物体がどの程度近づいているのかはデバイスに依存するが、1cm 程度で反応することが多い。

Proximity Events では、物体までの距離を取得する仕組みも規定されている。とはいえ、スマートフォンでは、前述のような用途で近接センサーが組み込まれているため、その精度は非常に低い。1cm 以内に物体が存在するかどうかを区別する程度だ。

## 光センサーで光の強さを取得し輝度を調節

スマートフォンを直射日光が当たる明るい場所で使うときと、室内で使うときとは、ディスプレイの輝度が異なる。これは、スマートフォンに組み込まれた光センサーが環境光の強さを計測しているからだ。

「Ambient Light Events」を使うと、デバイスの環境光の強さをリアルタイムに取得できる。この API は、光の強さが変化すると、イベントとしてその光の強さをルクスの単位で教えてくれるという、非常にシンプルな API だ。

多くのスマートフォンには光センサーが組み込まれているが、この API を実装しているのは Firefox のみだ。そのため、Firefox OS 搭載のスマートフォン、または Android 端末にインストールした Android 版 Firefox でないと、この API は利用できない。

## カメラやマイクの情報をブラウザーで活用

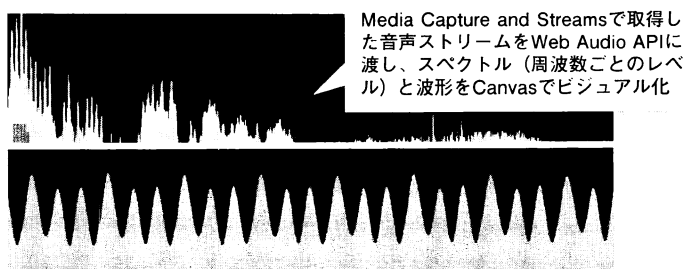
デバイスに内蔵された（またはデバイスに USB などで接続された）カメラやマイクロフォンからの映像や音声を取得するには、「Media Capture and Streams」を使う。この API によって、カメラの映像ストリームやマイクロフォンの音声ストリームを、HTML 上の video 要素や audio 要素に流し込めるようになる。2016 年 6 月時点で、Edge、Chrome、Firefox、Opera、Android 標準ブラウザーが実装している。

この API は、非常に応用範囲が広い。応用例の一つに、ビデオ会議などを実現する WebRTC がある。WebRTC は、相手とピアツーピアで接続するための仕組みと、ピアツーピアで接続した後に映像や音声のストリームを送受信する仕組みとを提供する。その際、Media Capture and Streams を使い、自身のカメラとマイクロフォンの映像と音声を取り出す。

Media Capture and Streams は、もともとは WebRTC の一環で作られた仕様だが、単独でも利用できる。例えば、カメラからの映像やマイクロフォンからの音声をリアルタイムに解析可能だ。カメラからの映像であれば、video 要素に流し込んだ後、そのコマを Canvas にコピーすれば、Canvas からピクセルごとの RGBA 値が得られる。何かしらの加工をして、写真としてクラウドにアップロードもできるし、リアルタイム解析に使うこともできる。

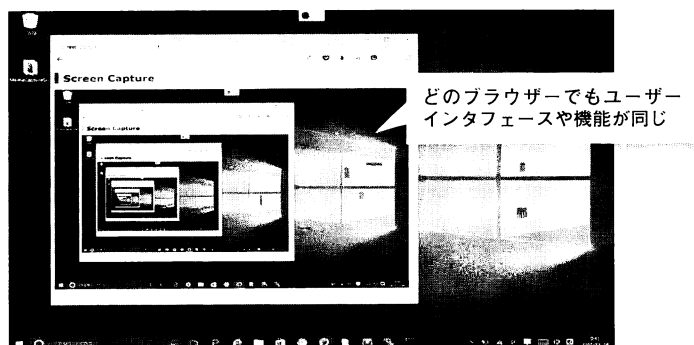
マイクロフォンからの音声であれば、「Web Audio API」を使ってさまざまな音声解析が可能となる。図 2-34 は、Media Capture and Streams から得られた音声ストリームを Web Audio API に引き渡し、そこからスペクトル（周波数ごとのレベル）と波形を、Canvas を使ってリアルタイムにビジュアル化した例だ。

このように映像や音声を JavaScript から詳細に解析できるようになるのだが、映像のリアルタイム解析は非常に重い処理だ。JavaScript を使ってメインスレッドで頻繁に解析すると、おそらく Web ブラウザーが固まった状態になってしまう。それを回避するためには、後述の Web Workers が役に立つ。



**図2-34** 音声のスペクトルと波形をビジュアル化

Media Capture and StreamsとWeb Audio API、2DグラフィックスのCanvasを使った音声解析の例



**図2-35** スクリーンキャプチャで取得した画像の表示例

無限の入れ子構造となって表示されるため、画面共有以外の用途はほぼない

## 画面共有に活用できるスクリーンキャプチャー

スクリーンキャプチャーは、WebRTC で画面共有時に使うことを想定した機能だ。というのも、キャプチャーした画面をそのままブラウザ上に表示してしまうと、マトリョーシカ人形のように、無限の入れ子構造になってしまうため、画面共有以外の用途では使いにくい（図 2-35）。

Chrome と Firefox は、Media Capture and Streams を独自に拡張して、スクリーンキャプチャーをサポートしている。当初、Chrome はこの機能を実験的に提供していたが、現在は中止している。Chrome Extension か Chrome Apps のみで利用可能だ。

一方、Firefox は設定を変更すれば、この機能を利用できる。アドレスバーに「about:config」と入力して設定画面を表示し、「media.getusermedia.screensharing.allowed\_domains」という設定値に、該当の Web アプリケーションがホスティングされたドメインを追加する必要がある。

## MIDI データを楽器と PC の間でやり取りする

MIDI とは、電子楽器の演奏データ交換の規格である。「Web MIDI API」は、MIDI データを、USB を通して交換するための API だ。実際には、Web ブラウザー

を実行する PC などのデバイスと、電子楽器を USB でつないで使う。

この API を使うと、スクリプトから MIDI データを電子楽器に送り込んで演奏させたり、逆に、電子楽器での演奏を MIDI データで取得したりできるようになる。現在は Chrome と Opera しか Web MIDI API を実装していないが、国内では電子楽器メーカーが集まる団体「AMEI（音楽電子事業協会）」が、W3C での標準化活動に積極的に関わっている。

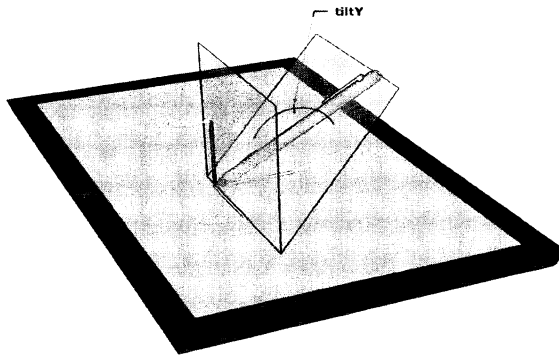
## マウスやペン、タッチの情報を包括的に扱う

従来、ポインティングデバイスといえば、マウスとタッチが主流だった。タッチによる操作は、「Touch Events」と呼ばれる標準仕様がスマートフォン向けブラウザに実装されている。現在は、この仕様を使って JavaScript からタッチ操作を把握している。

ポインティングデバイスとしてペンも多く使われるようになったため、Web アプリでは、マウスやタッチに加えペンもサポートしたいというニーズがある。実は、マウスのイベントは、どのポインティングデバイスでも機能するため、クリックとタップだけであれば問題はない。また、タッチに特有の操作は、Touch Events で規定されたイベントで詳細に把握可能だ。しかし、マウスによる簡単なクリックと、タッチによる細かい操作の両方をサポートしようとする、別々のイベントをハンドリングする必要がある。さらに、ペンが加わったため、ペン特有の機能も必要となってきた。

そこで提案されたのが「Pointer Events」だ。これまでポインティングデバイスごと別々に用意されていたイベント仕様を、この Pointer Events で包括的に扱えるようにした。イベントをキャッチすると、それがどのタイプのポインティングデバイスなのかを区別できる。また、そのポインティングデバイス特有の詳細情報も把握できる（図 2-36）。

現在、IE 11、Edge、Firefox が Pointer Events をサポートしている。ポインティングデバイスが多様化した今、その他のブラウザでの実装が待たれる。



**図 2-36** W3C Pointer Eventsで扱うペンの傾き情報

ペンの場合、傾きを3軸からの角度で把握できる。出典：W3C Pointer Events仕様  
(<http://www.w3.org/TR/pointerevents/>)

## バックグラウンド処理で描画や入力をスムーズに

近年のデバイスは、スマートフォンですら CPU に複数のコアを搭載したマルチコアとなっている。2015年の秋から冬にかけて発売されたスマートフォンの場合、8個のコアを持つオクタコアも登場している。Raspberry Pi のような安価な Single Board Computer ですら4個のコアを持つクアッドコアだ。

ところが、Web ブラウザーは一つのページをシングルスレッドで処理している。このスレッドには、JavaScript コードの実行だけではなく、レンダリングやユーザー入力といった処理も含まれている。もし JavaScript の処理が重いと、そのページは固まった状態になってしまう。これではマルチコアを生かせない。そこで登場したのが「Web Workers」だ。

Web Workers は、JavaScript の処理をワーカースレッドとしてバックグラウンドで処理する仕組みを提供する。とりわけ重くて時間がかかる処理を Web Workers に任せることで、メインスレッドで処理しているレンダリングやユーザー入力がスムーズに処理できるようになる。特にアニメーションを伴う Web アプリケーションでは、重要な API の一つといえる。

前述の Media Capture and Streams を使った映像ストリームのリアルタイム処理などでは、WebWorkers の利用は欠かせないだろう。



Web 技術の標準化を進める W3C では、IoT (Internet of Things) が流行っていることもあり、ハードウェアリソースにアクセスできる API の標準化はホットなテーマだ。これまでデスクトップアプリケーションとして作られてきたクライアントソフトの多くは、ブラウザーベースに置き換えられるのではないだろうか。

### 2-4 のポイント

- ハードウェアリソースにアクセスできるAPIが、Webブラウザーに実装されてきている。
- ストレージ系のAPIは複数あり、用途に応じて選択する必要がある。
- デスクトップアプリとして開発されたクライアントソフトのうち、Webブラウザーベースで実現できるものも増えていくだろう。

## 2-5 ネットワーク

# 表示をリアルタイムに更新 目的ごとに通信方式を選択

Web コンテンツにはリアルタイム性がますます求められるようになってきた。以前から使われてきた AJAX も現役だが、さまざまな通信方式が Web ブラウザーから利用できるようになっている。最新の通信系 API を一挙に紹介する。

従来の Web コンテンツでは、Web ブラウザーのアドレスバーに URL を入力して該当のページが表示された後、ユーザーが何かアクションを起こさない限りは変化しなかった。一方で、現在の Web コンテンツは、ページが表示されたら終わりではなく、一度表示されたページがリアルタイムに更新されていくものが増えている。

リアルタイムに更新される Web コンテンツとしては、Facebook や Twitter などが分かりやすいだろう。直近のタイムラインが表示され、しばらくすると、新しい未読の投稿が画面上に刻々と表示されたり、ログイン中の友達がリアルタイムに表示されたりする。

リアルタイム性は高くはないが、Gmail などの Web メールクライアントもそうだ。デスクトップのメールソフトと同様に、新着のメールが届けば、画面には新着メールのサブジェクトが自動的に表示される。

このように Web 技術を使ってリアルタイムにコンテンツを表示する仕組みを「リアルタイム Web」と呼ぶ。リアルタイム Web は最近の Web を使ったサービスでは、ごく当たり前の機能になってきた。以下では、リアルタイム Web を支える API について解説する。

## HTTP による非同期通信を実現する API

以前から使われてきた通信系 API として、「XMLHttpRequest」がある。これは HTTP クライアントとして JavaScript から HTTP 通信を実現するための API だ。非同期にサーバーと通信して新たな情報を取得し、それをページ上に反映す

るために使う。こういった手法はAJAXと呼ばれ、最近でもまだ使われている。

当初、XMLHttpRequest は、XML データとテキストデータしか扱えなかった。しかし、W3C がXMLHttpRequest の仕様をバージョンアップして、現在は、JSON、バイナリーデータ、HTML ドキュメントも扱えるようになっている。現在のメジャーな Web ブラウザーであれば、この新しい機能を利用できる。

XMLHttpRequest でバイナリーデータを扱えるようになった点は、Web の進化では重要だ。例えば、XMLHttpRequest で画像ファイルやビデオファイルをダウンロードできるようになる。こういったファイルはバイナリーデータとして取得しても、そのままの状態だと JavaScript で可能な操作は限定される。

そこで画像データを、Indexed Database API を使ってバイナリーデータのままストレージに保存したり、File API: Directories and System を使ってファイルとしてストレージに保存したりする。これにより、保存後に URL としてデータを参照できるようになり、その後はサーバーからデータを取得せずに、画像やビデオをページ上に表示でき、パフォーマンス向上につながる。

もちろん、バイナリーデータを JavaScript から扱える。ArrayBuffer と呼ばれる形式でバイナリーデータを読み取り、それを JavaScript から解析できる。

## HTTP でのダウンロードの進捗を通知する

XMLHttpRequest でバイナリーデータなど巨大なファイルをダウンロードできるようになると、そのダウンロードの進捗を知りたくなる。ファイルが大きいほどダウンロード時間は長くなるため、プログレスバーなどを表示したい。これを実現するのが「Progress Event」だ。

Progress Event は、XMLHttpRequest を使った HTTP ダウンロードの進捗を

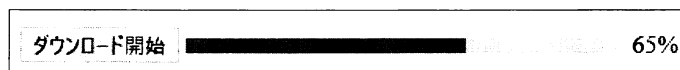


図2-37 ダウンロードの進捗を表示するprogressタグ

イベントとして通知する仕組みを提供する。ダウンロードを開始する前にコールバック関数をセットしておく、対象ファイルのサイズと、ダウンロード済みのサイズを刻々と通知してくれる。この情報と、HTML5で規定された progress タグを組み合わせれば、簡単にプログレスバーをユーザーに提示できる（図 2-37）。

## 元サイトとは別のサイトと通信可能にする

XMLHttpRequest は、基本的には、元のサイトとは異なるサイトにリクエストを送れないよう制限されている。これは、クロスサイトスクリプティング<sup>注2-2</sup>などの脆弱性からユーザーを守るために導入された。このポリシーを「同一オリジンポリシー」と呼ぶ。

オリジンとは、URL のうち、スキーム（URL の「:」から左側の部分）からポート番号までを指す。例えば、http://example.jp と https://example.jp は同一オリジンではない。http と https といった具合にスキームが異なるからだ。

また、http://example.jp と http://www.example.jp も同一オリジンではない。後者には www が入っているためだ。たとえサイトの運営者が同じでも、オリジンが少しでも異なると、XMLHttpRequest ではクロスサイトと判定され、通信できない。

これでは困る場合も多いはずだ。例えば、HTML や JavaScript のファイルをホスティング（格納）しているサーバーと、XMLHttpRequest で取得したいデータを提供するサーバーのオリジンが違うケースは、ごく当たり前だからだ。これを解決するために用意された仕組みが「Cross-Origin Resource Sharing」である。一般的に、略して CORS と書く。

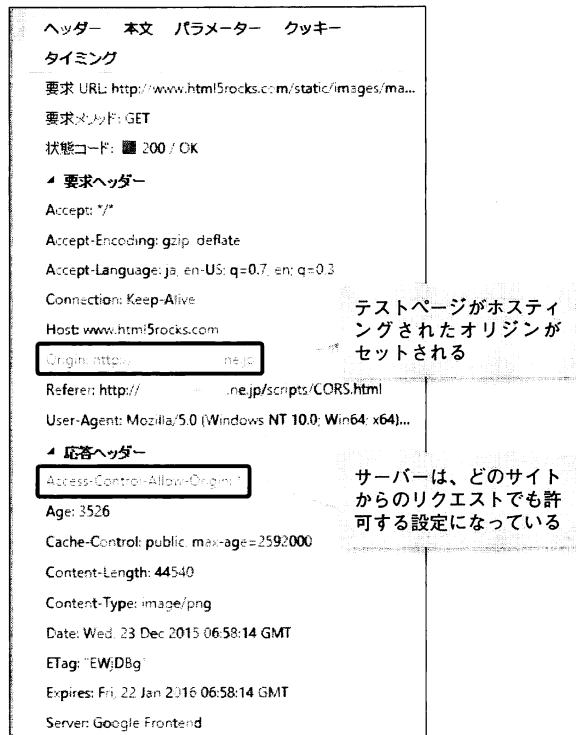
## クロスサイト通信のためのヘッダーを定義

CORS は、実際に Web アプリケーションを開発する上で、非常に重要な仕組

---

### 注2-2 クロスサイトスクリプティング

ブラウザ上で動作する不正なスクリプトを混入する攻撃手法または脆弱性



**図 2-38** クロスサイトリクエストのHTTPヘッダー

Cross-Origin Resource Sharing (CORS)をサポートしている「HTML5 Rocks」(<http://www.html5rocks.com/>)にホスティングされた画像ファイルを取得する際の通信内容。Windows 10のEdgeのF12開発者ツールで表示した

みのため、ここでは重点的に解説しよう。CORSは、JavaScriptから利用できるAPIの仕様ではなく、HTTPリクエストヘッダーとレスポンスヘッダーを定義し、CORSを実現するためのWebブラウザの要件を規定したものだ。

クロスサイトのリクエストを実現するには、サーバー側でCORSに対応する必要がある。米Googleが運営する「HTML5 Rocks」(<http://www.html5rocks.com/>)というサイトは、CORSをサポートしている。これを使ってHTTPヘッダーを覗いてみよう。

図 2-38 は、XMLHttpRequestを使って、クロスサイトとなるHTML5Rocks

にホスティングされている画像ファイルを取得する際に、Web ブラウザーとサーバーとの間でやり取りされた HTTP ヘッダーの内容だ。Windows 10 の Edge の F12 開発者ツールを使って取得した結果である。

まず、要求ヘッダーを見てほしい。Origin というヘッダーに、テストページがホスティングされているオリジンがセットされている。クロスサイトでリクエストを受けたサーバーは、そのリクエストに対して、Access-Control-Allow-Origin というヘッダーを付与しているのが分かるだろう。その値はアスタリスクとなっている。これは、どのサイトからのリクエストでも許可する設定を意味する。

## プリフライトリクエストで設定を事前確認

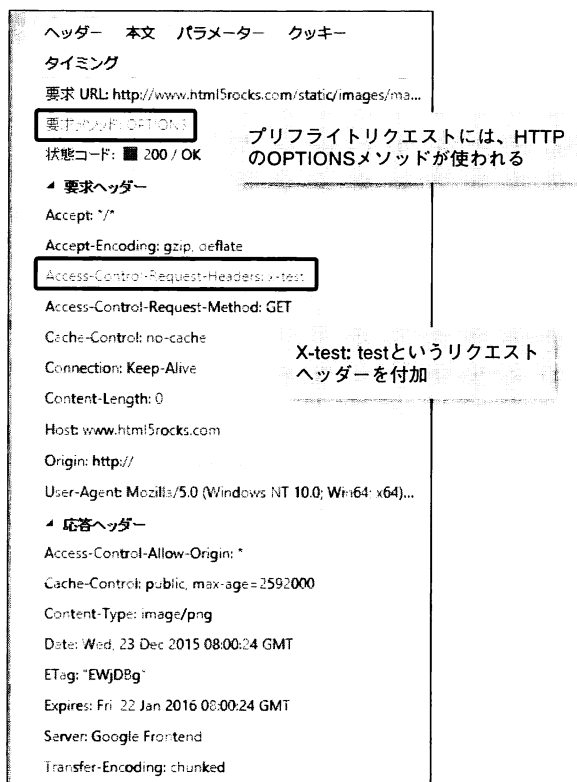
もし、XMLHttpRequest でクロスサイトのリクエストを送る際に独自リクエストヘッダーを付けると、動作が少し変わる。実際のデータを取得する前に、「プリフライトリクエスト」と呼ばれる事前のリクエストを送信するのだ。

試しに、XMLHttpRequest の `setRequestHeader()` メソッドを使って、「X-test: test」というリクエストヘッダーを加えてみよう。すると、図 2-39 のようなプリフライトリクエストを発行していることが分かる。プリフライトリクエストには、HTTP の OPTIONS メソッドが使われる。

実は、このリクエストは失敗に終わる。プリフライトの要求ヘッダーでは、Access-Control-Request-Header に `x-test` がセットされたが、応答ヘッダーには、それが許可されたことを意味するヘッダーが存在しないからだ。そのヘッダーとは、Access-Control-Allow-Headers だ。サーバー側では、Access-Control-Allow-Headers ヘッダーに、クライアントからのリクエストに対する許可ヘッダーをセットする必要がある。

このように、もしプリフライトリクエストで許可されていないことが分かったら、Web ブラウザーは、XMLHttpRequest の本来のリクエストは行わず、失敗したとして処理する。

プリフライトリクエストを送信する条件は CORS 仕様で細かく規定されている。簡単にまとめると、単なる画像の取得のように、通常の HTML の処理でも行われていることであればプリフライトリクエストは実行しない。だが、JavaScript



**図2-39** プリフライトリクエストのHTTPヘッダー

独自リクエストヘッダーをつけると、実際のデータを取得する前に、プリフライトリクエストと呼ばれるリクエストが送信される

から XMLHttpRequest を使わなければ実現できないような処理（独自ヘッダーを加えるなど）が含まれると、プリフライトリクエストを実行する。

## サーバーからイベントに応じてデータを送信

リアルタイム Web を実現するためには、Web ブラウザーからのリクエストに応じてサーバーが応答するのではなく、何かしらのイベントが発生した時点でサーバーから情報を送信するのが望ましい。リアルタイム性を実現する方法とし

て手っ取り早いのは、Web ブラウザーが一定間隔でサーバーに問い合わせるポーリングだ。

しかし、ポーリングの間隔が長いとリアルタイム性を損なってしまう。また、短い間隔で Web ブラウザーが何度もリクエストを繰り返せば、サーバー側のパフォーマンスにも影響を及ぼしてしまう。

そこで HTTP を接続したままにして、データを逐次送るロングポーリングという仕組みが使われるようになった。これを実現する手法を「Comet」と呼ぶが、あくまでも Comet 手法を表す名前であって、その仕組みは、Web サービス事業者が独自に作るか、何かしらのフレームワークを使う必要があった。当然、これらに互換性はない。

そのような状況の中、データのフォーマットを含めて標準化したものが「Server-Sent Events」だ。Server-Sent Events では、Content-Type は text/event-stream と規定されている。サーバーから送信されるデータフォーマットは、次の通りだ。

```
data: 1回目のメッセージ
```

```
(空白行)
```

```
data: 2回目のメッセージ
```

```
(空白行)
```

```
data: 3回目のメッセージ
```

```
(空白行)
```

このように、データを送る際には、最初に data: を入れ、その後ろにメッセージを付け加える。空白行を送ると、メッセージが終了したとみなされる。

Server-Sent Events は、一般的な Web サーバーがあれば簡単に実現できる。その手軽さがメリットといえる。PHP や Perl など、Server-Sent Event に対応したサーバーアプリケーションが簡単に作れる。

ただし、実際の運用では注意が必要だ。すべてのクライアントからの HTTP コネクションが維持され続けるため、同時接続のクライアントの数が増えれば、サーバーのリソースを食いつぶしてしまう。もし、採用している HTTP サーバーが、



リクエストごとに子プロセスを生成する仕組みを採用している場合は、注意が必要だ。

また、Server-Sent Events は、HTTP のロングポーリングの仕組みを使っている。そのため、一度確立した HTTP コネクション上で、クライアントからサーバーに情報を送ることはできない。情報を送るには、クライアントが別途 HTTP リクエストを送る必要がある。

Server-Sent Events は、一般消費者向けサービスでは使いづらいかもしれない。だが、企業内システムのように同時接続数が正確に見積もれる環境であれば、一方方向とはいえ、リアルタイム情報提供の有力な手段の一つといえる。

## 双方向のリアルタイム通信を実現する WebSocket

Server-Sent Events は一方方向のリアルタイム情報通知であったが、双方向でリアルタイムに情報を送受信できる仕組みもある。これが「WebSocket API」だ。

WebSocket API は Server-Sent Events と違い、HTTP ではなく WebSocket プロトコルを使う。このため WebSocket を利用するには、WebSocket サーバーが必要となる。現在では、さまざまな WebSocket サーバーがオープンソースで提供されているため、サーバー構築に困ることはないだろう。

有名な WebSocket サーバーとしては、Apache Tomcat や Jetty<sup>注23</sup> が挙げられる。また、Node.js とそのモジュールである Socket.IO<sup>注24</sup> を使う方法もよく用いられる。WebSocket は、プロトコルは IETF で、Web ブラウザー上で

---

### 注2-3 Apache TomcatやJetty

Apache Tomcat <https://tomcat.apache.org/>

Jetty <http://www.eclipse.org/jetty/>

---

### 注2-4 Node.jsとそのモジュールであるSocket.IO

Socket.IO <http://socket.io/>

---

### 注2-5 プロトコルはIETFで、Webブラウザ上でJavaScriptから扱うためのAPIはW3Cで標準化

IETF - RFC 6455 The WebSocket Protocol

<http://tools.ietf.org/html/rfc6455>

W3C - The WebSocket API

<http://www.w3.org/TR/websockets/>

JavaScript から扱うための API は W3C で標準化<sup>注25</sup> されている。

近年のブロードバンドルーターなどの通信機器には、ネットワークセキュリティを確保するために、必要なプロトコルしか通さないようファイアウォール機能が標準的に装備されている。既知のプロトコルなら問題ないが、新たなプロトコルでは、多くのユーザーが利用できなくなることは想像に難くない。とりわけ、企業内ネットワークでは、外部への接続にかなりの制限が課せられている。

それを解決するために、WebSocket はいったん HTTP を使ってサーバーに接続する。次に HTTP のアップグレードを使って、プロトコルを WebSocket に変更するのだ。通信機器に対しては、あたかも HTTP 通信を行っているかのよう

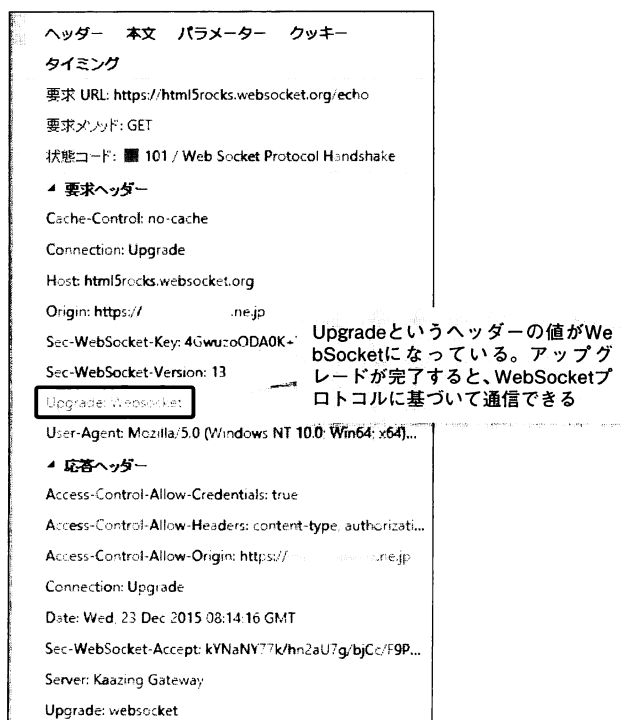


図2-40 WebSocketプロトコルへのアップグレードの様子

WebSocket APIを使ってWebSocketサーバーに接続をリクエストしたときの通信内容

に見せかける。実際に WebSocket API を使って WebSocket サーバーに接続をリクエストすると、Web ブラウザーは前ページの図 2-40 のようなヘッダーをサーバーに送信する。

要求ヘッダーに、Upgrade というヘッダーが存在するのが分かる。その値は WebSocket だ。アップグレードが完了すると、その後は、WebSocket プロトコルに基づいて、データの送受信ができるようになる。WebSocket コネクションが確立すれば、テキストだけでなく、バイナリーデータも送受信可能だ。

WebSocket は、すでに多くのサービスで利用実績がある。単純にリアルタイムデータをサーバーからプッシュするために使われるケースが多いが、中には、デバイスを制御するためのリモコン代わりに使われることもある。スマートフォンのアプリとデバイスがそれぞれ WebSocket サーバーに接続し、スマートフォンからデバイスを制御するのだ。一度コネクションが確立すれば、データの遅延は非常に小さいため、一般的に使われている赤外線リモコンと比べても遜色はない。

また、大規模なコネクションを受け付けるための研究開発も盛んだ。東芝は 100 万台以上の IoT 機器を遠隔から操作するためのクラウド集約型システム<sup>注26</sup>を開発している。NTTはWebSocketをはじめとしたWeb配信基盤の研究開発<sup>注27</sup>に取り組んでいる。

なお、2-3 で紹介した「WebRTC」にもデータ送信の機能がある。WebRTC は、ピアツーピアで相手と直接接続して映像や音声ストリームを送り合う仕組みだ。ピアツーピアで相手と接続するための手順は非常に複雑だが、一度コネクションが確立してしまえば、メッセージの到達遅延は、サーバーを経由する WebSocket と比べてかなり短くなる可能性がある。あくまでも一対一の通信ではあるが、もし遅延を少しでも短くしたいケースを想定するなら、WebRTC のデータ通信も選択肢の一つになるだろう。

---

**注2-6 クラウド集約型システム**

[https://www.toshiba.co.jp/rdc/detail/1501\\_01.htm](https://www.toshiba.co.jp/rdc/detail/1501_01.htm)

---

**注2-7 Web配信基盤の研究開発**

<http://www.ntt.co.jp/journal/1301/files/jn201301042.pdf>

## ブラウザー内の別ページと連携

これまでと少し毛色が違うが、外部との通信ではなく、Web ブラウザー内部での通信について紹介しよう。

対象となるのは HTML の `iframe` タグだ。iframe タグを使うと、ページ内に別のページを組み込み表示できる。しかし、セキュリティ対策のため、組み込まれた iframe 内のページの挙動はいくつか制限されている。オリジンが異なるページを iframe で組み込むと、iframe で組み込まれたページから、親のページには一切アクセスできない。その逆も同様だ。

それぞれのページが連携する仕組みを入れたい場合には、この制約が壁になってしまう。これを解決するのが「HTML5 Web Messaging」である。HTML5 Web Messaging では、それぞれのページ同士で通信コネクションを確立する。これを「メッセージチャネル」と呼ぶ。

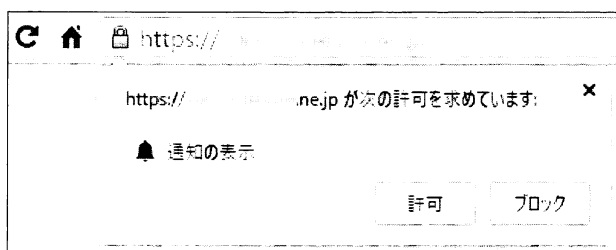
このメッセージチャネルを通して、お互いに何かしらのメッセージを送り合って、ページ連携が可能となる。このメッセージチャネルは、テキストデータだけでなくオブジェクトの送信もできる。

HTML5 Web Messaging では、メッセージを送る際に、相手のオリジンを指定する。そのため、もし一つのページに、異なるオリジンのページを組み込んだ iframe が複数存在しても、指定のオリジンのページにしかメッセージを送れない。

## Web ブラウザーにも通知機能を実装できる

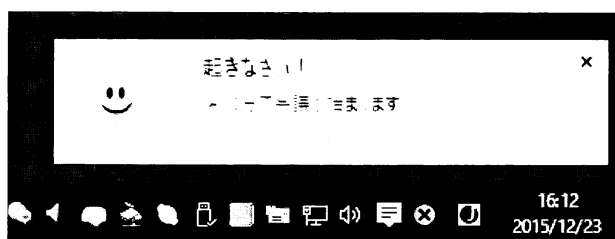
最後に紹介するのは、デスクトップアプリやスマートフォンのネイティブアプリなどでよく使われている通知機能だ。実は Web でも通知機能が用意されている。それが「Web Notifications」という仕様だ。これによって、Web ブラウザーを通して JavaScript から通知を表示させることができる。すでに Edge、Firefox、Chrome、Safari、Opera が Web Notifications をサポートしている。

通知機能を使うためには、事前にユーザー許可が必要だ。JavaScript からユーザーの許可を求める処理を加えておくと、ダイアログが表示される（図 2-41）。一度ユーザーが許可すれば、以降は JavaScript から通知を表示できる。図 2-42 は、



**図2-41** 通知機能を使うためのユーザー許可ダイアログ

一度許可されると、次回以降はJavaScriptから通知を表示できる



**図2-42** Windows 10のデスクトップに通知を表示

メッセージだけでなくアイコンも指定できる

Chrome を使って Windows 10 のデスクトップに通知を表示した画面だ。通知には、メッセージだけでなくアイコンも指定できる。

Web Notification は、Web ブラウザーが起動している間でなければ通知を表示させられない。利用シーンが限定されてしまうが、長時間使い続ける Web アプリケーションでは、Web ブラウザーが最小化される場合があるため、そういった場合の通知としては役に立つだろう。

通知と聞くと、スマートフォンでよく使われるプッシュ通知をイメージした読者も多いだろう。Web アプリケーションでも、ネイティブアプリのプッシュ通知と同じように、Web ブラウザーを起動していなくても外部からプッシュ通知する仕組みが用意されている。次パートでは、プッシュ通知を実現する仕組みを含め、W3C で提案中の API のうち、将来有望と考えられる API について解説する。

## 2-5 のポイント

- AJAXはテキストだけでなくバイナリーデータも扱えるよう進化。
- クロスサイト通信の制約を超えるCORSの理解は重要。
- リアルタイムWebを実現する方式として、Server-Sent Event、WebSocket、WebRTCが有望。

# プッシュ通知や BLE に対応 ネイティブアプリに近づく

ネイティブアプリになかなか近づけなかった Web アプリだが、Web の最前線では着々とネイティブアプリに近づけるための API が標準化中だ。一部は既に Web ブラウザーへ実装されている。最前線の API について紹介する。

これまで、Web アプリケーションの機能や性能を拡充するさまざまな API を紹介してきた。ネイティブアプリにできて Web アプリケーションではできないことは残るが、それでもかなり多くのことができるようになってきた。本パートでは、W3C などの標準化団体で提案された API のうち、注目すべき最新の API を厳選して紹介する。

### ■ Service Workers

恐らく、今 Web 界限で最もホットな API は「Service Workers」だろう。Service Workers を一言で表すと、「Application Cache の次世代バージョン」となる。Application Cache は、一度データがキャッシュされると、オフライン時にあたかもオンラインであるかのように Web アプリを実行する枠組みを提供する。

しかし、実際には使い勝手に難があり、Web 開発者からの評判は低かった。キャッシュするファイルのリストを「マニフェスト」と呼ばれるテキストファイルに静的に記述する必要がある、柔軟なキャッシュ制御ができなかったからだ。現在では、オフラインでアプリケーションを動作させる目的よりも、Web ブラウザーにファイルをキャッシュさせてサーバーの負荷を下げるために使われることが多い。

Service Workers は、単に Application Cache をグレードアップしたものではない。実態に合わせて Service Workers を説明するなら、「Web ブラウザーの HTTP 通信を乗っ取る API」というのが正しいだろう。加えて、スクリプトからキャッシュを制御できる。ローカルにプロキシサーバーを作るようなものと考え

えてよい。

Service Workers は、名前の通り、バックグラウンドでタスクを処理するワーカーレッドで動作する。ただし、同じくワーカーレッドで動作する「Web Workers」とは違う。Service Workers のワーカーレッドは、状況によっては Web ブラウザーを閉じて（終了しても）常駐させておけるのだ。そういう意味では、Service Workers は Web Workers の拡張版ともいえる。

もともとは Application Cache をグレードアップする目的で Service Workers が考案された。だが、常駐可能なワーカーレッドを使うというアーキテクチャーを採用したことで、その用途はかなり広がった。現時点ではキャッシュの制御と、後述するプッシュ通知のみだが、その他の機能も今後提案されるだろう。

## 初回はアプリ登録の処理が必要

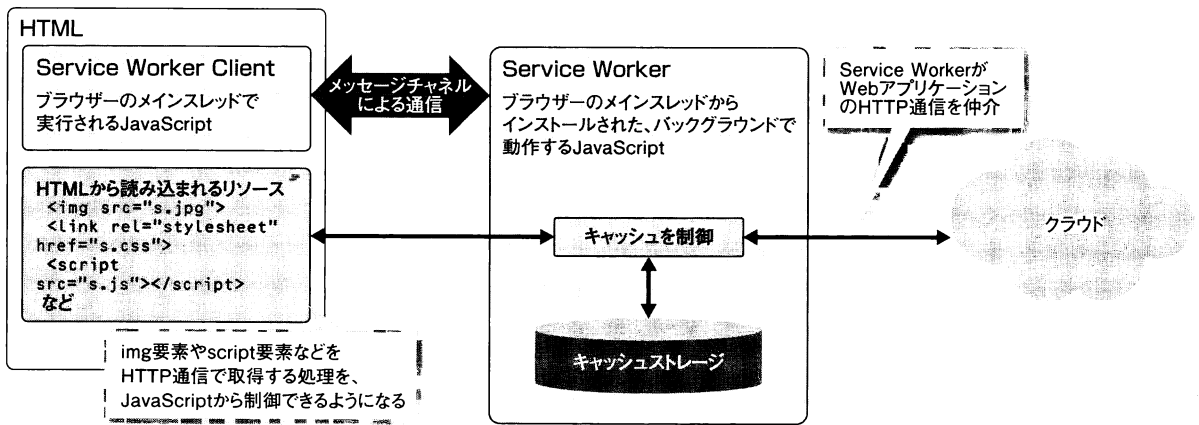
Service Workers はインストールという概念を持つ。ユーザーが初めて Service Workers を使う Web アプリケーションにアクセスすると、登録処理を実行する。もちろん、勝手に登録されるわけではなく、Web アプリケーション側に登録処理を実装しなくてはならない。

一度、Web ブラウザーに Service Worker が登録されると、それ以降は、その Service Worker が Web アプリケーションと Web サーバーの間の HTTP 通信を仲介できるようになる（図 2-43）。これにより従来、JavaScript からは手の届かないところで実施されていた HTTP 通信を制御できるようになる。例えば、HTML の `img` 要素や `script` 要素などは、Web ブラウザーが自動的に HTTP で Web サーバーからデータを取得していた。こうした処理を JavaScript から制御できるようになった。

このほか、Web アプリケーションがリクエストした URL ごとに、キャッシュの存在を調べ、キャッシュがあれば、それを返すこともできる。もちろん、改めてネットワークにアクセスし、サーバーへ取りに行くこともできる。ネットワークにアクセスしてリソースを取得するときは、後述する「Fetch API」を使う。

W3C での Service Workers の標準化はまだ道半ばだが、2016 年 6 月現在、既に Chrome、Firefox、Opera、Android Browser、Chrome for Android の最新





**図2-43** Service Workerのキャッシュ制御

Service Workerは、WebアプリケーションのHTTP通信を仲介し、ファイルをキャッシュする

版で、一部の機能を利用できる。ただし、Service Workersを使うには、WebアプリケーションがSSLで提供される必要がある。非SSLでは動作しないので注意してほしい。

## Fetch API

WebブラウザからHTTPで通信するためのAPIとしてはXMLHttpRequestがあるが、Service Workersのワーカースレッド内では使えない。Service WorkersでHTTP通信のために利用するのがFetch APIだ。Fetch APIは、もともとService Workersのワーカースレッドで使うことを想定して作られたが、XMLHttpRequestと同等の機能を持った汎用的なAPIであるため、メインスレッドでも利用できる。

Fetch APIを使う際の、JavaScriptのコーディングのスタイルについても触れておこう。JavaScriptの標準の言語仕様である「ECMAScript 2015 (ECMA Script 6)」には、非同期処理のために「Promise」と呼ばれる仕組みが新たに導入された。近年、W3Cなどの標準化団体で新たに設計されるAPIでは、非同期処理にPromiseを採用するのが主流となっている。Fetch APIもPromiseを採

## ■XMLHttpRequest

```
var xhr = new XMLHttpRequest();
xhr.open("GET", "test.json", true);
xhr.responseType = "json";
xhr.onload = function() {
  console.log(xhr.response);
};
xhr.send();
```

同じ操作でも、簡単な  
コードで実行できる

## ■Fetch API

```
fetch("test.json")
  .then(response => {return response.json()})
  .then(data => {console.log(data)})
```

**図2-44** Fetch APIは簡単なコードでファイルを取得できる

JSONファイルをダウンロードするコードを、XMLHttpRequestとFetch APIで比較した

用している。

Fetch API は、簡単なことなら少ないコードで実現できるのが特徴だ。実際のコードを見てほしい（図 2-44）。図 2-44 の上はこれまで使われてきた XMLHttpRequest だ。これは単に JSON ファイルをダウンロードするだけのコードである。Fetch API を使うと図 2-44 の下のようなになる。もちろん、XMLHttpRequest と同様に、リクエストヘッダーを加えたり、レスポンスヘッダーを取得したりできる。

Fetch API は、仕様上、断続的にデータを受信するストリームもサポートしている。ダウンロードしたデータをまとめて処理するのではなく、受信する都度、小さい塊ごとに処理する仕組みだ。

Fetch API は、2016 年 2 月時点で、Chrome、Firefox、Opera がサポートしていた。また 2016 年夏ごろにリリース予定の最新の Edge でもサポートされる予定だ。Web ブラウザーを限定すれば今すぐに利用可能とはいえ、仕様そのものがまだ安定していないため、実際のサービスに使うのは時期尚早である。しかし、XMLHttpRequest の代わりに Fetch API が当たり前のように使われるようになるのは、さほど遠い未来の話ではないだろう。

## ■ Push API

プッシュ通知は、ネイティブアプリでは当たり前の機能だが、Web アプリケーションでは実現できていなかった。プッシュ通知がないために、Web アプリでの提供を断念した事例は多いだろう。

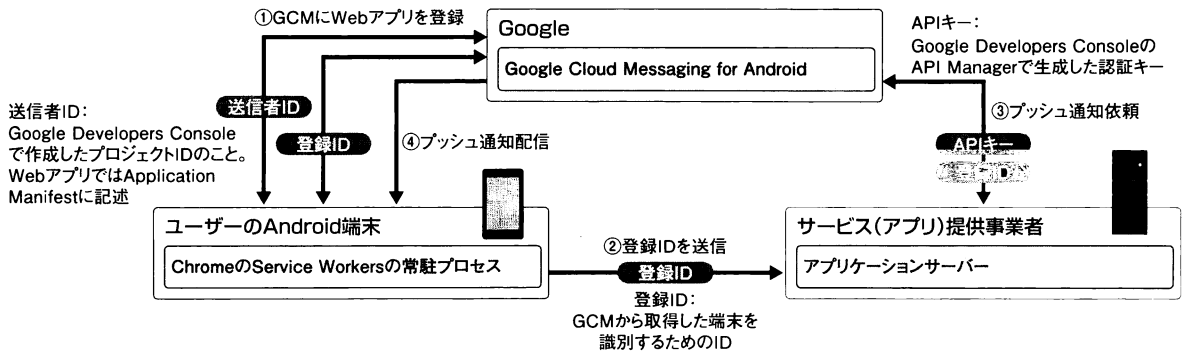
プッシュ通知の API として、以前から W3C で「Push API」が提案されていた。しかし、プッシュ通知では、Web ブラウザーが起動していなくても通知を受けるための仕組みが必要となる。この点は、Web ブラウザーをベースとした Web アプリの最も大きな弱点といえる。そのため Push API は、Web ブラウザーベンダーの協力を得られず、実現には至らなかった。

ところが、Service Workers の提案がきっかけで、風向きが変わってきた。前述の通り、Service Workers は、Web ブラウザーを閉じてバックグラウンドでプロセスを常駐させておける。この Service Workers で生成されたワーカーによって、プッシュ通知の実現の目途が立ったのだ。

現在は、Google と Mozilla が積極的に Push API の標準化に取り組み、Web ブラウザーへ実装を進めている。すでに Chrome の最新バージョンでは、Service Workers によって生成したワーカースレッドで、Push API を使ってプッシュ通知を受信できる。さらに、Android 版 Chrome では、Chrome が起動していなくてもプッシュ通知を受信可能だ。ただし PC 版では、Chrome を閉じた状態でのバックグラウンドのプロセス常駐はサポートされていない。

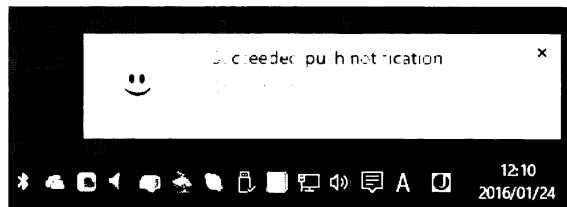
プッシュ通知の送信には、「Google Cloud Messaging (GCM)」を使う（図 2-45）。GCM とは、Android アプリにプッシュ通知を送信するサービスで、Chrome も同じ仕組みを使う。つまり、ネイティブアプリと同じ処理を JavaScript で開発することになる。図 2-46 は、実際に筆者が Push API を試し、Windows 10 のデスクトップに表示した通知だ。既に Facebook は、Chrome に対して Service Workers を使ったプッシュ通知をサポートしている。

今のところ、米 Apple は独自にプッシュ通知の仕組みを用意しており、Service Workers と Push API の実装に関しては動きがない。iPhone はスマートフォン市場の半分近くを占めるだけに、Apple の対応を期待したいところだ。



**図2-45** Google Cloud Messaging (GCM) でのプッシュ通知の流れ

GCMはAndroidアプリにプッシュ通知を送信するサービス。GCMにWebアプリを登録した上で、サービス提供事業者のアプリケーションサーバーからプッシュ通知を依頼すると、ユーザーの端末に通知を配信する。Chromeも同じ仕組みを使い、プッシュ通知をJavaScriptで開発できる



**図2-46** Windows 10のプッシュ通知

## ■ ORTC (Object Real-Time Communications)

2.3などでピアツーピアでテレビ電話を実現するWebRTCについて触れた。まだ標準化作業中のAPIであるにもかかわらず、ChromeやFirefoxが実装し、SkyWayという名称のクラウドサービスも登場している。それだけ期待されている API だ。

WebRTC が向くのはテレビ電話に限らない。監視カメラの映像を一方方向に取得したり、複数の映像を同時に受信したりするなどのケースで利用を考える人もいるだろう。WebRTCが登場して期待が高まる中、新たなニーズが顕在化したが、WebRTC のスコープには入っていない。

というのも、WebRTCではできる限りアプリケーションが開発しやすくなるよう、APIをハイレベルで規定<sup>注28</sup>しているためだ。つまり、かゆいところに手

が届かない。そのため、いくつかのケースにおいては、実現が難しい、または実現できない可能性がある。

この問題の最も大きな要因は、WebRTC がピアツーピア接続の確立のために SDP (Session Description Protocol) と呼ばれる記述構文を採用している点にある。SDP は、主に VoIP (IP 電話) の世界で使われてきた。SDP によって、Web ブラウザー同士がビデオエンコーディングなどの情報を交換し合い、最終的にどの方式で接続するのかを決定する。

WebRTC では、SDP の記述をコントロールする API が用意されていない。そのため、接続確立に必要な手続きも API 化しようという流れになった。こうして登場したのが「ORTC (Object Real-Time Communications)」だ。ORTC は、WebRTC と比べてローレベルの API だ。やりたいことを実現するための自由度は増したが、その分コーディングは難しくなる。将来は、多くの人がやりたいと思うことが簡単にできるよう、JavaScript ライブラリが登場するだろう。

ORTC に関しては、当初から Microsoft が積極的に関わっている。Microsoft は WebRTC を実装していないが、ORTC に関しては、既に Edge に実装済みだ。また、Google も ORTC の標準化には積極的だ。いずれ、ORTC の標準化が成熟した際は、Chrome にも実装されることを期待したい。

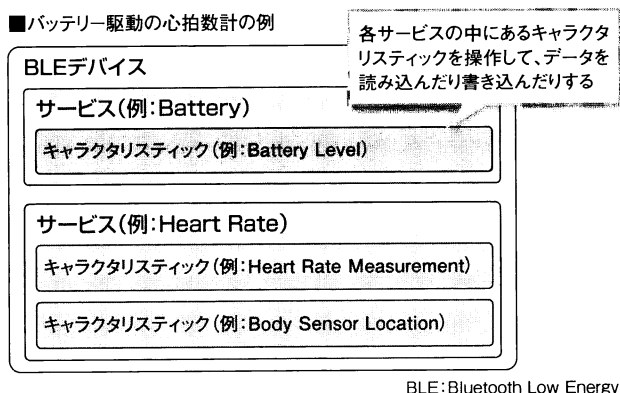
## ■ Web Bluetooth

「Web Bluetooth」は Bluetooth を扱うための API だが、Bluetooth すべてを扱えるわけではない。Web Bluetooth は、「Generic Attribute Profile (GATT)」と呼ぶ通信仕様をサポートしたデバイスの発見と通信をサポートしている。つまり、低消費電力の Bluetooth 規格である「BLE (Bluetooth Low Energy)」をサポートしたデバイスが対象となる。Web Bluetooth は、Web ブラウザー起動時に設定を有効にする必要はあるが、既に Chrome に実験的に実装されている。

BLE をサポートしたデバイスの発見や通信は、場合によっては、セキュリティ

### 注2-8 APIをハイレベルで規定

ここでいう「ハイレベルなAPI」とは、「やりたいこと」を基準に設計されたAPIを指す。その目的を達成する上では、できる限りシンプルに使いやすい設計となる。一方、「ローレベルなAPI」とはあらゆる機能を提供したAPIを指す。



#### 図2-47 GATT (Generic Attribute Profile) の構造

GATTはBluetooth Low Energy (BLE) での通信を規定したプロファイル。GATTには複数のサービスが規定されており、各サービスはユーザーデータを読み書きするキャラクターリスティックを持つ

やプライバシーの脅威になる可能性がある。そのため、Web アプリケーションをSSL サイトから提供しなければいけない。また、JavaScript から自動的に処理を開始することもできない。必ずタップなどユーザーの操作をきっかけに処理を開始する必要がある。

Web Bluetooth は、GATT の構造に従って API が設計されている (図 2-47)。まず、GATT には「サービス」が規定されている。一つのデバイスにサービスは一つとは限らない。通常はいくつものサービスを持つことになる。サービスの中には「キャラクターリスティック」が含まれる。これも一つのサービスにつき一つのキャラクターリスティックが存在するとは限らず、サービスによって、複数のキャラクターリスティックが存在する場合もある。BLE デバイスから何かしらの値を読み取ったり書き込んだりする行為は、このキャラクターリスティックを操作することを意味する。

Web Bluetooth でのやり取りの流れを見ていこう。まず BLE デバイスを発見する際は、サービスの識別子 (UUID など) を指定する。指定したサービスの識別子を持つ BLE デバイスを発見すると、そのデバイスの名称やベンダー ID、プロダクト ID などの情報を取得する。

次に、見つかった BLE デバイスに対して接続を開始する。そして、該当の BLE デバイスからサービスの詳細を取得する。そのサービスの情報からキャラクタリスティックを取得した後、キャラクタリスティックから値を取得、またはセットする。これら一連の流れはすべて非同期で行う。

### ■ Web NFC API

Bluetooth 以外の、スマートフォンでおなじみの通信方式といえば、近距離無線通信の NFC (ISO/IEC 18092) だろう。もちろん、W3C では NFC の API として「Web NFC API」の標準化が検討されており、既に仕様のドラフトも存在する。この活動は主に米 Intel が引っ張っている。Android 向けに Chromium を改造して Web NFC API を実験的に実装してみるなど、その活動は積極的だ。まだ、Chrome などの誰もが利用可能な Web ブラウザーには実装されていないものの、今後の動向が注目される。

Web NFC API が想定するユースケースは、NFC タグからのデータの読み取り、NFC タグへのデータの書き込み、NFC 対応デバイス間でのデータ転送などである。API の使い方は至ってシンプルなので、仕様のドラフトに記載されたサンプルコードをベースに紹介しよう。

```
navigator.nfc.push(
  '{ "prop1": "value1", "prop2": "value2" }'
).then(() => {
  console.log("送信成功");
}).catch((error) => {
  console.log("送信失敗");
});
```

このコードでは単に文字列（ここでは JSON 文字列）を送信し、その結果を評価しているだけの簡単な処理をする。もちろん、データ送信だけでなく、受信の待ち受けも可能だ。

## セキュリティとプライバシーを強化する API

近年、Web ブラウザーに実装された API や W3C にて提案されている API は、OS と密着した機能が多く、セキュリティや利用者のプライバシーに影響を与えるものが複数ある。それぞれの API で組み込まれているセキュリティやプライバシー保護の対策は、Web アプリ開発では非常に重要なので、ここでは詳しく紹介しよう。

セキュリティやプライバシー保護の対策には、いくつかの方法がある。以前から採用されてきた方式の代表例は、利用者にダイアログを表示して許可を得るというものだ。位置情報を取得する Geolocation API、カメラとマイクロフォンを扱う Media Capture and Streams、通知機能を提供する Web Notifications などが代表的である。

すでに説明したように、通信を伴う API の場合は、「同一オリジンポリシー」と呼ばれる機構が組み込まれている。Web アプリと同じサイトにしか通信できないように制限する仕組みだ。代表的な API としては XMLHttpRequest が挙げられる。その他、HTML の iframe タグに読み込むコンテンツに対しても同一オリジンポリシーが適用され、そのコンテンツの挙動は制限される。

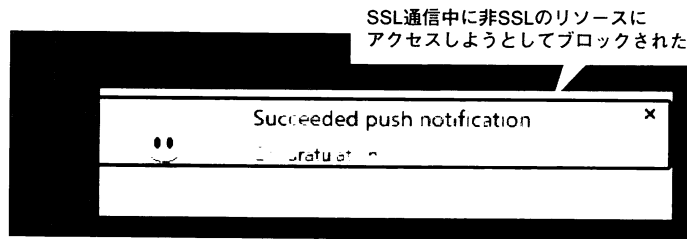
近年は、さらに強力な API が提案されていることもあり、新たなセキュリティやプライバシーの対策が提案され、一部の API で既に実装されている。それは「Mixed Content」と「Secure Contexts」という仕組みだ。最近よく聞く「すべてのサイトは SSL で提供されるべき」というトレンドを反映したものと言ってもよいだろう。

### ■ Mixed Content

Mixed Content とは、SSL で提供された Web コンテンツの中に、JavaScript ファイルや CSS ファイル、画像ファイルなど非 SSL で提供されたリソースが混在、もしくは、XMLHttpRequest など非 SSL のリソースにアクセスする状態を表す。

基本的にこういった状態が発生した場合、Web ブラウザーはユーザーに警告を表示し、デフォルトでは非 SSL コンテンツを表示しない、もしくはロードしない。非 SSL のリソースは、盗聴や中間者攻撃の恐れがあるため、すべて信用し





**図2-48** Mixed Contentのエラー

XMLHttpRequestで非SSLのリソースにアクセスしようとしたためエラーが表示された

ないというスタンスなのだ。

XMLHttpRequest では、同一オリジンポリシーによる制約は、2-5 で説明した CORS (Cross-Origin Resource Sharing) を使うことで回避できたが、Mixed Content に関しては回避策はなく、強制的に処理が中断される。つまり Web ブラウザーは、Mixed Content と判断するとリクエストすらしない。この場合、コンソールにはエラーが表示される (図 2-48)。

ただし、このエラーについては、XMLHttpRequest のコードからは検知できない。そのため、Web アプリケーション開発では、SSL コンテンツと非 SSL コンテンツが混在しないように注意する必要がある。

### ■ Secure Contexts

Secure Contexts とは、強力な API は SSL で提供された Web アプリケーションにしか有効にしないというポリシーのことだ。これまで紹介してきた API のうち、Encrypted Media Extensions や Service Workers などが該当する。このような強力な API は、非 SSL で提供された Web アプリケーションでは全く動作しない。Web ブラウザーが API の呼び出しをブロックするからである。

今後 Web は、アプリケーションだけでなく、通常の Web サイトですら、すべてを SSL で提供するのが当然となってくるだろう。

## 2-6 のポイント

- ネイティブアプリに近づくAPIが続々と登場。
- プッシュ通知や近距離無線を使ったデバイス連携などが可能に。
- セキュリティやプライバシー保護のための制約がより強くなる。

# 索引

---

## [A]

Adobe Flash ..... 8、10、20  
AJAX ..... 21、22、23、24、25、26、137  
AltJS (Alternative JavaScript) ..... 40、41、42  
Ambient Light Events ..... 130  
AngularJS ..... 32、35、36、37、40  
Apache Cordova ..... 12、50、76  
ArrayBuffer 137

## [B]

Backbone.js ..... 32、33、35、36、38  
BLE (Bluetooth Low Energy) ..... 155、156、157  
BYOD (Bring your own device) ..... 55

## [C]

Canvas ..... 66、110、111、112、114、115、131  
Clipboard API and events ..... 101  
CoffeeScript ..... 41  
Comet ..... 142  
Composition Event ..... 100、101  
contenteditable属性 ..... 100  
CORS (Cross-Origin Resource Sharing)  
..... 138、139、140、159  
CrossWalk ..... 75  
CSS3 ..... 92  
CSS Animations ..... 94、95  
CSS Transforms ..... 93、94、95  
CSS Transitions ..... 94、95  
Custom Elements ..... 73、108

## [D]

Dart ..... 41  
DeviceOrientation Event ..... 128  
DRM (Digital Rights Management、デジタル著作権管理)  
..... 65、66、120、121

## [E]

ECMAScript ..... 61、63、151  
Ember.js ..... 38  
Encrypted Media Extensions (EME) ..... 66、120  
Extensible Web ..... 72

## [F]

Fetch API ..... 150、151、152  
File API: Directories and System ..... 66、67、126、137  
Form validation API ..... 65  
Fullscreen API ..... 102、103

## [G]

GATT (Generic Attribute Profile) ..... 155、156  
Geolocation API ..... 126、127、158  
Google Chrome Apps ..... 76  
Google Cloud Messaging (GCM) ..... 153

## [H]

Haxe ..... 41  
HTML5とは ..... 9  
HTML5 Web Messaging ..... 146  
HTTP/2 ..... 14、28

HTTPアダプティブストリーミング ..... 117、118、120  
 HTTP Live Streaming (HLS) ..... 117

## [I]

iframe ..... 146、158  
 In-Vehicle Infotainment (IVI) ..... 18、79  
 Indexed Database API ..... 60、67、125、126、137

## [J]

JSON ..... 15、24、137、152

## [K]

KnockoutJS ..... 38

## [M]

Media Capture and Streams ... 130、131、132、134、158  
 Media Queries ..... 92、93  
 Media Source Extensions (MSE) ..... 65、66、118、120  
 Microsoft Silverlight ..... 8、10、59  
 Mixed Content ..... 158、159  
 Monaca ..... 13、51  
 M-V-whatever(MVW) ..... 26  
 MV\* ..... 26、32、35  
 MVCフレームワーク ..... 53、80  
 MVCモデル ..... 25、26

## [N]

NaCl (Native Client) ..... 78

## [O]

ORTC (Object Real-Time Communications) ..... 155

## [P]

PhoneGap Build ..... 13、50、51  
 PNaCl (Portable Native Client) ..... 78  
 Pointer Event ..... 108、133  
 Polyfill ..... 97  
 Progress Event ..... 137  
 Promise ..... 151  
 Proximity Event ..... 130  
 Push API ..... 153

## [R]

React ..... 38  
 requestAnimationFrame ..... 115、116  
 REST ..... 22、23、34  
 RESTful ..... 22、23、28  
 RTP (Realtime Transport Protocol) ..... 117  
 RWD (Responsive Web Design) ..... 58、92、93

## [S]

SDP (Session Description Protocol) ..... 155  
 Secure Contexts ..... 158、159  
 Selection API ..... 99  
 Server-Sent Events ..... 142、143  
 Service Workers ..... 149、150、151、153、159  
 Session history API ..... 29、30  
 Shadow DOM ..... 73

Single Board Computer (SBC) ..... 18、134  
 Single Page Application (SPA)  
     ..... 14、15、27、28、29、34、35  
 SPDY ..... 13、14、28  
 SVG ..... 112、113

**[T]**

Timing control for script-based animations ..... 115  
 three.js ..... 94、115  
 Touch Events ..... 133  
 TypeScript ..... 41

**[U]**

UIWebView ..... 47  
 Underscore.js ..... 32、34

**[W]**

W3C Screen Orientation API ..... 103  
 W3C Pointer Lock ..... 104、105  
 W3C Gamepad ..... 105、106  
 W3C Vibration API ..... 106  
 WAI-ARIA ..... 86、87  
 WebAssembly ..... 78、79  
 Web Audio API ..... 131  
 Web Bluetooth ..... 155、156  
 Web Cryptography API ..... 67  
 WebGL ..... 66、112、114、115  
 WebKit ..... 74、125  
 Web MIDI API ..... 132、133

Web NFC API ..... 157  
 Web Notifications ..... 146、158  
 WebRTC ..... 120、121、131、145、154、155  
 WebSocket ..... 13、28、29、143、144、145  
 WebSocket API ..... 143  
 Web SQL Database ..... 67、125  
 Web Storage ..... 67、125  
 WebView ..... 47、48、50、74、76  
 Web Workers ..... 131、134、135、150  
 WHATWG ..... 71、72、102  
 WKWebView ..... 47

**[X]**

XHTML ..... 63  
 XHTML1.0 ..... 10、88  
 XMLHttpRequest ..... 21、22、24、26、136、137、138、139、  
 140、151、152、158、159

**[あ行]**

アクセシビリティ ..... 86、87  
 アプリカン ..... 13、52、54  
 アプリマーケット ..... 46、47、48、54

**[か行]**

クロスサイトスクリプティング ..... 138

**[さ行]**

スナップショットモデル ..... 70  
 ストリーミング ..... 65、117、118、120

セマンティクス…………… 85、86、87、88、89、90

## 【た行】

ダイアログボックス…………… 97

同一オリジンポリシー…………… 138、159、159

ドラッグ&ドロップ…………… 107

## 【な行】

ネイティブアプリ…………… 12、44、45、46、47、48、49、55

## 【は行】

ハイブリッドアプリ…………… 44、46、47、48、49、53、55

ハイブリッドアプリ開発  
…………… 12、47、49、50、51、52、53、54、74、75、76

ハイブリッドキャスト…………… 15

プッシュ通知……………45、147、150、153

フルスタックフレームワーク…………… 37

## 【ま行】

モバイルファースト…………… 12

## 【や行】

ユニバーサルアプリプラットフォーム…………… 59

## 【ら行】

リアルタイムWeb …………… 13、14、136、141

リビングスタンダード…………… 71

## 【HTML5 タグ】

article要素 …………… 85、89

aside要素 …………… 85、89

audio要素 …………… 91、116、117、130

canvas要素 …………… 91

dialog要素 …………… 97

div要素 ……………85、87、88、93

em要素 …………… 90

footer要素 …………… 85

header要素 …………… 85

nav要素 …………… 85、89

i要素 …………… 90

img要素 …………… 86、94、150

input要素 …………… 87、99、108

ruby要素 …………… 91

section要素 …………… 85、89

small要素 …………… 90

span要素 …………… 87

strong要素…………… 90

textarea要素 …………… 99

track要素 …………… 91

u要素 …………… 90

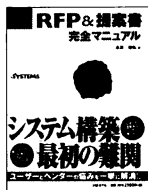
video要素 …………… 91、116、117、118、130、131

# 挑戦するITエンジニアを支援

上級

## ビジネス力をつける!

(システム企画・提案、ヒューマンスキル関連)



RFP & 提案書  
完全マニュアル



事例で学ぶ  
RFP作成術  
実践マニュアル



誰も教えてくれなかった  
システム企画・提案  
実践マニュアル



IT現場で使える!  
ロジカルシンキング



ひと目で伝わる!  
システム企画・提案の  
図解術



ITエンジニアのための  
通じる文章にする  
五つの力

## マネジメント力をつける!

(プロジェクトマネジメント、見積もり関連)



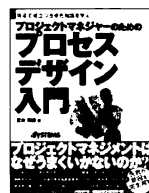
プロジェクト成功の道しるべ  
システム開発のための  
WBSの作り方



プロジェクトを成功に導く  
人を動かす調整力



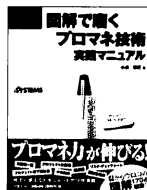
ITプロジェクトをやり遂げる  
最強リーダーシップ



現場で役立つ  
生きた知識を学ぶ  
プロジェクトマネジャーのための  
プロセスデザイン入門



ソフトウェア開発を  
成功に導く  
本当に使える  
見積もり技術  
[改訂第3版]



図解で磨く  
プロマネ技術  
実践マニュアル



失敗から学ぶ  
プロマネ技術  
36のQ&A



基礎から学ぶ  
プロマネ技術  
必携マニュアル



比較でわかる  
プロマネ技術

初級

# する 日経 SYSTEMS の本

## システム構築力をつける！

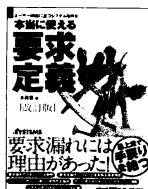
(要件定義、設計、テスト関連)



システム設計の先導者  
ITアーキテクトの  
教科書



手戻りなしの  
要件定義  
実践マニュアル[増補改訂版]



ユーザー役に立つシステムを作る  
本当に使える  
要求定義[改訂版]



システム開発をより速く確実に  
本当に使える  
開発プロセス



なぜ重大な問題を見逃すのか  
間違えだらけの  
設計レビュー



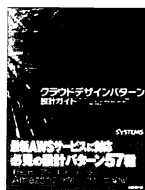
ステップアップのための  
ソフトウェアテスト  
実践ガイド[改訂版]

## 技術力をつける！

(製品、テクノロジー関連)



Amazon Web Services  
定番業務システム12パターン  
設計ガイド



Amazon Web Services  
クラウドデザインパターン  
設計ガイド[改訂版]



Amazon Web Services  
クラウドデザインパターン  
実装ガイド[改訂版]



要件定義から役立つ必須知識  
基本がわかる  
安全設計のWebシステム



アーキテクトからHTML5まで  
Webシステム入門



## 著者紹介

---

羽田野 太巳（はたの ふとみ）

futomi 代表取締役社長、ニューフォリア取締役 最高技術責任者。Web システム開発および Web コンサルティングに携わる。HTML5 の気運が高まる以前から HTML5 の探求を始め、HTML5 専門サイト「HTML5jp」を立ち上げ、HTML5 の普及啓蒙に関わる。HTML5 関連書籍や雑誌記事執筆も行う。現在は、Web ベースのアプリケーション開発やデジタルサイネージ・システムの研究開発の指揮を執る



# アーキテクチャーからHTML5まで Webシステム入門

---

2016 年 7 月 15 日 第 1 版第 1 刷発行

著 者 羽田野 太巳

発 行 人 寺山 正一

発 行 日経 BP 社

発 売 日経 BP マーケティング

〒 108-8646

東京都港区白金 1-17-3

カバーデザイン 葉波 高人（ハナデザイン）

デザイン・制作 ハナデザイン

印刷・製本 図書印刷

© 2016 Futomi Hatano

ISBN 978-4-8222-3776-9

---

●本書の無断複写・複製（コピー等）は著作権法上の例外を除き、禁じられています。購入者以外の第三者による電子データ化及び電子書籍化は、私的使用を含め一切認められておりません。